SIYEENOVE robot arm kit For micro:bit

MicroPython Tutorial

2026.3.17

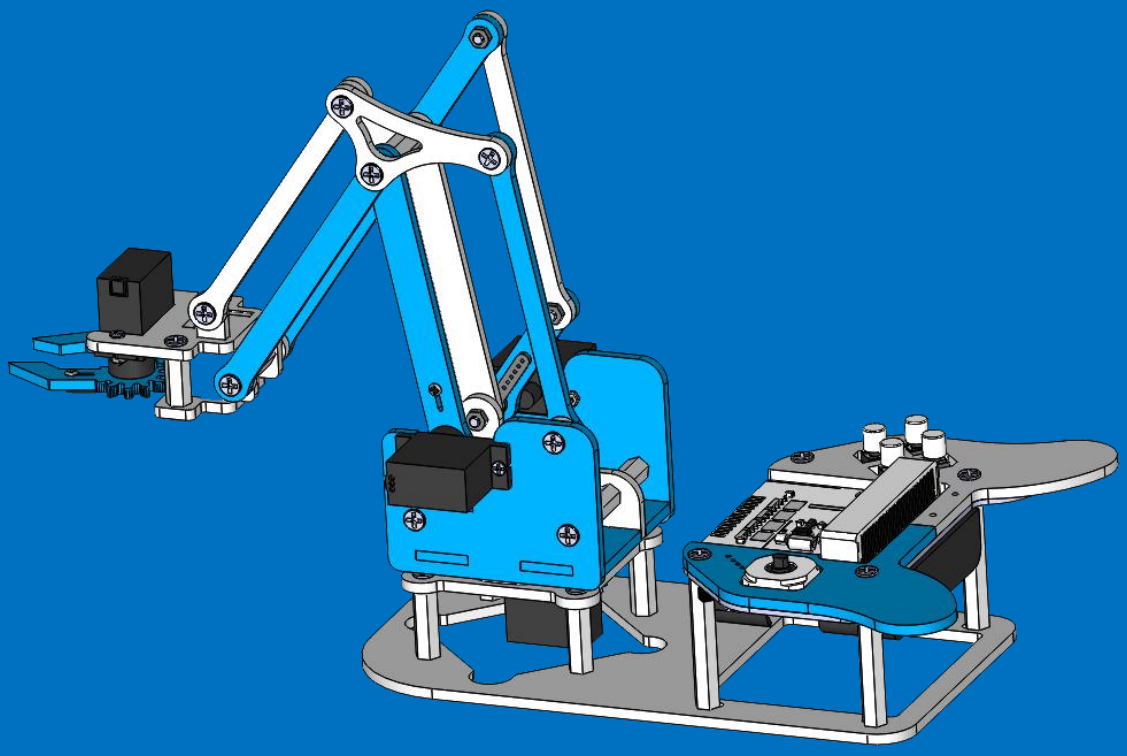
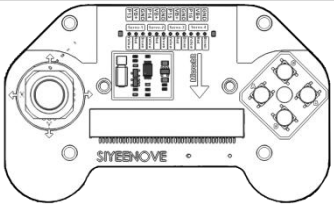
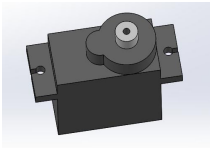


Table of Contents

1. Introduction to Robot Arm

The SIYEENOVE Micro:bit robotic arm is an educational and programmable robotic arm designed to teach students and hobbyists the fundamentals of robotics, coding, and automation. Powered by the BBC Micro:bit microcontroller, this robotic arm provides a hands-on learning experience in STEM (Science, Technology, Engineering, and Mathematics) fields.

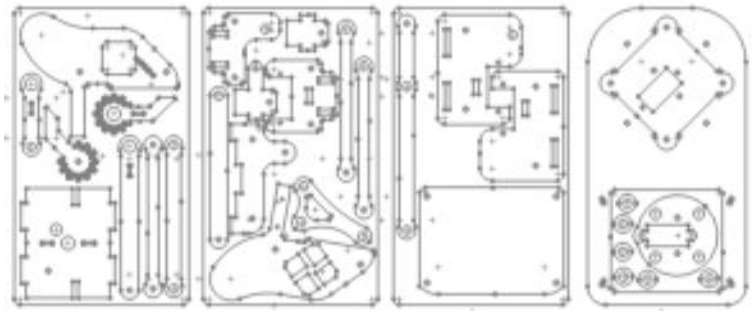
1.1 Package Contents

#Part 1 : Electronics Components		
Joystick 1PCS	Servo 4PCS	3P wire 1PCS
		

#Part 2 : Fasteners / Accessories The following accessories listed in sequential order are all packaged in a small bag with a label and serial number attached. You can quickly locate them by referring to the serial number and name on the bag.		
1 M3×12mm Screw 6Pcs 	2 M3×9mm Screw 43Pcs 	3 M2×8mm Screw 10Pcs 
4 M1.4×5mm Screw 7Pcs 	5 M3 Nut 12Pcs 	6 M2 Nut 10Pcs 
7 M3×14mm Standoff 3Pcs 	8 M3×17+6mm Standoff 2Pcs 	9 M3×26mm Standoff 2Pcs 
10 M3×18mm Standoff 4Pcs	11 M3×20mm Standoff 5Pcs	12 M3×38mm Standoff 4Pcs

		
⑬ 5×3.2×3mm Spacer 8Pcs 	⑭ 5×3.2×6mm Spacer 6Pcs 	⑮ Spiral Conduit 
⑯ Steel Ball 4Pcs 	⑰ Button Cop 4Pcs 	

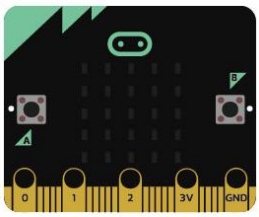
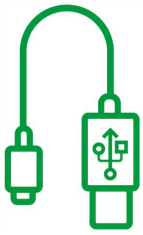
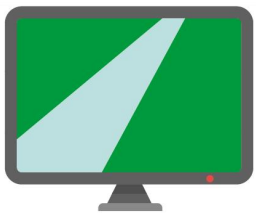
#Part 3 : Acrylic Sheet



#Part 4 : Tools

		
M3 screwdriver 1PCS	M1.5 screwdriver 1PCS	wrench 1PCS

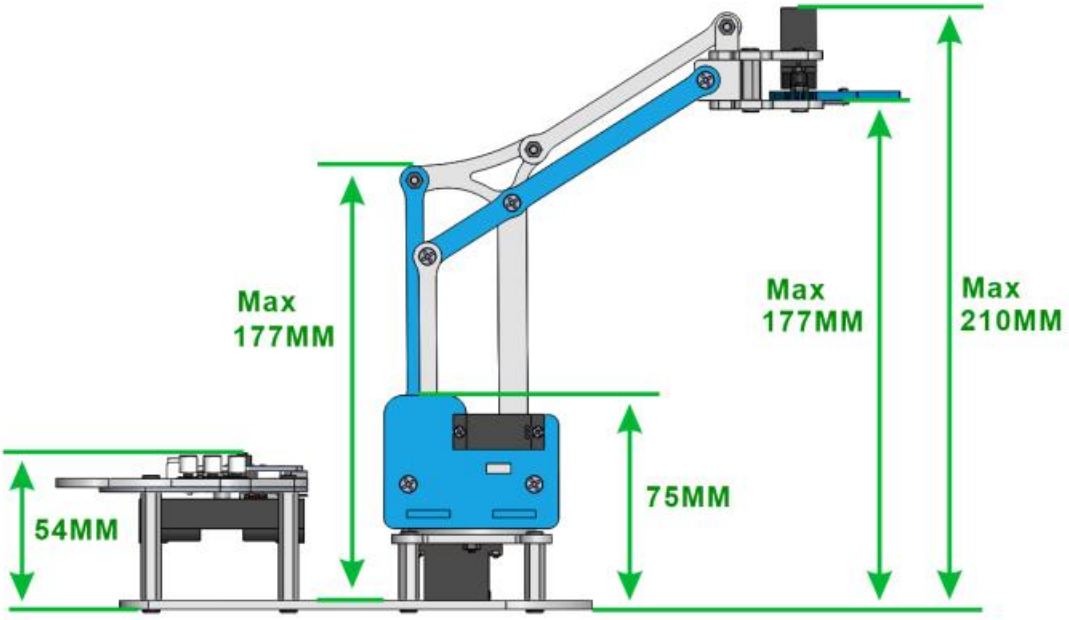
What you'll need (not included)

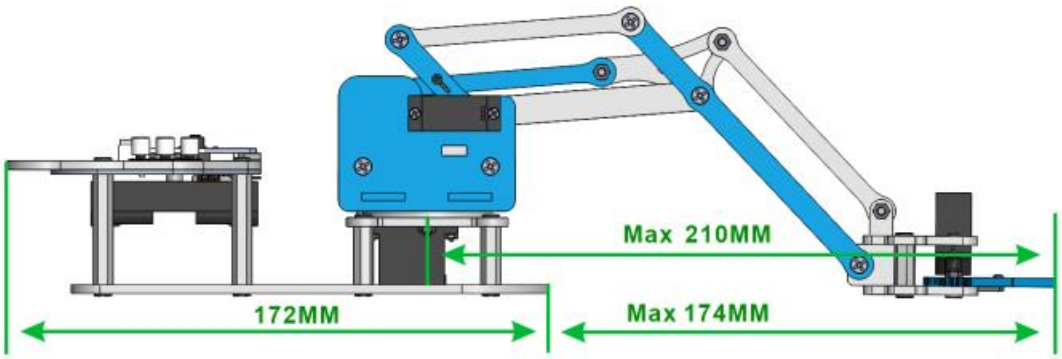
			
Micro:bit V2 1PCS	Micro USB Cable	computer with internet & a USB port	1.5V AA Battery 4 pcs

1.2 Specification

Control Method: SIYEENOVE Joystick (micro:bit not included)
Programming Method: MakeCode
Required Battery Type: 4 × AA Batteries
Servo Type: MG90S 180°
The maximum lifting capacity: ≤60g

1.3 Product Dimensions





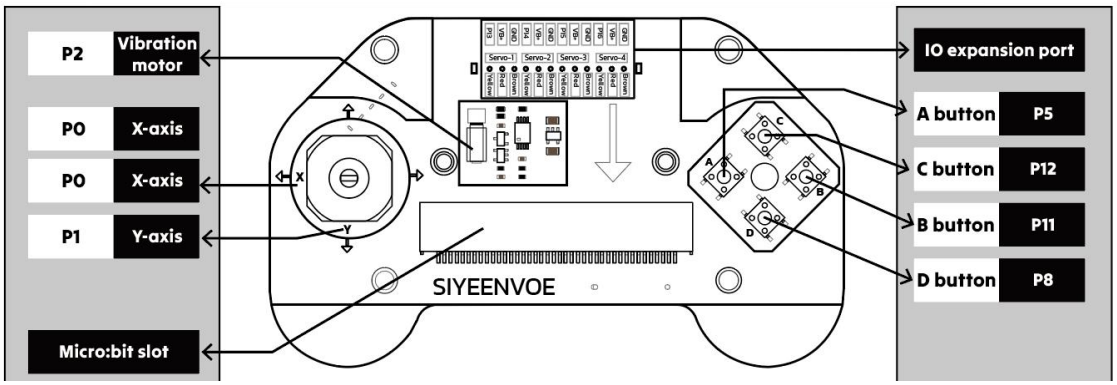
1.4 SIYEENOVE Joystick Introduction

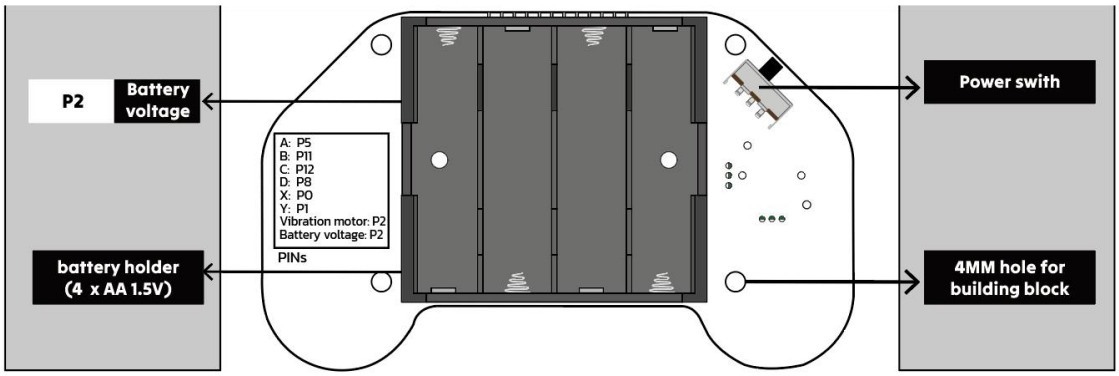
The SIYEENOVE Joystick Controller is a manual input device used in robotics, gaming, and education applications. Featuring dual-axis (X/Y-axis) movement and integrated Button, it enables high-precision directional control, making it ideal for operating robotic arms, drones, and other motion-based systems.

1.4.1 Joystick Specification

Shield	
Name	mJoystick
SKU	M1E0001
USB connector	
Name	Micro USB
Applicable motherboard	
Name	Micro:bit
Pins	
Digital I/O Pins	4 (P13, P14, P15, P16)
Servo pins	4 (P13, P14, P15, P16)
Communication	
SPI	Yes (P13, P14, P15, P16)
Power	
Input voltage (nominal)	6V (4 AA batteries, 1.5V/PCS)
VB	6V (4 AA batteries, 1.5V/PCS)
Vibration motor	
Model	3610 Vibration motor
Rated voltage/current	2.7V/75mA
Rotation speed	14000±2500RPM
Dimensions	
Width	72.5 mm
Length	120 mm
Height	29mm
Weight	50 g

1.4.2 Interface specification





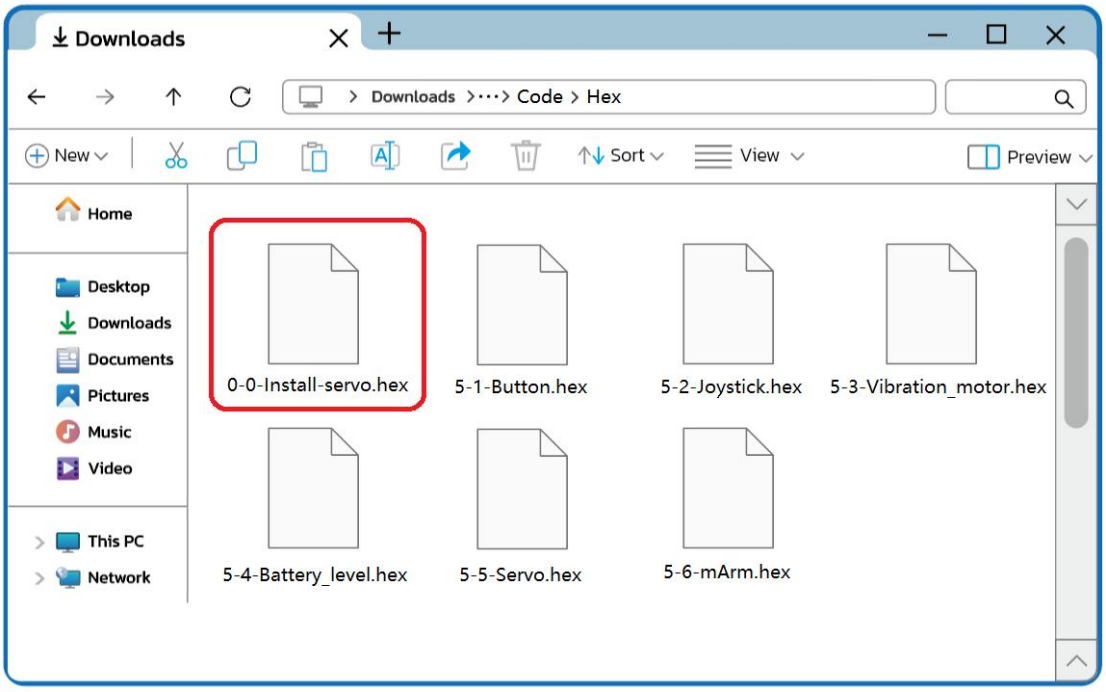
Name	Description
Vibration motor	Provides haptic feedback to the handle, controlled by Micro:bit's
X-axis	The joystick's X-axis, whose analog value can be read through Micro:bit's P0 pin
Y-axis	The joystick's Y-axis, whose analog value can be read through Micro:bit's P1 pin
Micro:bit slot	Designed for inserting the Micro:bit main control board
4 x AA 1.5V battery case	The battery case accommodates 4 batteries, each with standard 1.5V voltage, approximately 14.5mm in diameter and 50.5mm in height
Battery voltage	When 4 AA batteries are installed, the voltage can be read through Micro:bit's P2 pin
IO extension	For connecting external servos or sensors. The VB pin voltage equals the series voltage of 4 AA batteries (1.5V×4 = 6V)
A button	Its value can be read through Micro:bit's P5 pin
B button	Its value can be read through Micro:bit's P12 pin
C button	Its value can be read through Micro:bit's P11 pin
D button	Its value can be read through Micro:bit's P8 pin
Power switch	It controls the main power supply of the handle.
4MM hole	The handle features 4×4mm holes compatible with LEGO

2. Assembly

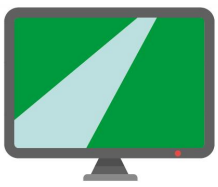
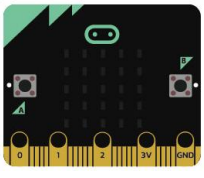
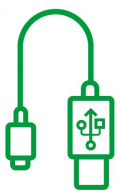
2.1 Assembly Preparation: Initialize the servo angle (adjust to 90 degrees).

> Note!
> The microbit-0-0-Install-servo.hex code must be uploaded before assembly because the servos require angle initialization (Set to 90 degrees). If installed without proper servo initialization, the robotic arm will fail to execute preset programs and tasks. In severe cases, power-on activation may cause servo jamming, potentially damaging the servos.

The initialization code is stored in the "**Code>Hex**" folder within the downloaded tutorial package, as shown in the figure below:

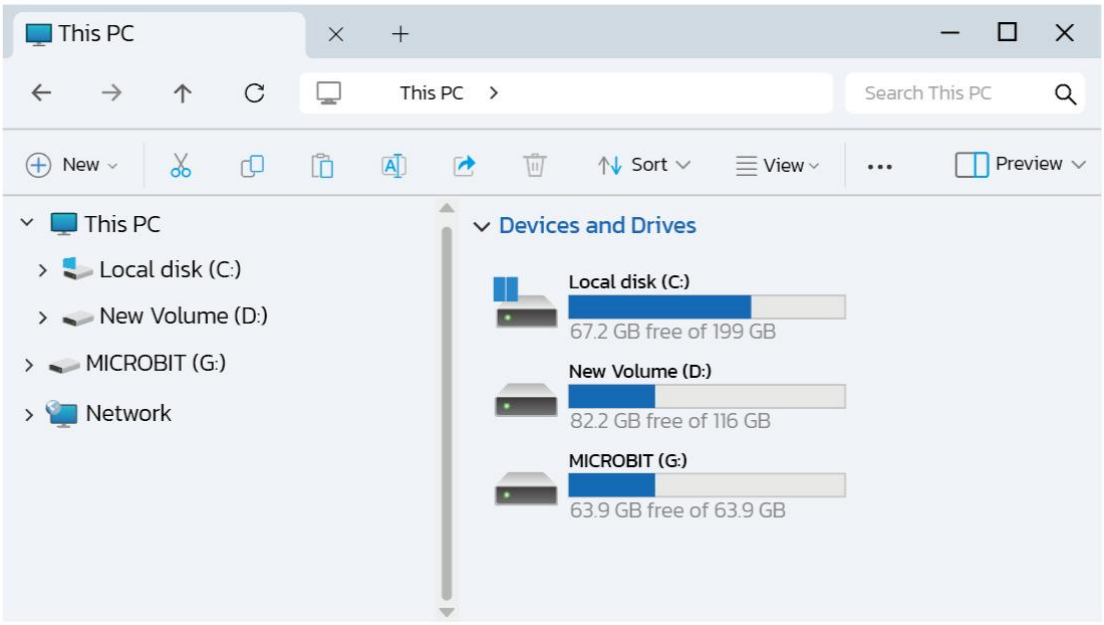


Tools Required:

PC	Micro:bit v2.x.x	Micro USB cable
		

Steps:

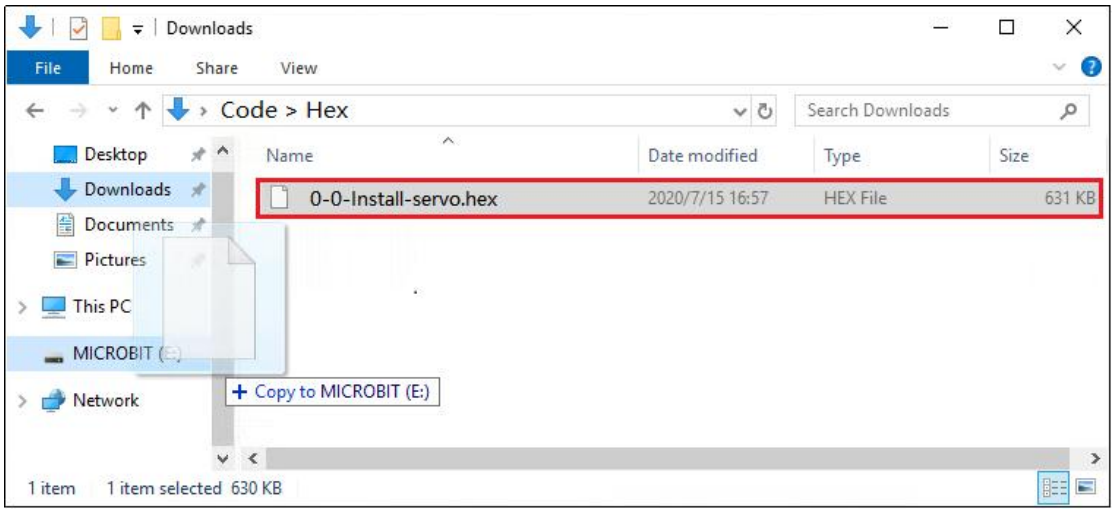
1. Connect the micro:bit to your PC using the Micro USB cable. The PC will detect the micro:bit as a removable drive (e.g., "MICROBIT").



2. Locate the HEX file (microbit-0-0-Install-servo). Use one of the following two methods to quickly upload code to the micro:bit.

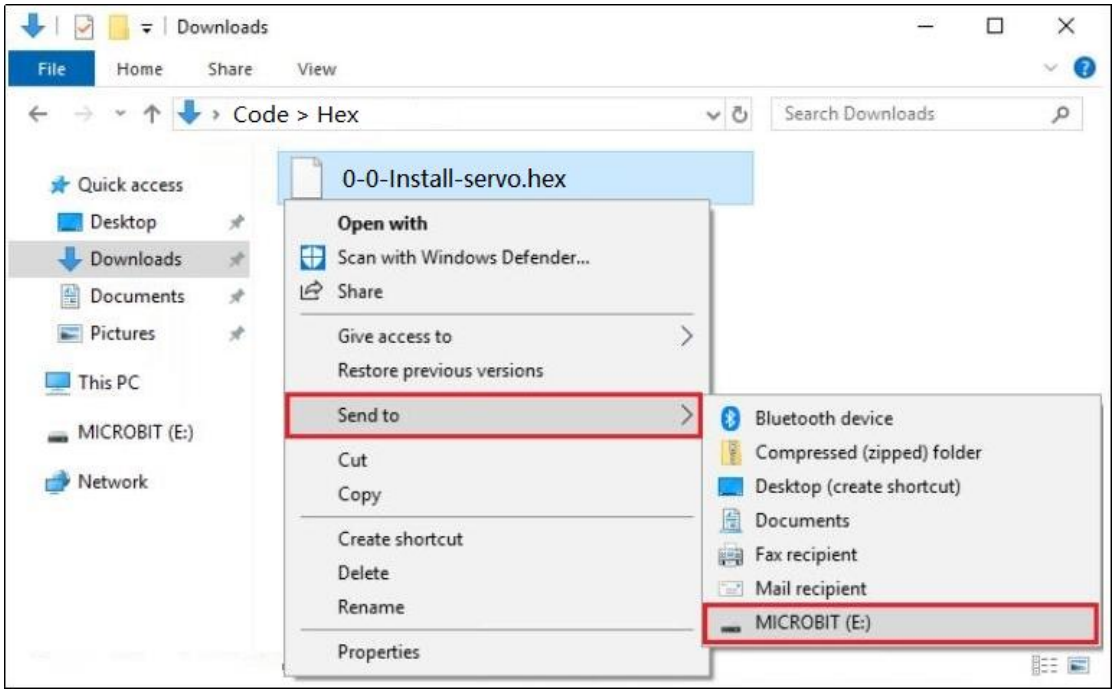
Method 1 (Drag & Drop):

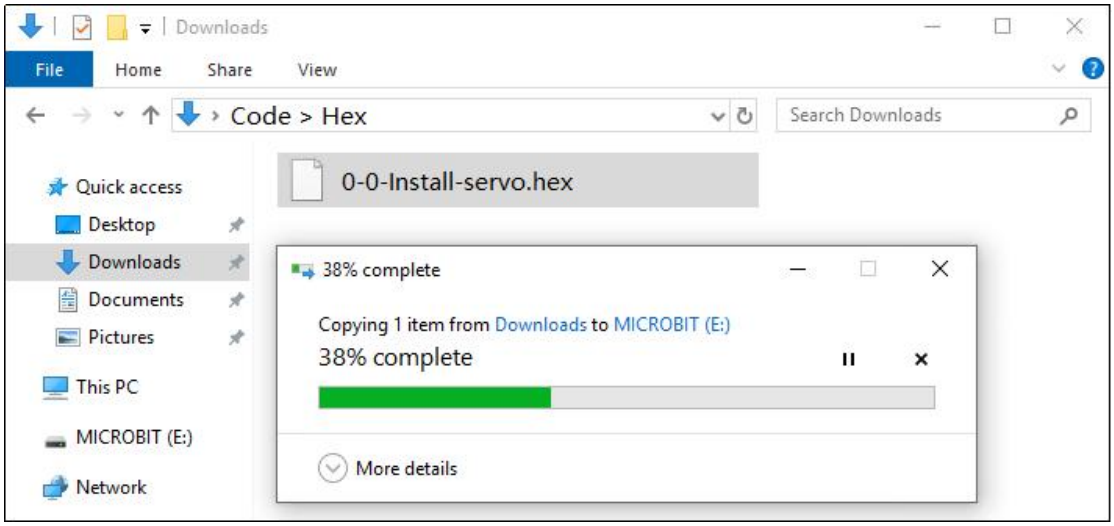
Simply drag and drop the HEX file onto the MICROBIT drive.



Method 2 (Send to):

Right-click the HEX file and select "Send to" → MICROBIT drive.





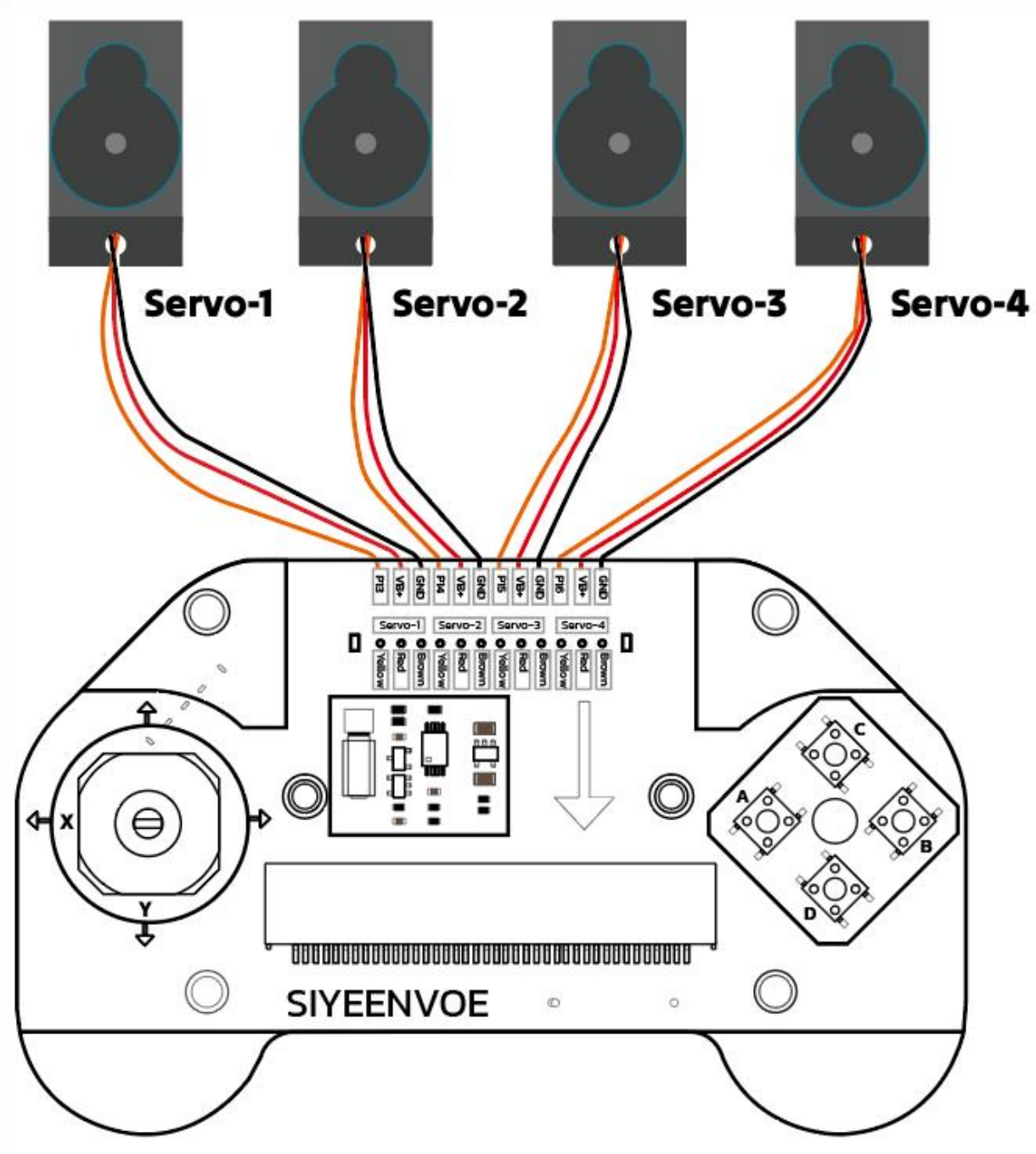
Attention：

Make sure your Micro:bit is properly connected before uploading.

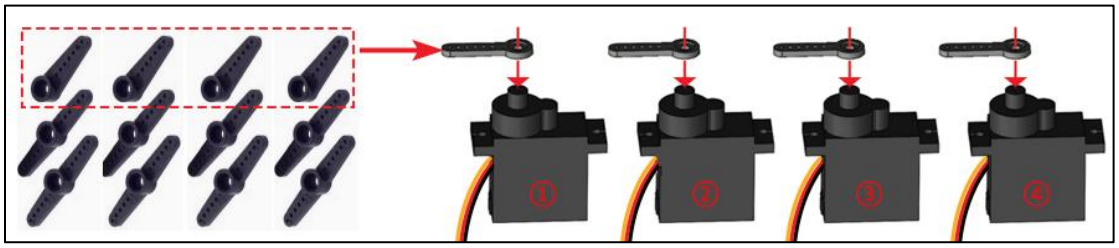
Do not disconnect the device while the firmware is flashing.

3. Connecting the Servo to the Joystick:

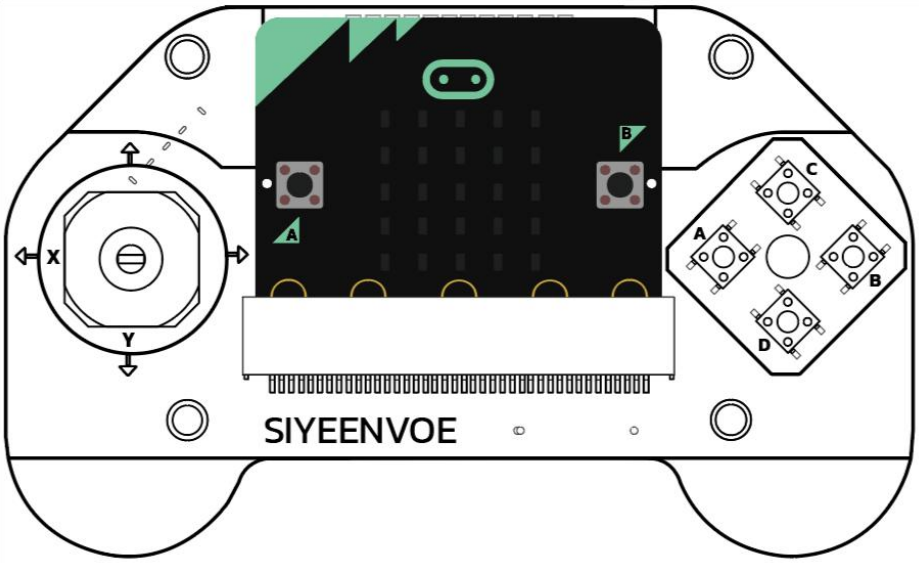
Servo-1			Servo-2			Servo-3			Servo-4		
P13	VB+	GND	P14	VB+	GND	P15	VB+	GND	P16	VB+	GND



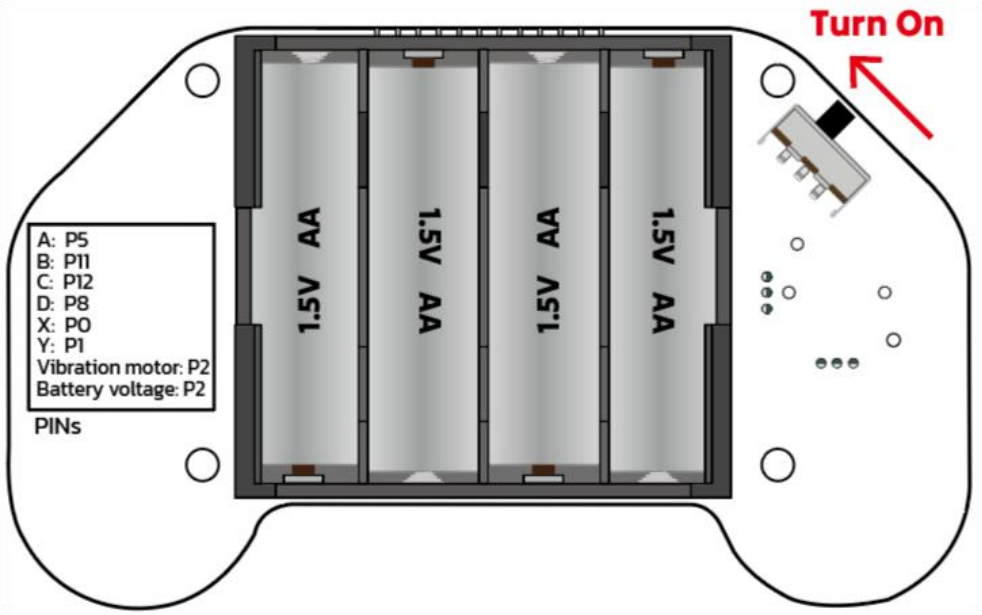
4. For easy observation of the servo shaft's rotation and its ability to stop at the initialized position, we recommend attaching a temporary servo horn (arm) to the servo shaft without screw fixation during this process.



5. Insert the micro:bit to the joystick



6.Insert 4*AA batteries and turn on the power switch



7.Result:

1) **Battery Level Indication:**

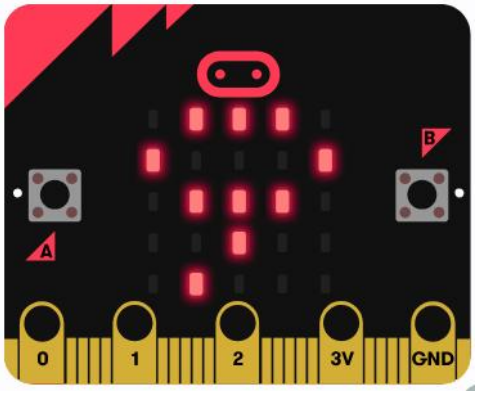
After powering on, the Micro:bit's LED matrix will continuously cycle through displaying the battery level (0%–100%).

Important: When the battery level drops below 30%, replace all batteries immediately.

If no battery level is displayed:

Check if the Micro:bit is fully inserted into the handle

Verify correct battery installation



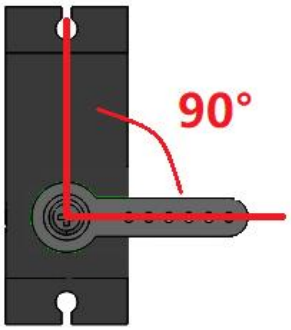
2) Servo Initialization Sequence:

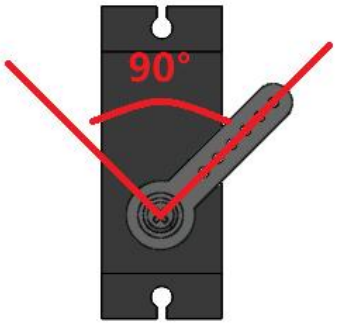
First Power-On:

Allow 5 seconds to pass, all 4 servos will rotate from random positions → 0° → slowly to 90° (holding at 90°)

On subsequent power-ups:

Allow 5 seconds to pass, all servos will move from 90° → 0° → slowly back to 90° (holding position)



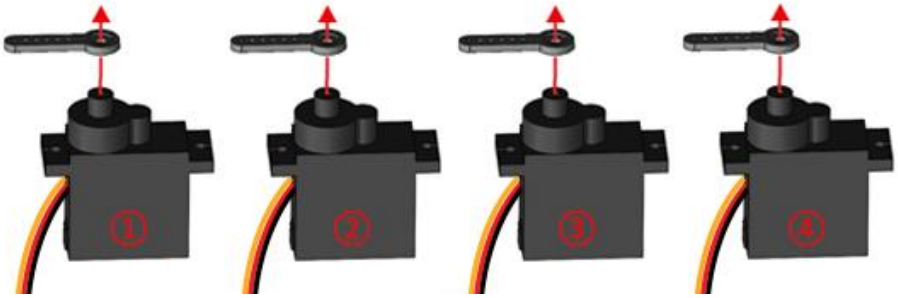


....

If servos fail to rotate:

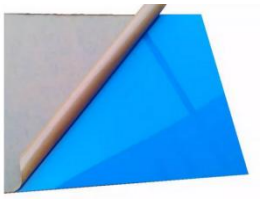
- Inspect servo wiring connections
- Re-upload the code if wiring is correct but servos remain unresponsive
- Clean Micro:bit's edge connectors and reinsert firmly
- Contact support if issues persist (possible servo defect)

8. After the servo angle initialization is completed, gently remove the servo horn from each servo for subsequent installation. Don't rotate the servo shaft to avoid damage gear trains or disrupt calibrated positions.



2.2 Precautions before robot assembly

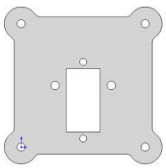
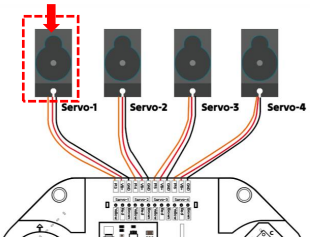
1) Remove protective film before assembly! Both top and bottom surfaces are coated with protective film

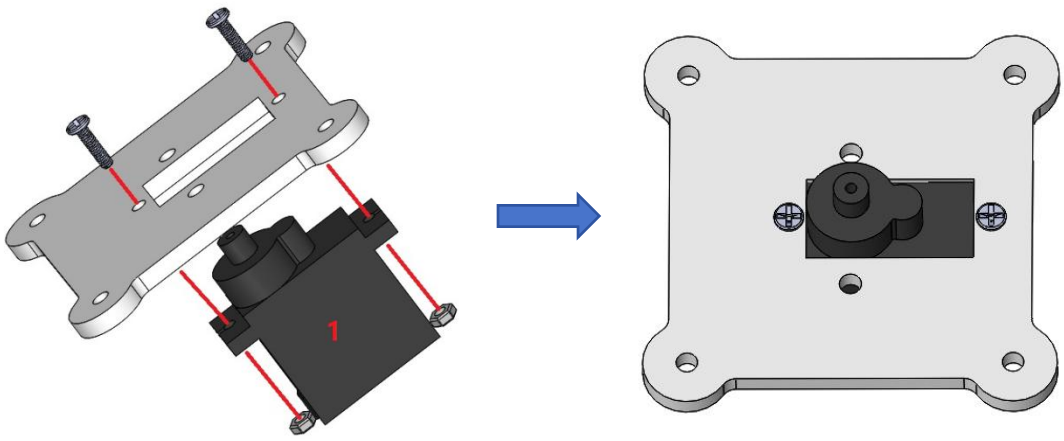
 and 

2) Do not empty all parts from their bags at once to avoid mixing and confusion. Open part bags sequentially according to the instructions in the manual to prevent mixing small components. Some screws may be included as spares.
3) Assemble facing same direction as diagrams; colors of components may vary.
4)If any parts are missing or damaged, do not assemble or use the product. Please contact us for after-sales support.

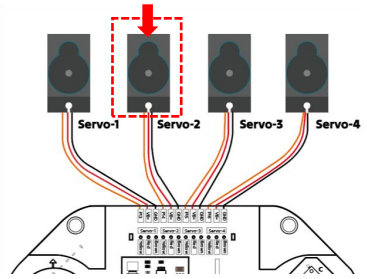
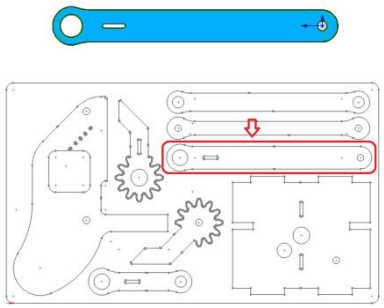
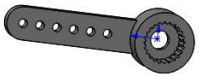
2.3 Start Assembly

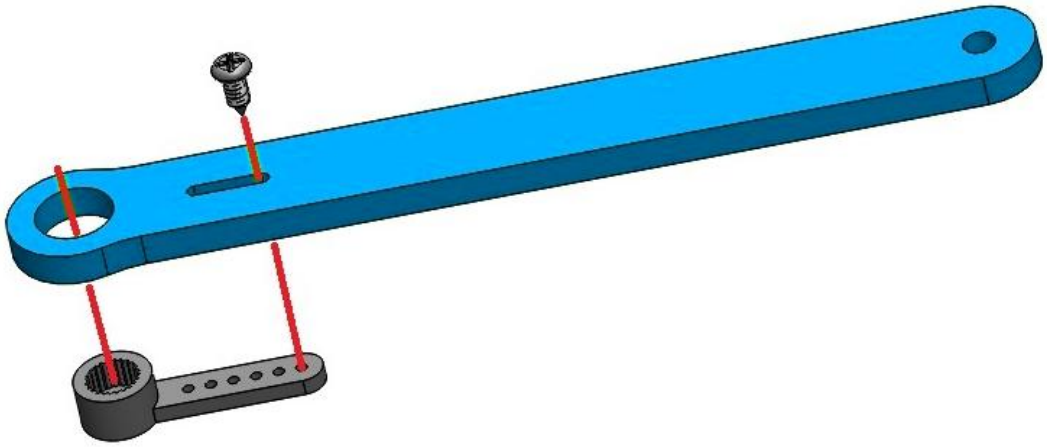
Step 1 - Assembly the Servo-1

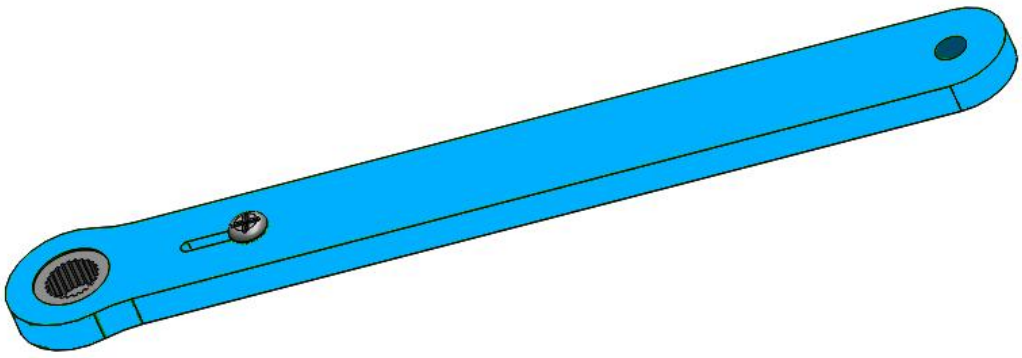
Acrylic board 1PCS	Servo-1	**Bag No.③** M2x8mm screw 2PCS	**Bag No.⑥** M2 nut 2PCS
			

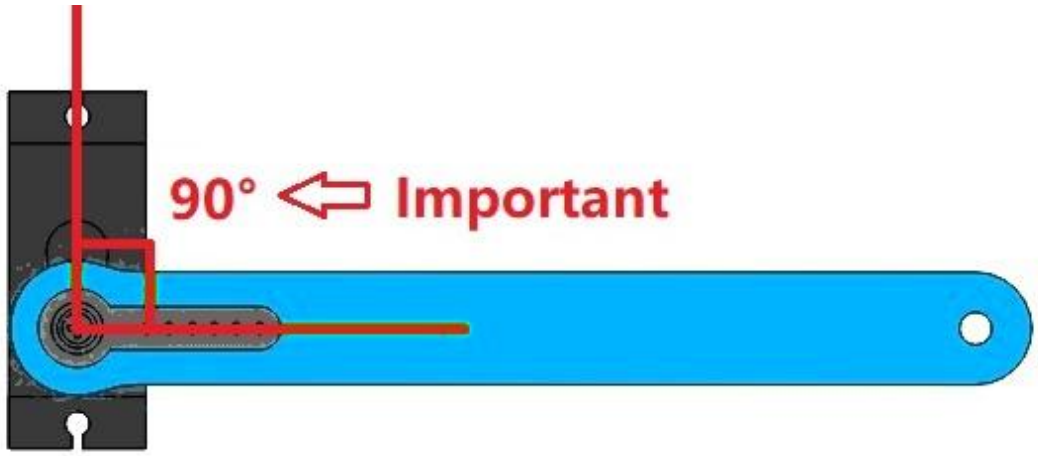


Step 2 - Assembly the Servo-2

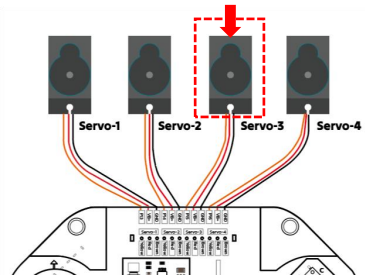
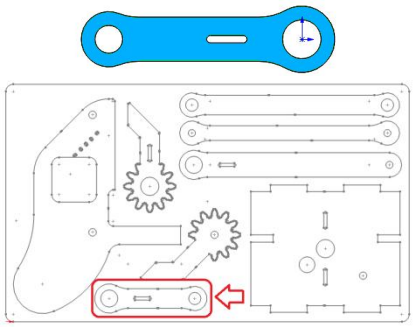
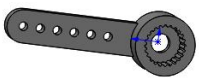
Servo-2	Acrylic board 1PCS
	
Servo horn 1PCS	**Bag No.④** **M1.4X5 Self-tapping screw 1PCS**	**M2.5X4mm screw 1PCS**
		

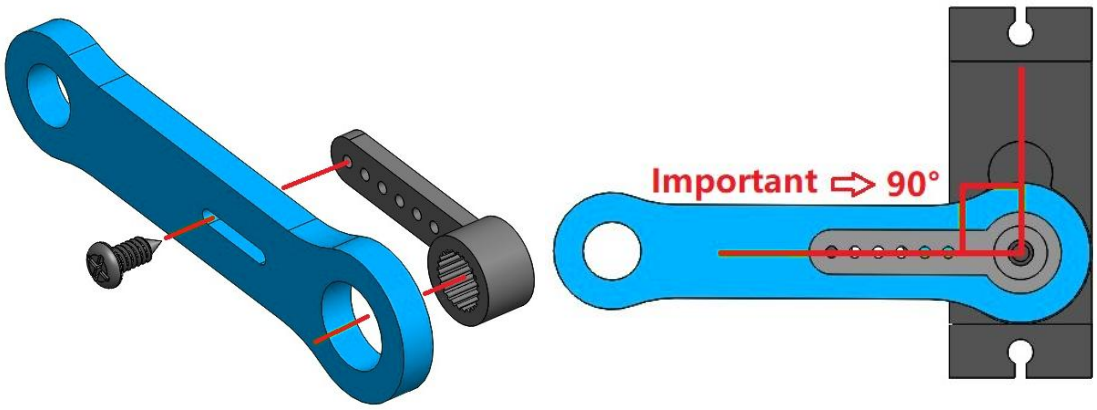


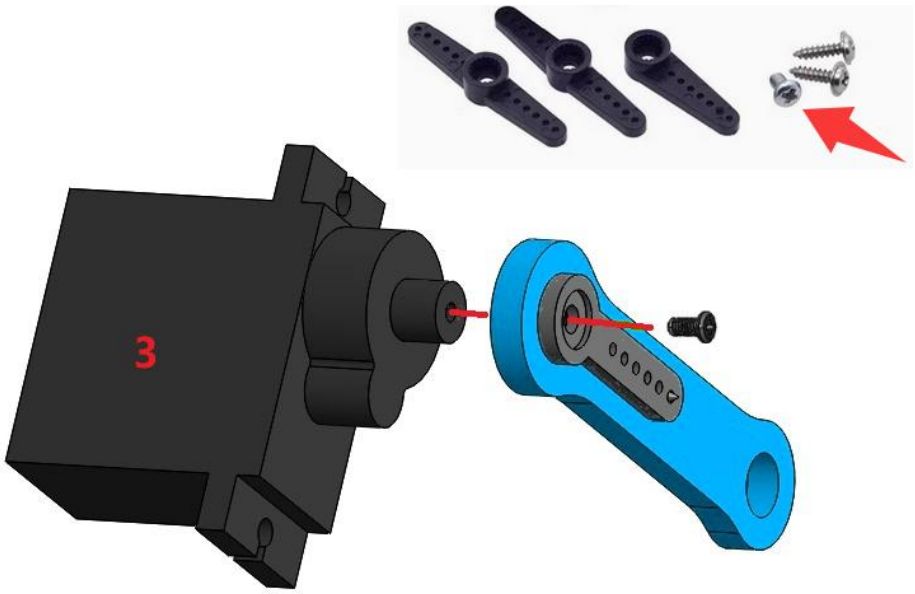


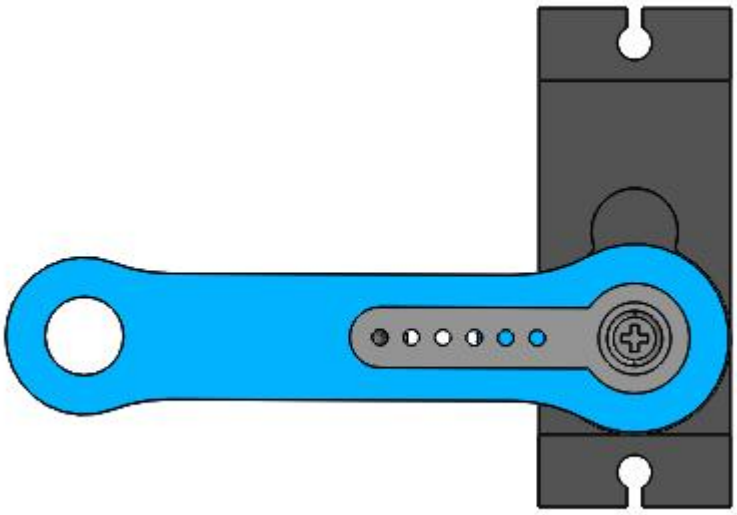


Step 3 - Assembly the Servo-3

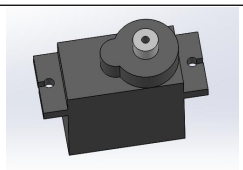
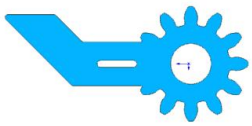
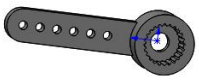
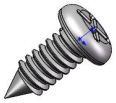
Servo-3	Acrylic board 1PCS
	
Servo horn 1PCS	**Bag No.④** M1.4X5 self-tapping screw 1PCS	M2.5X4mm screw 1PCS
		

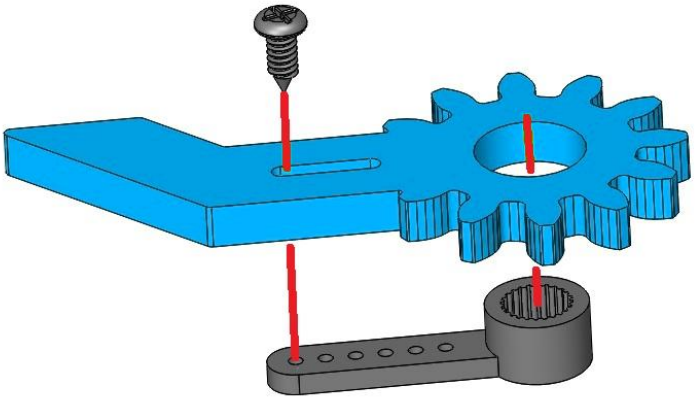

Important ⇨ 90°

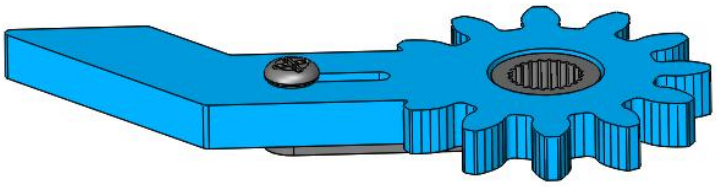

3

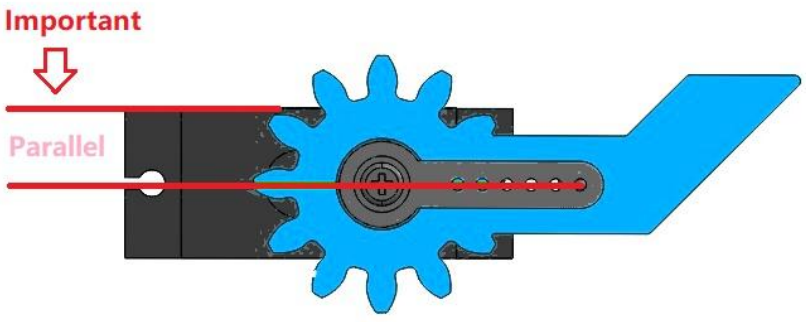


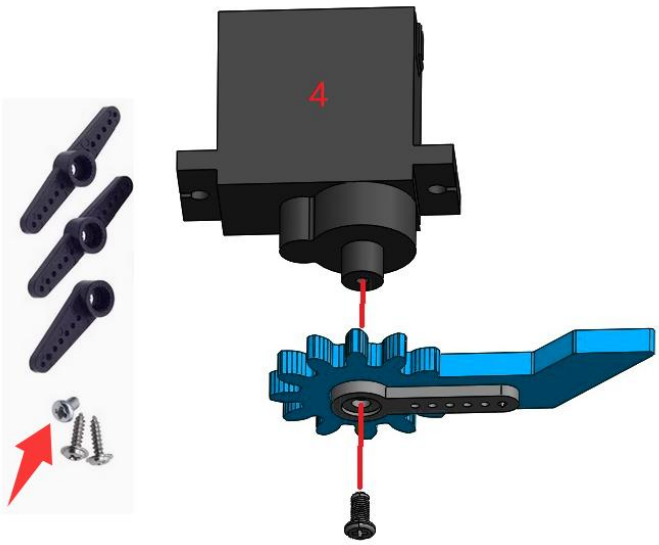
Step 4 - Assembly the Servo-4

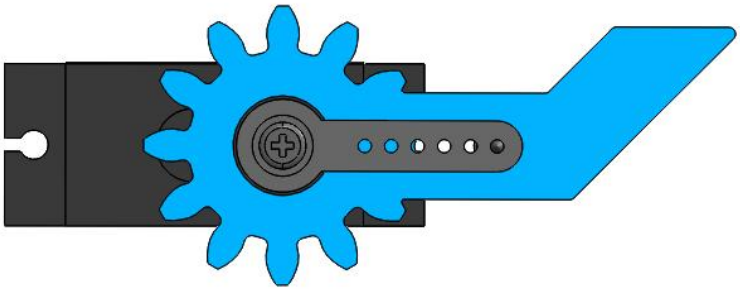
Servo-4		Acrylic board 1PCS
		
Servo horn 1PCS	**Bag No.④** M1.4X5 self-tapping screw 1PCS	M2.5X4mm screw 1PCS
		





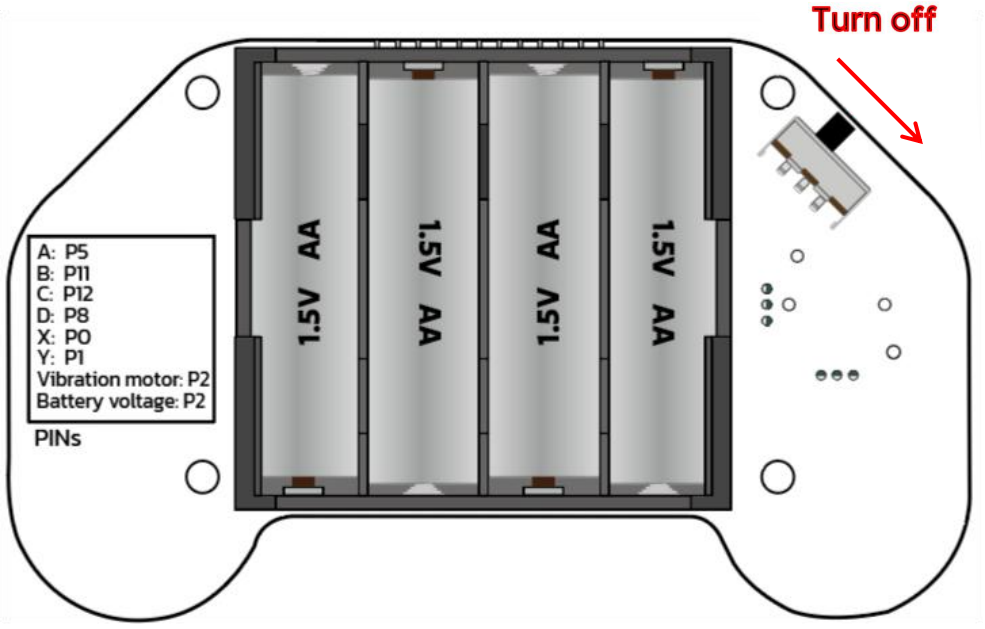




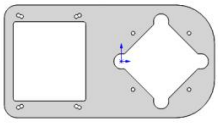
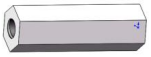


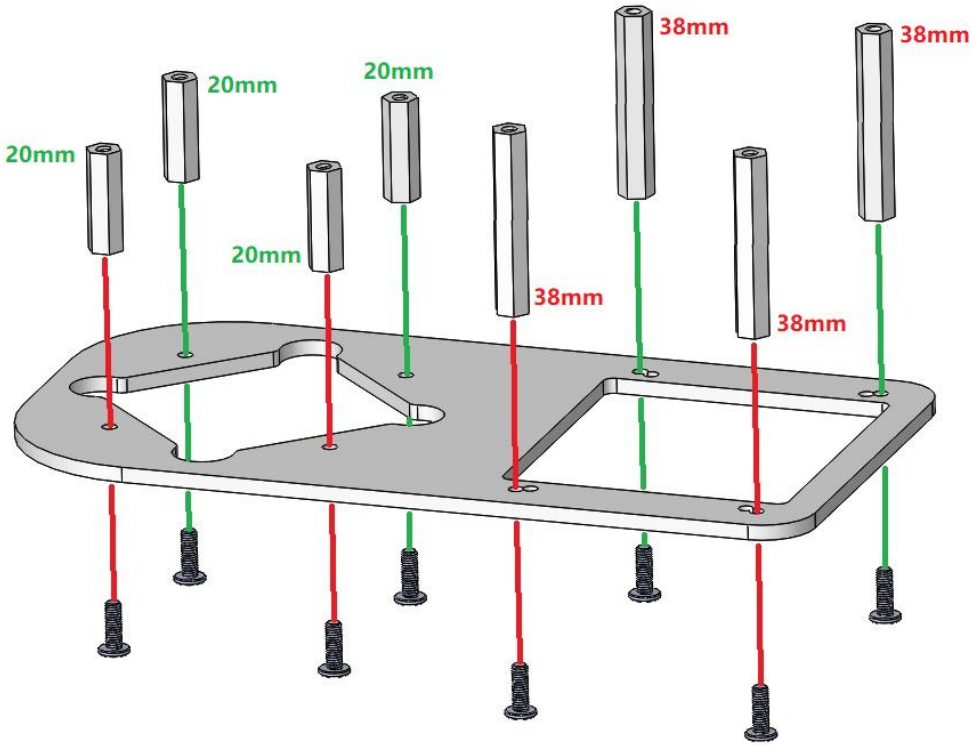
Step 5 - Power off the Joystick and disconnect servos

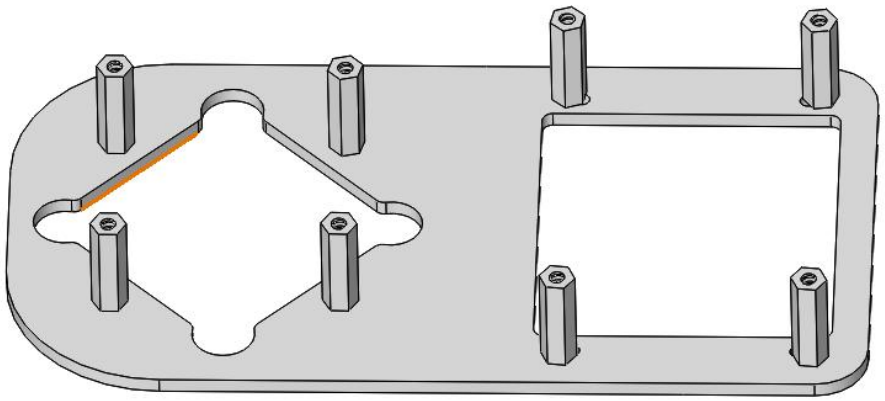
To prepare for the next installation steps: Power off the Joystick and unplug all servo cables from its ports.



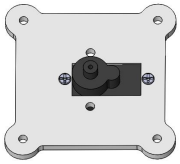
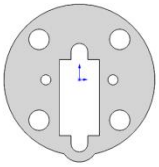
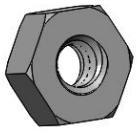
Step 6 - Assembly Base (Nylon Standoffs)

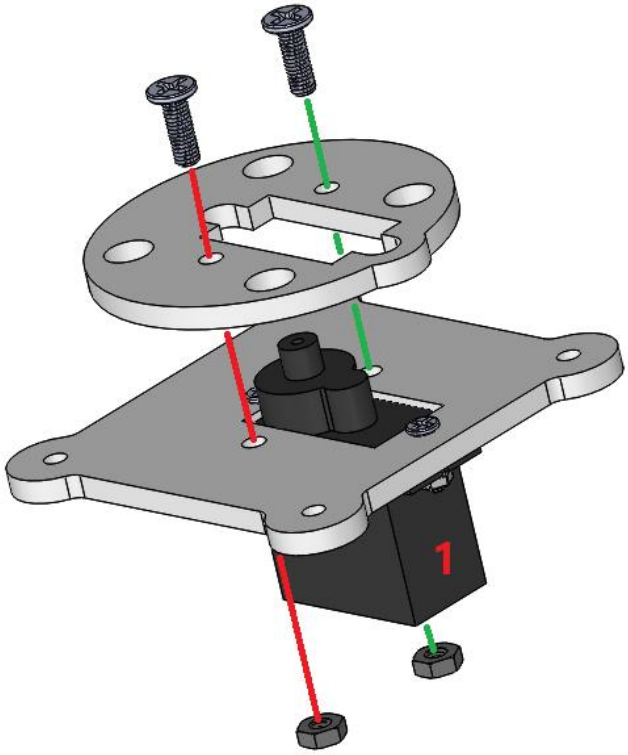
Acrylic board	**Bag No.⑪** M3x20MM Nylon Standoff 4PCS	**Bag No.⑫** M3x38MM Nylon Standoff 4PCS	**Bag No.②** M3X9MMscrew8 PCS
			

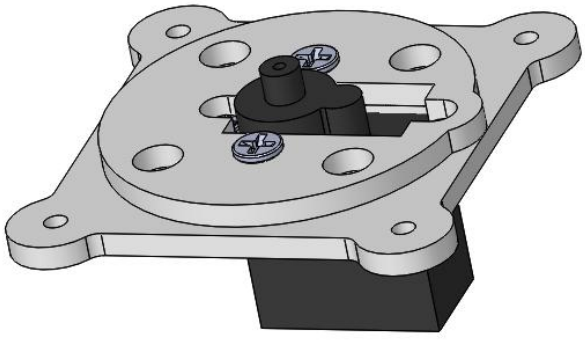




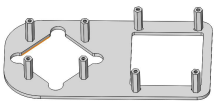
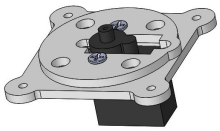
Step 7 - Assembly Base (Steel Ball Bearing Plate)

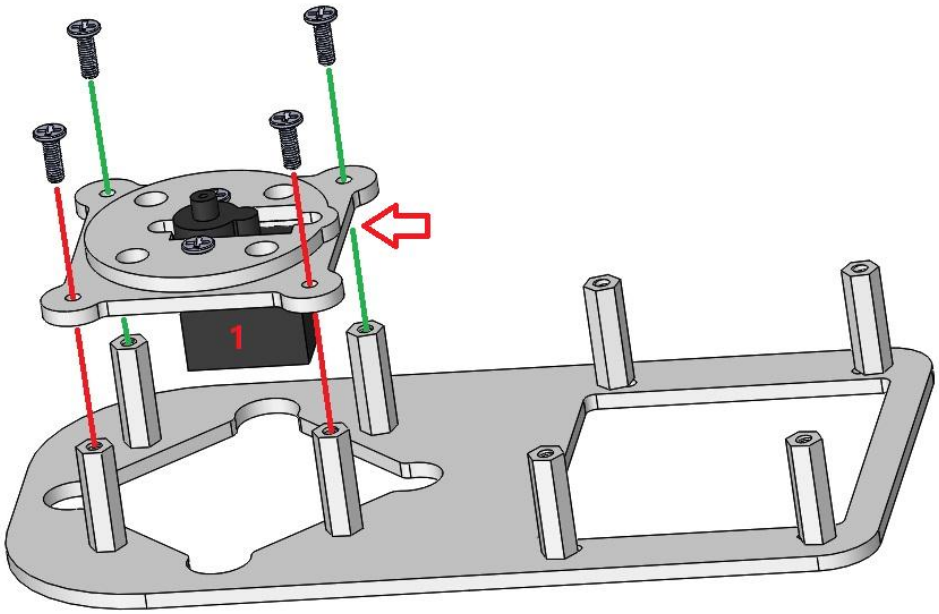
The structure in Step 1	Acrylic board	**Bag No.②** M3X9MMscrew2PCS	**Bag No.⑤** M3 nut 2 PCS
			

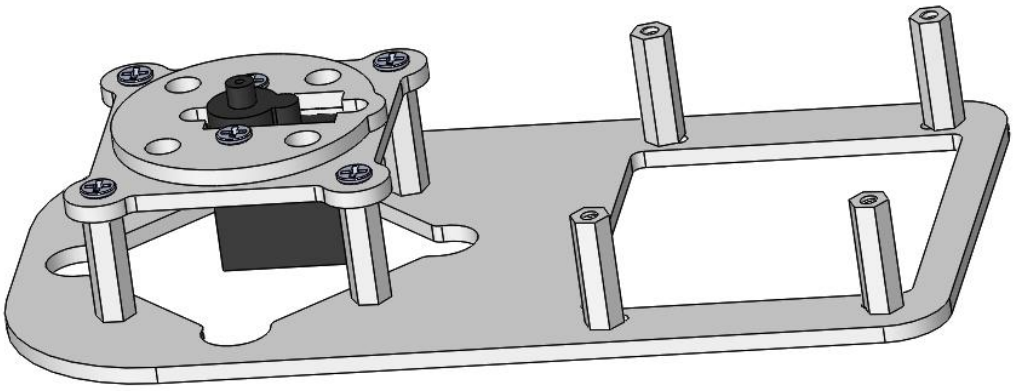




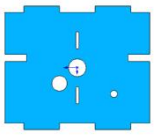
Step 8 - Assembly Base (Servo Structure)

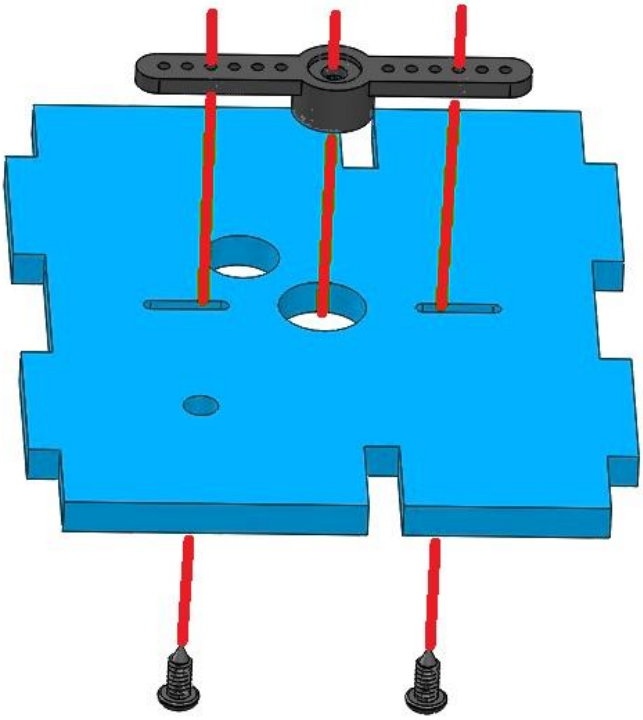
The structure in Step 6	The structure in Step 7	**Bag No.②** M3X9MMscrew4PCS
		

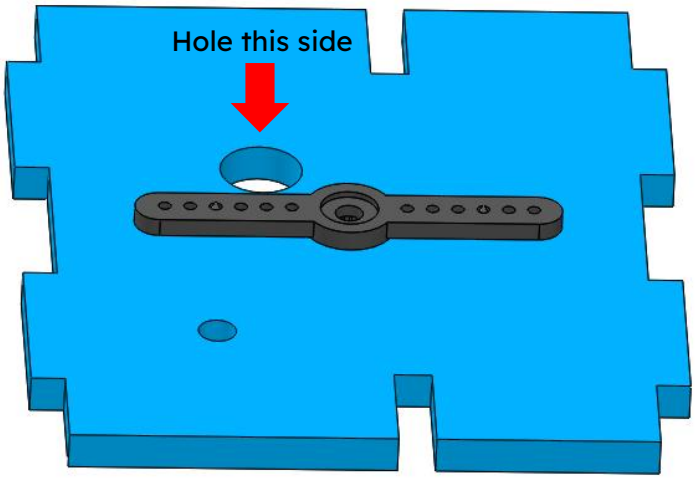




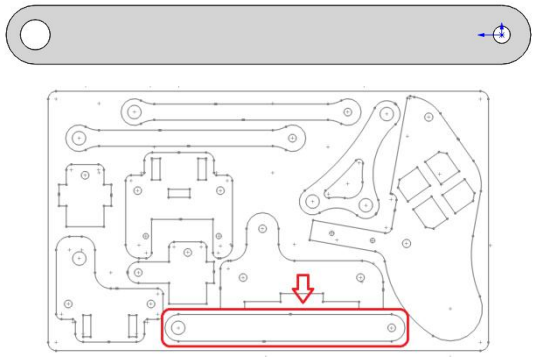
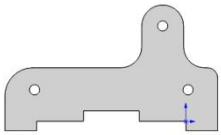
Step 9 - Attach Servo Horn to Rotating Base

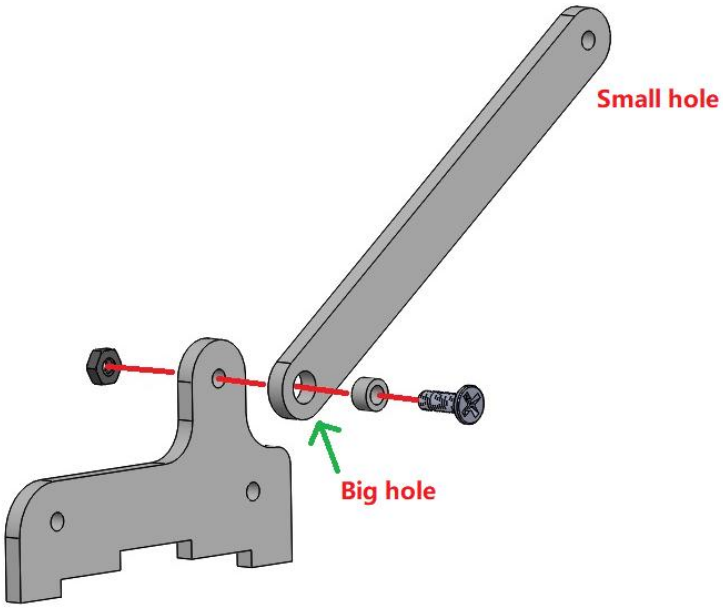
Acrylic board	Servo horn 1PCS	**Bag No.④** M1.4X5 self-tapping screw 2PCS
		

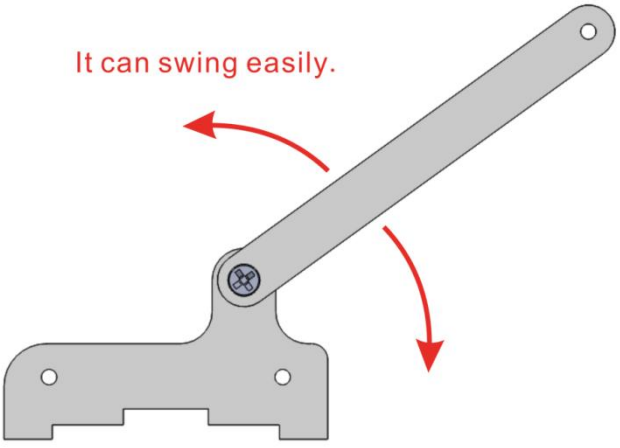




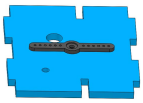
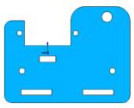
Step 10 - Upper Arm Linkage Installation

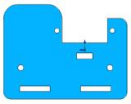
Acrylic board		Acrylic board
		
Bag No.⑬ 5x3.2x3MM spacer 1PCS	**Bag No.④** M3X9MM screw 1PCS	**Bag No.⑤** M3 nut 1PCS
		

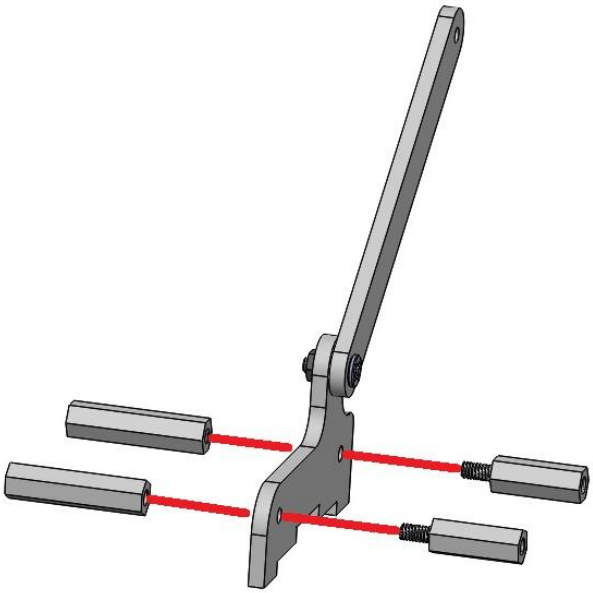


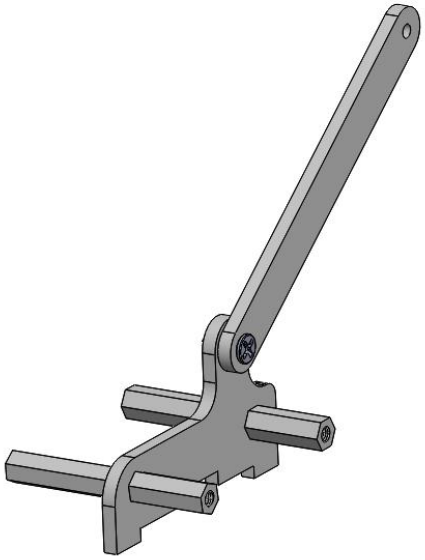


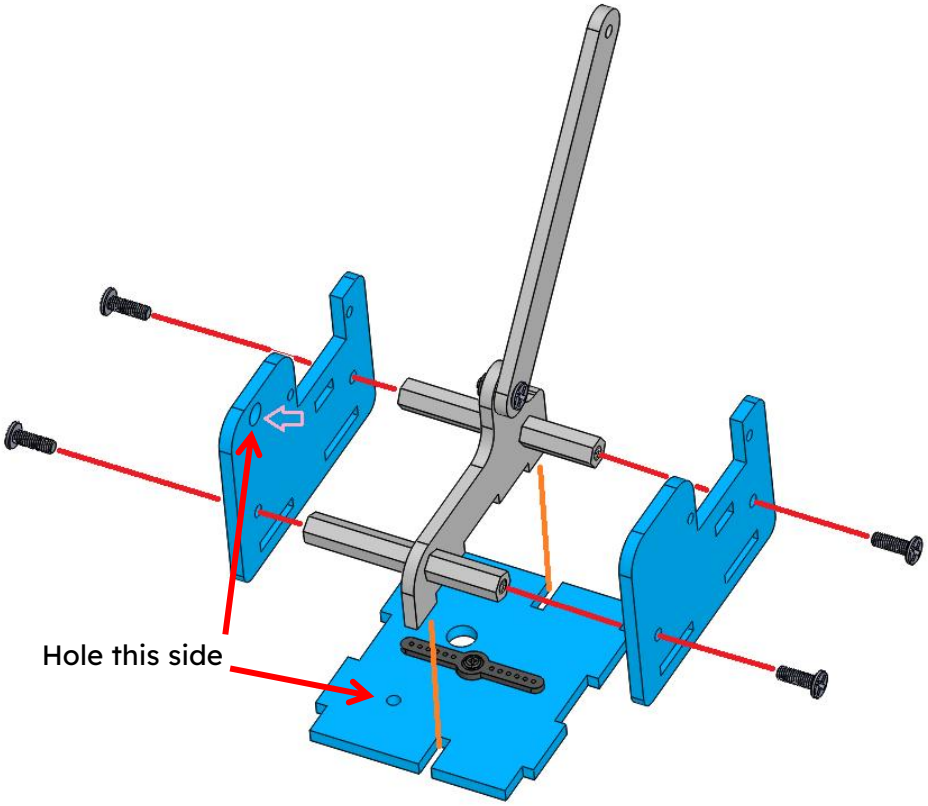
Step 11 - Assemble Rotating Base

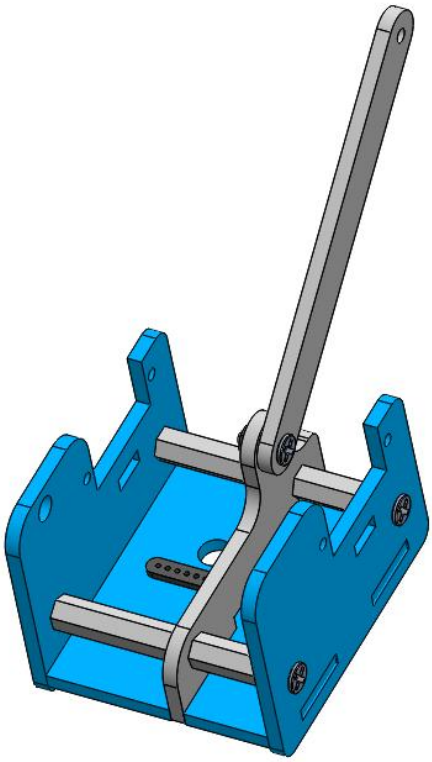
The structure in Step 9	The structure in Step 10	Acrylic board 1PCS
		

Acrylic board 1PCS	**Bag No.⑧** M3x17+6MMNylon Standoff 2PCS	**Bag No.⑨** M3x26MMNylon Standoff 2PCS	**Bag No.②** M3X9MMscrew4PCS
			

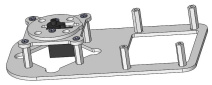


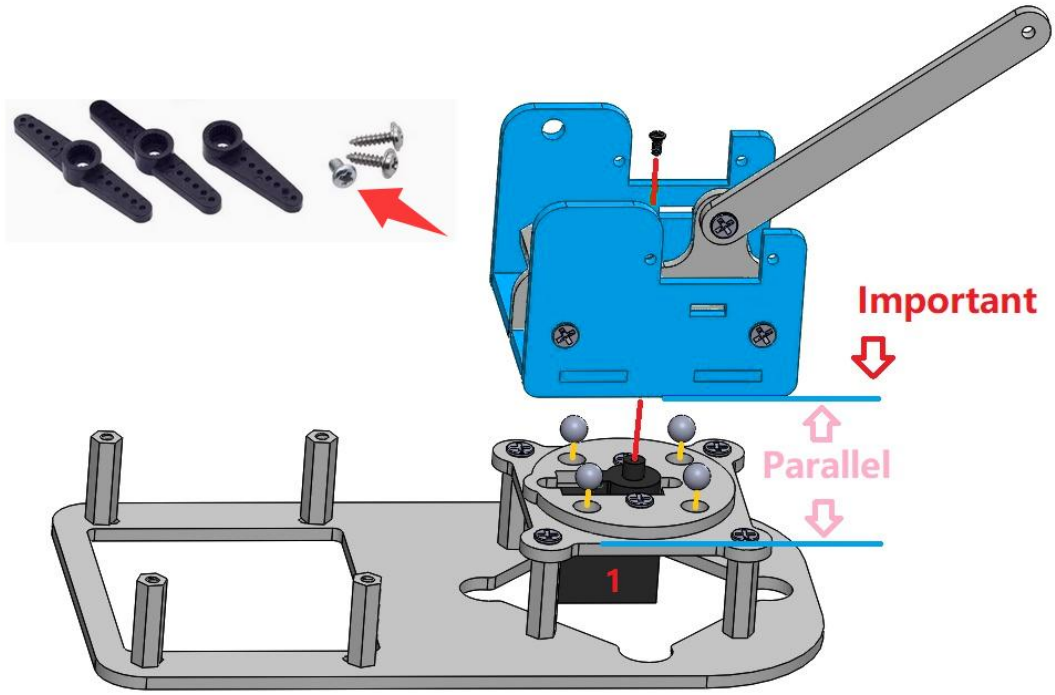


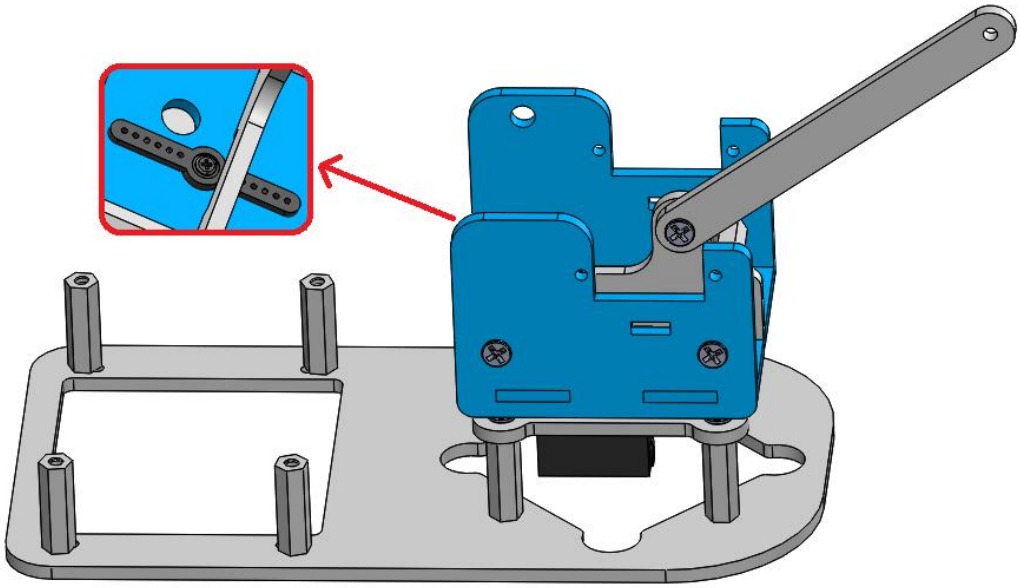

Hole this side



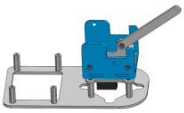
Step 12 - Mount Rotating Base to Main Frame

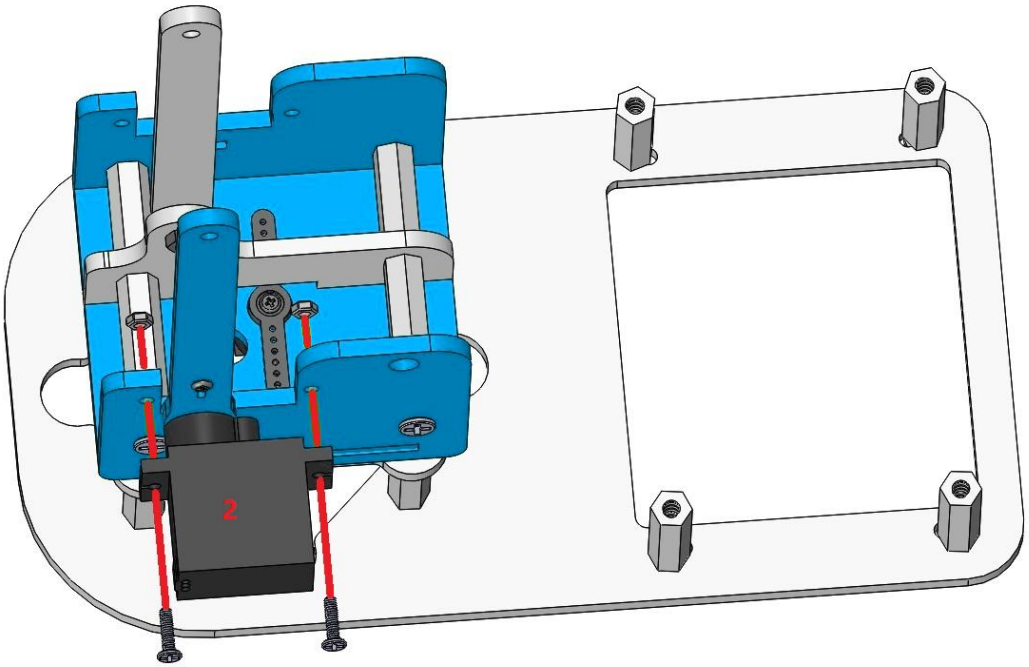
The structure in Step 8	The structure in Step 11	Steel board 4PCS	M2.5X4mm screw 1PCS
			

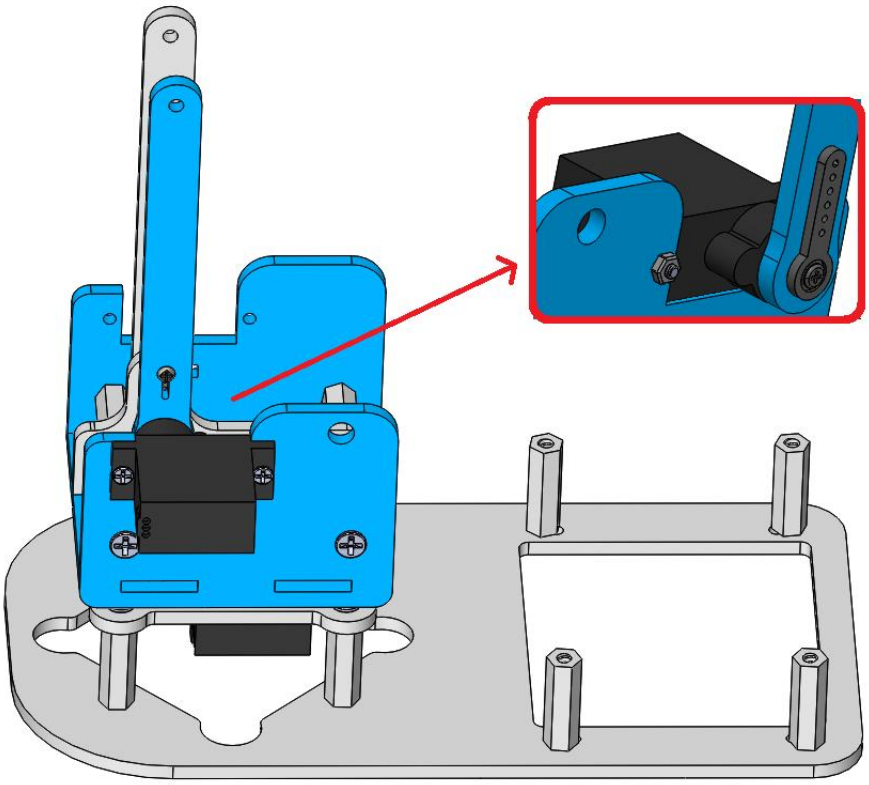




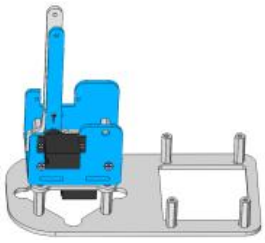
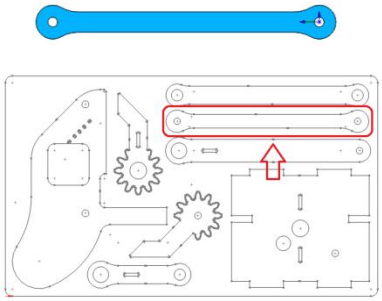
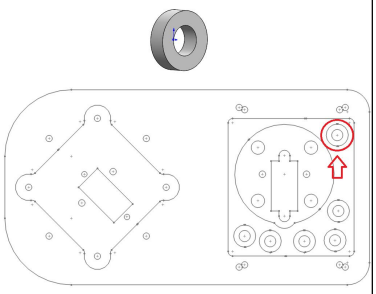
Step 13 - Assembly Left Servo Structure on Turntable

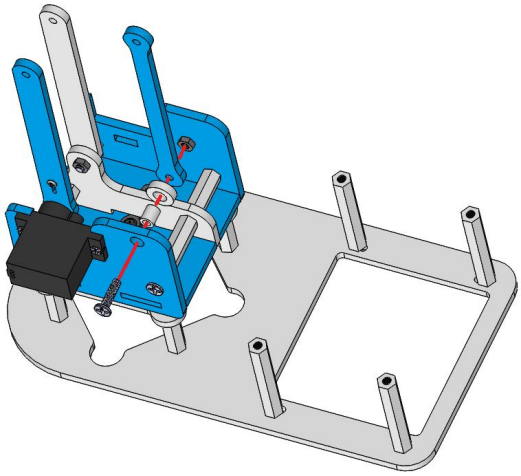
The structure in Step 2	The structure in Step 12	**Bag No.③** M2x8mm screw 2PCS	**Bag No.⑥** M2 nut 2PCS
			

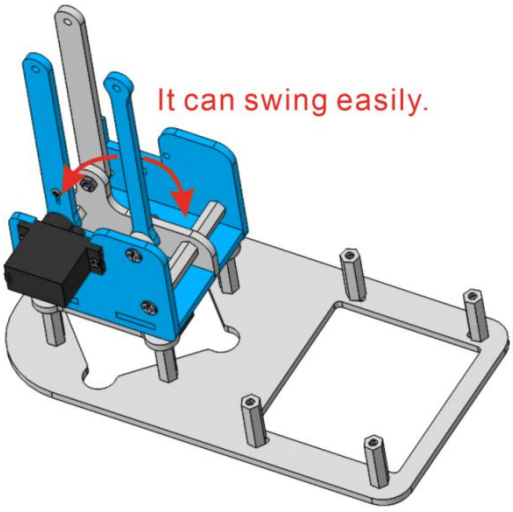




Step 14 - Attach Upper Arm Linkage (Left Side)

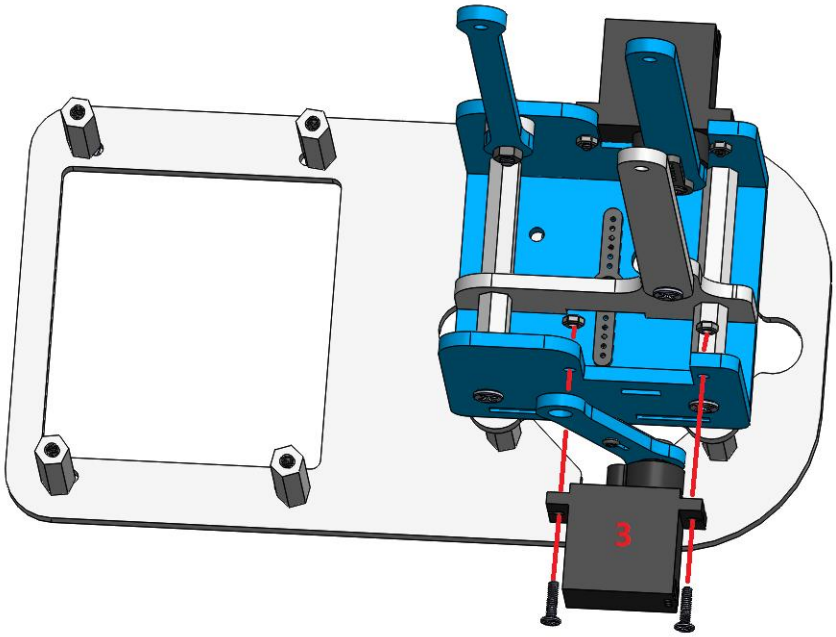
The structure in Step 13	Acrylic board 1PCS	Acrylic spacer 1PCS
		
Bag No.⑭ 5x3.2x6MM spacer 1PCS	**Bag No.①** M3X12MM screw 1PCS	**Bag No.⑤** M3 nut 1PCS
		

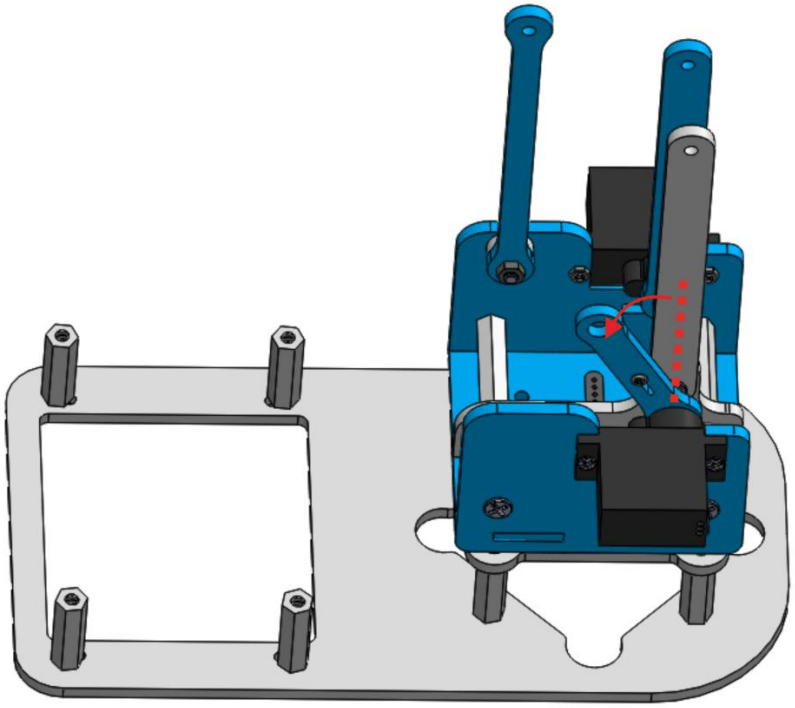




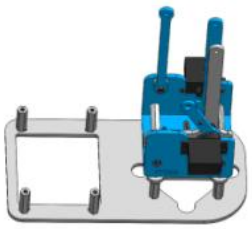
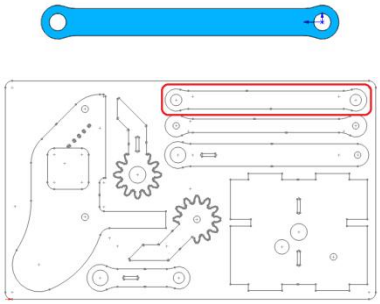
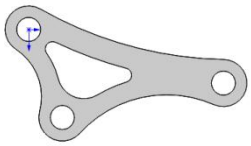
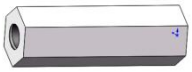
Step 15 - Assembly Right Servo Structure on Turntable

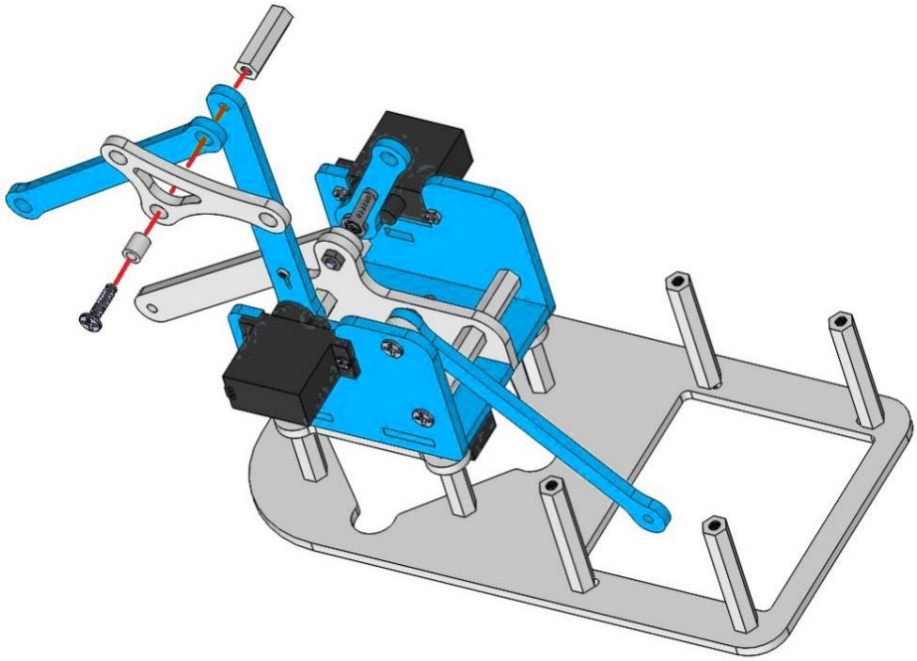
The structure in Step 14	The structure in Step 3	**Bag No.③** M2x8mm screw 2PCS	**Bag No.⑥** M2 nut 2PCS
			

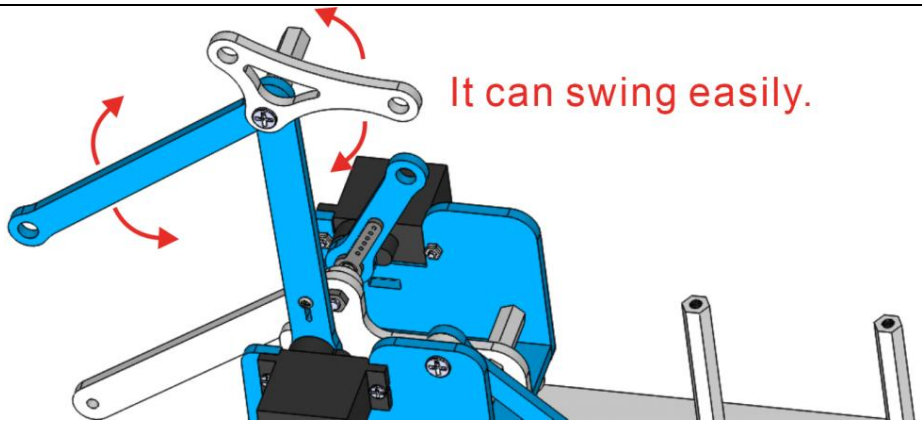




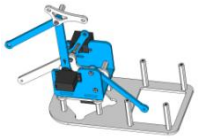
Step 16 - Assembly Primary Arm/Elbow Connector(1)

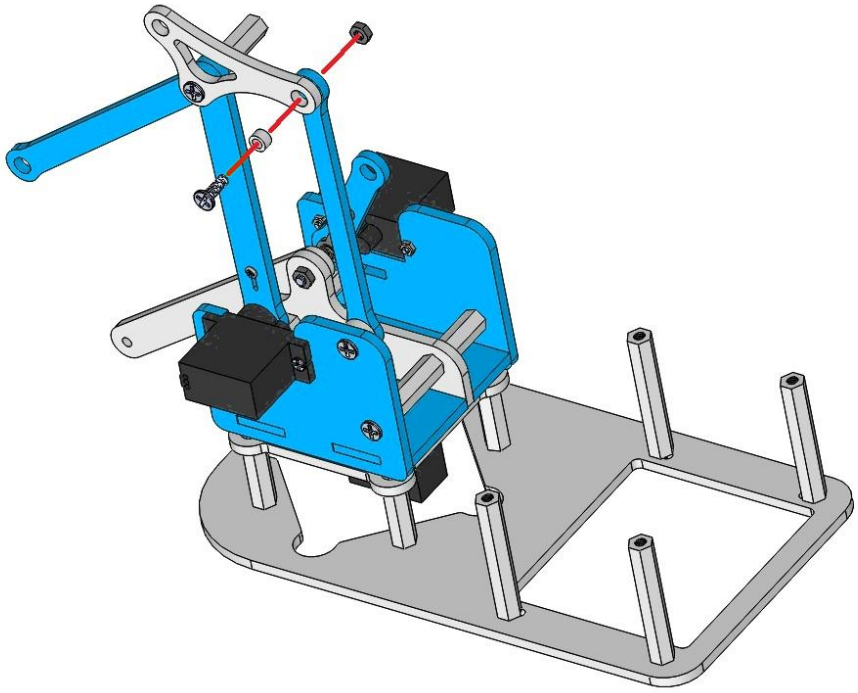
The structure in Step 15	Acrylic board 1PCS	Acrylic board 1PCS
		
Bag No.⑭ 5x3.2x6MM spacer 1PCS	**Bag No.⑪** M3x20MM Nylon Standoff 1PCS	**Bag No.①** M3X12MM screw 1PCS
		

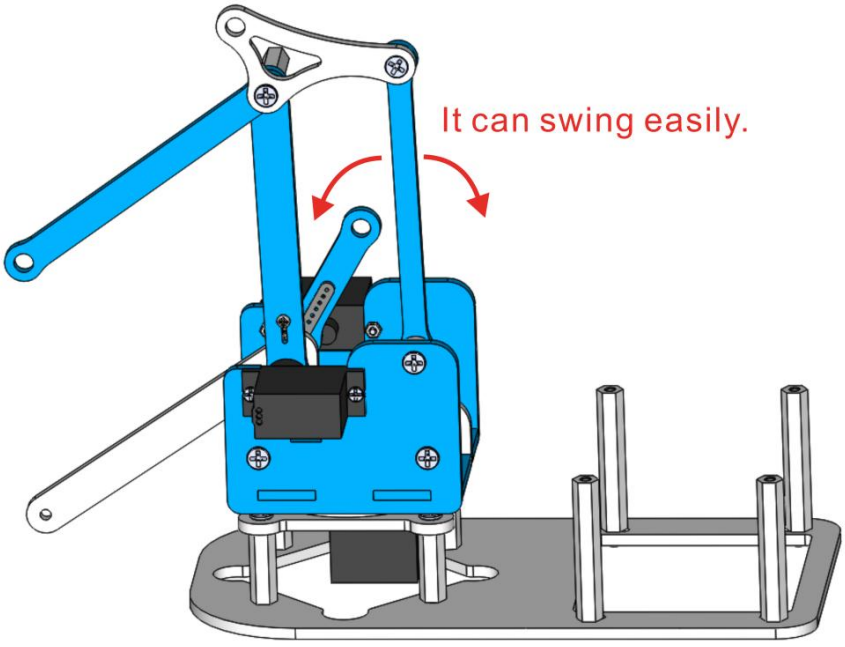




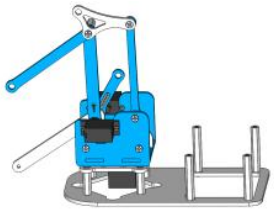
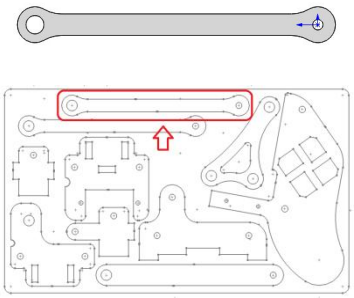
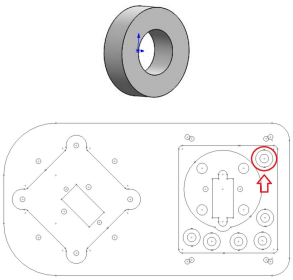
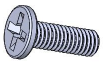
Step 17 - Assembly Primary Arm/Elbow Connector(2)

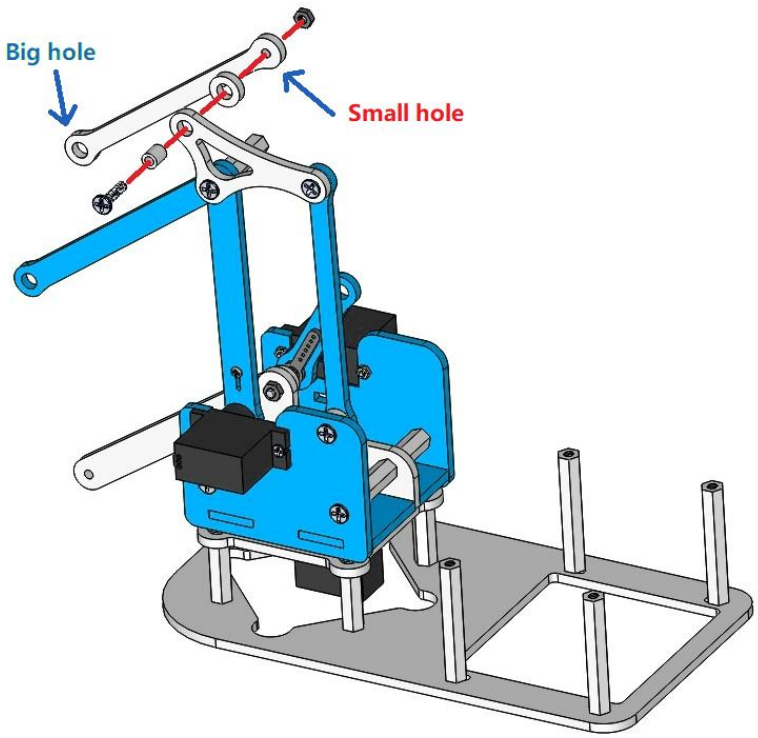
The structure in Step 16	**Bag No.⑬** 5x3.2x3M Mspacer 1PCS	**Bag No.②** M3X9MM screw 1PCS	**Bag No.⑤** M3 nut 1PCS
			

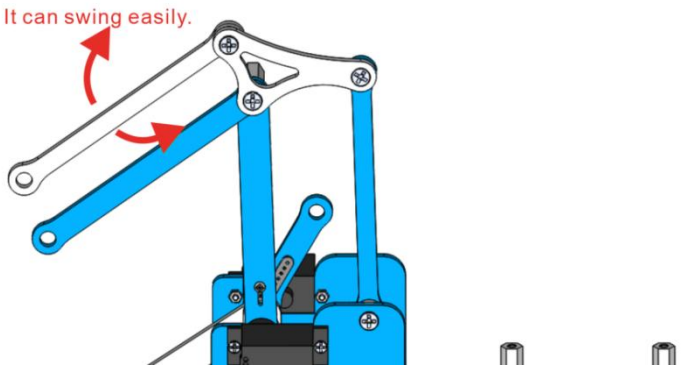




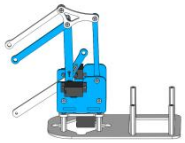
Step 18 - Attach Forearm Linkage (First Position)

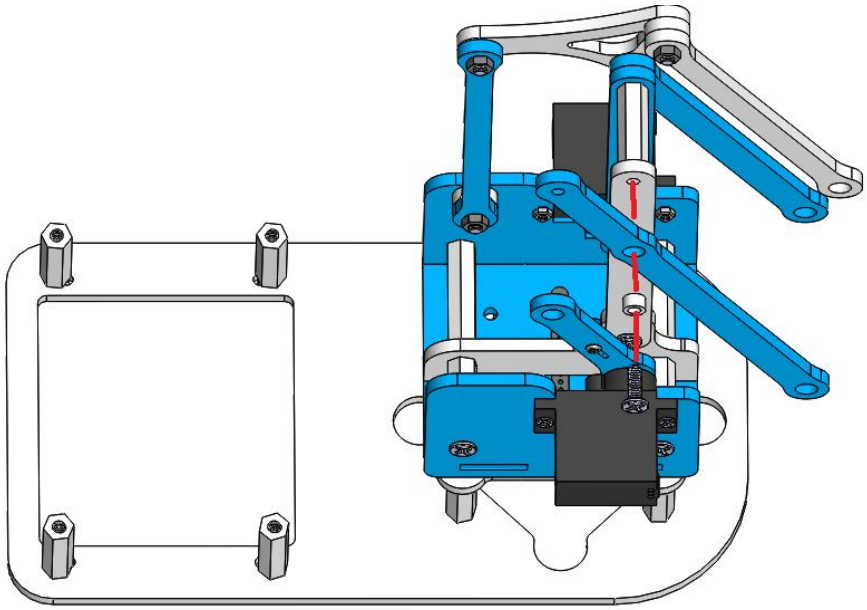
The structure in Step 17	Acrylic board 1PCS	Acrylic spacer 1PCS
		
Bag No.⑭ 5x3.2x6MM spacer 1PCS	**Bag No.①** M3X12MM screw 1PCS	**Bag No.⑤** M3 nut 1PCS
		

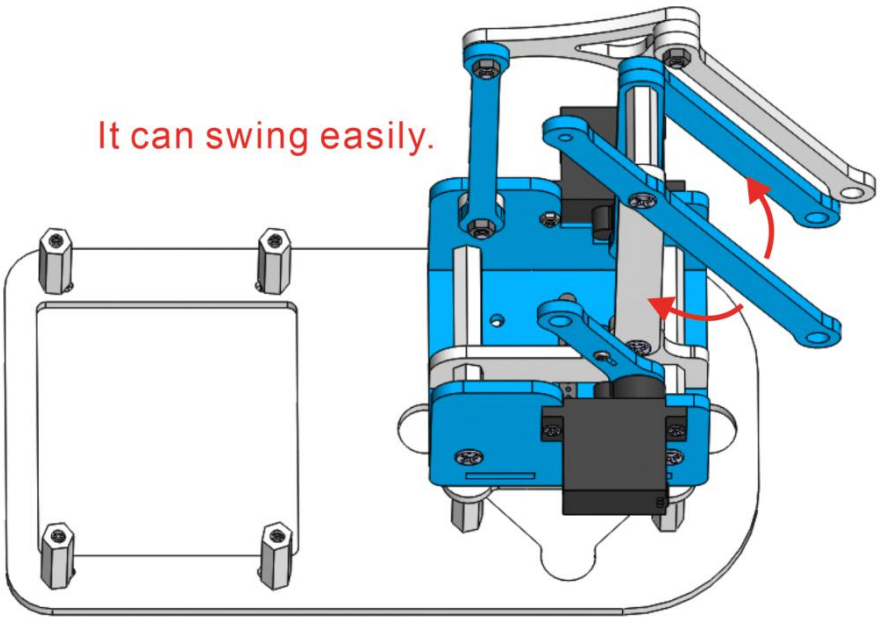




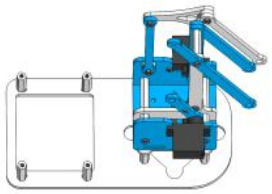
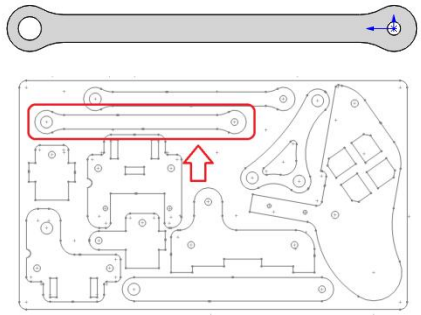
Step 19 - Attach Forearm Linkage (Second Position)

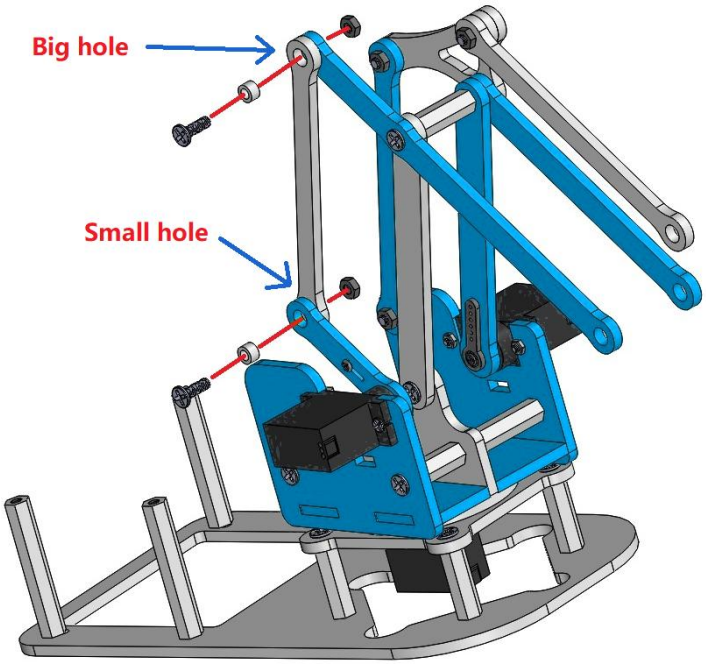
The structure in Step 18	Acrylic board 1PCS
	
Bag No.⑬ 5x3.2x3MM spacer 1PCS	**Bag No.②** M3X9MM screw 1PCS
	

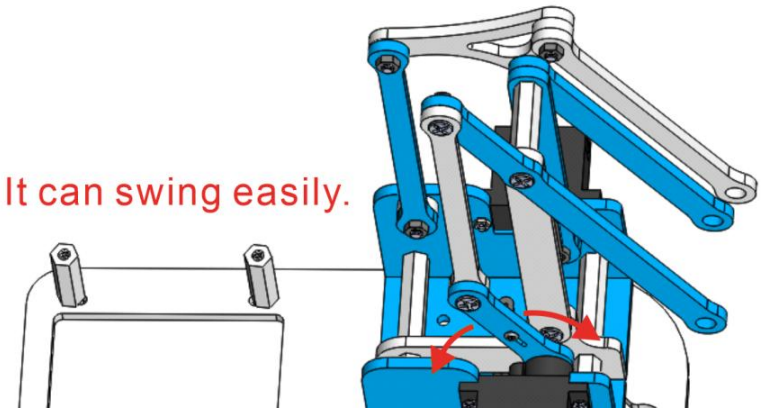




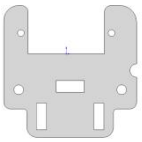
Step 20 - Connect Linkage between Upper Arm and Forearm

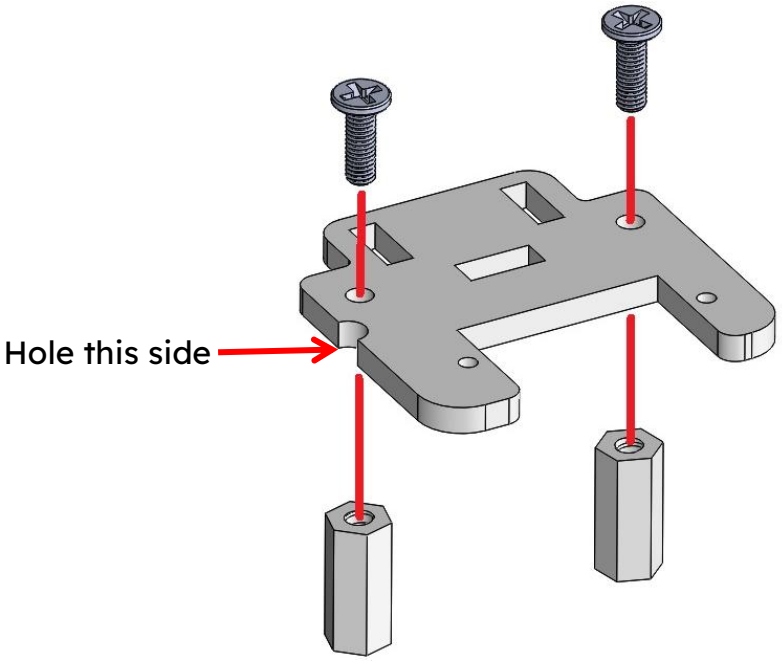
The structure in Step 19	Acrylic board 1PCS	
		
Bag No.⑬ 5x3.2x3MM spacer 2PCS	**Bag No.②** M3X9MM screw 2PCS	**Bag No.⑤** M3 nut 2PCS
		

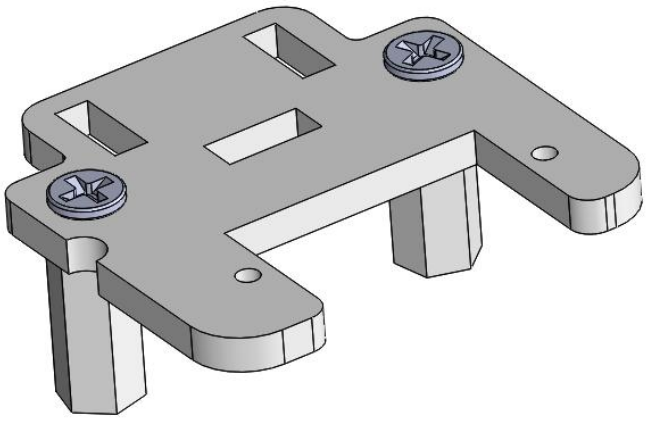


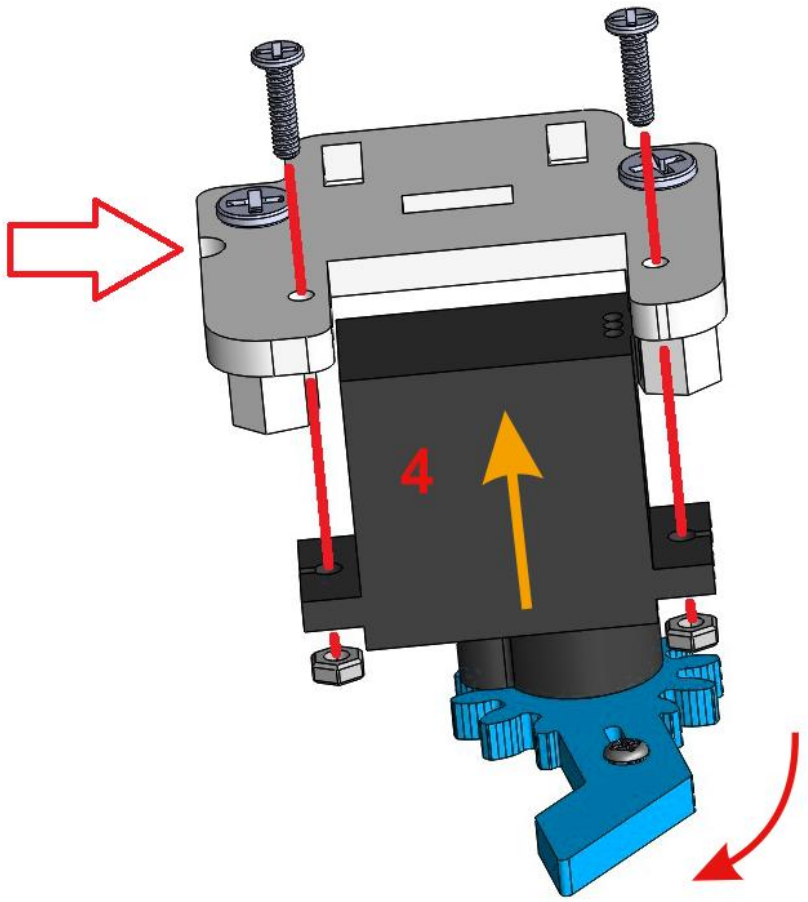


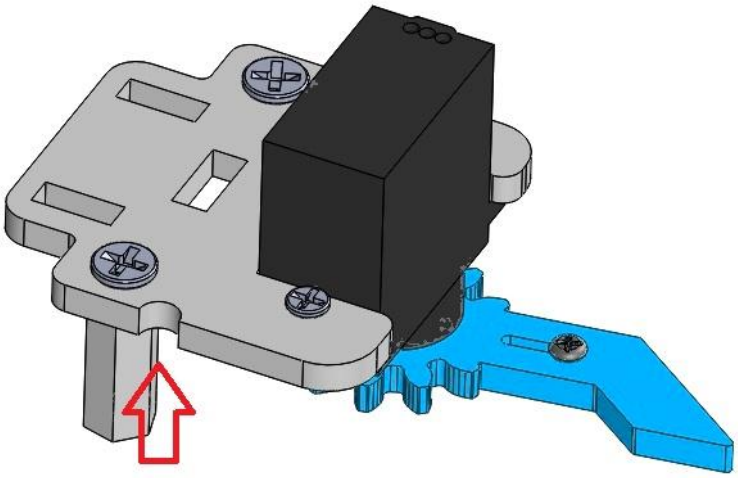
Step 21 - Mount Left Gripper Jaw

Acrylic board 1PCS	**Bag No.②** M3X9MM screw 2PCS	**Bag No.⑦** M3x14MM Nylon Standoff 2PCS
		
The structure in Step 4	**Bag No.③** M2x8mm screw 2PCS	**Bag No.⑥** M2 nut 2PCS
		

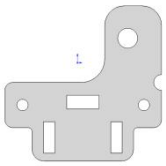
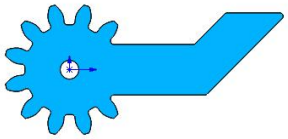


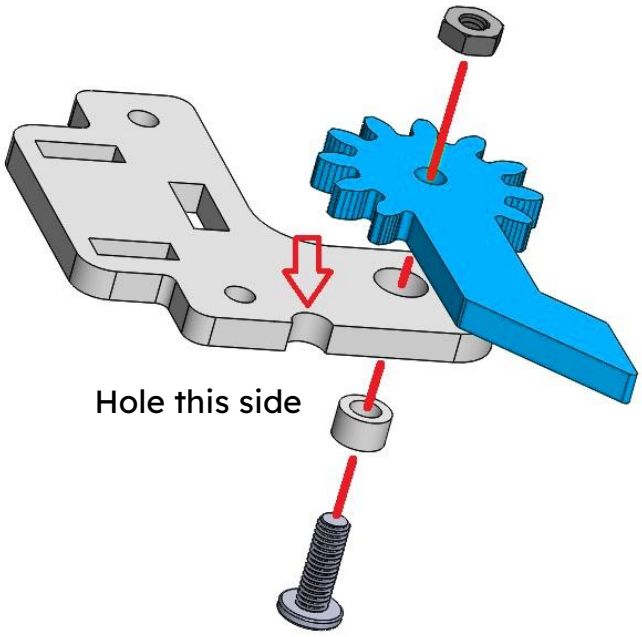


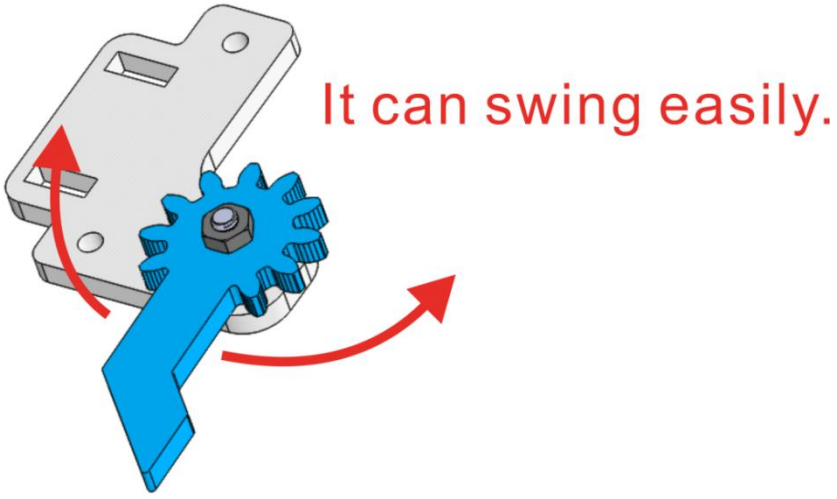

4



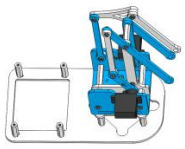
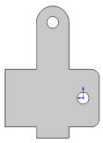
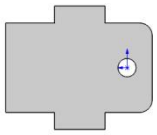
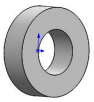
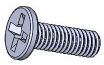
Step 22 - Mount Right Gripper Jaw

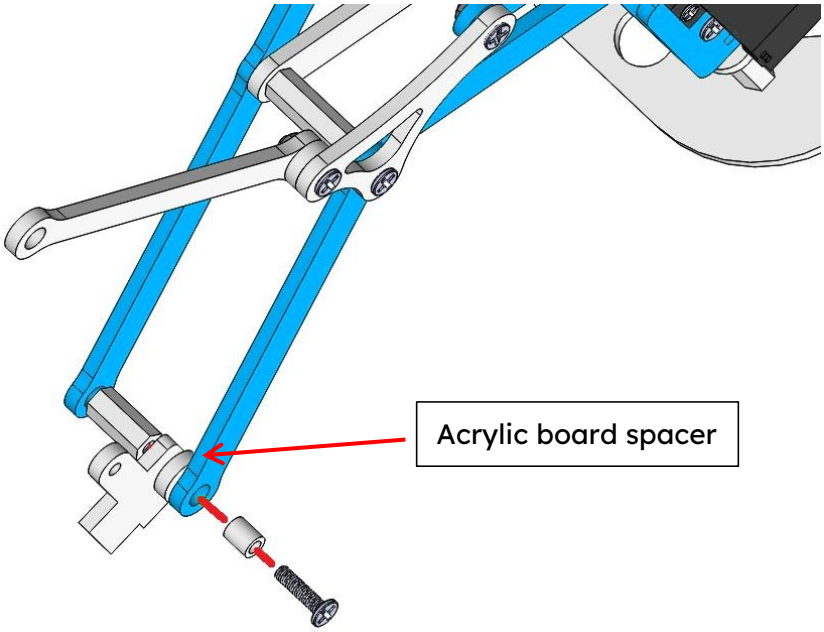
Acrylic board 1PCS		Acrylic board 1PCS
		
Bag No.⑬ 5x3.2x3MM spacer 1PCS	**Bag No.②** M3X9MM screw 1PCS	**Bag No.⑤** M3 nut 1PCS
		

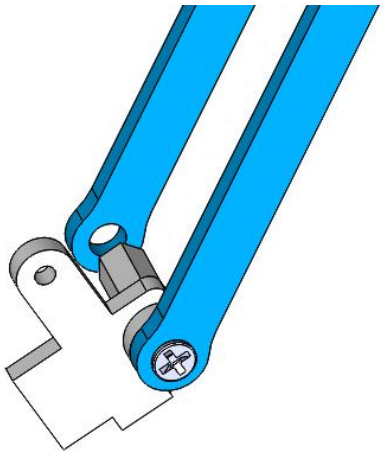


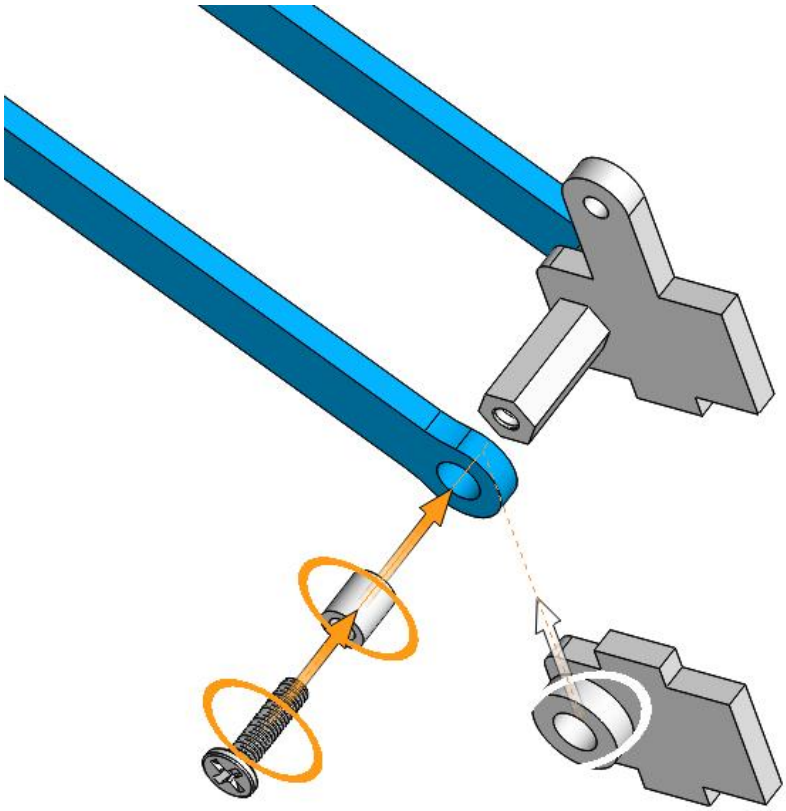


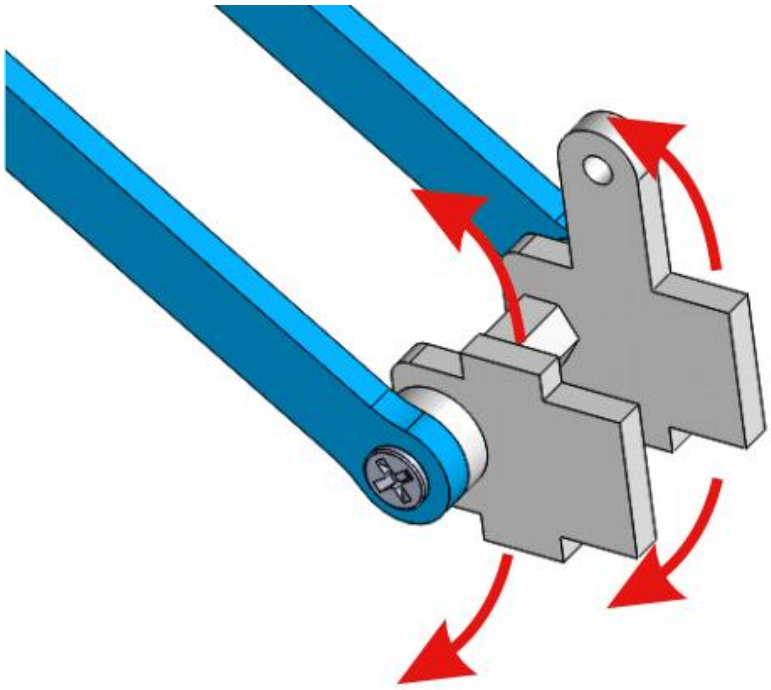
Step 23 - Assembly Wrist Structure

The structure in Step 20	Acrylic board 1PCS	Acrylic board 1PCS	Acrylic board spacer 2PCS
			
Bag No.① M3X12MM screw 2 PCS	**Bag No.⑦** M3x14MM Nylon Standoff 1PCS		**Bag No.⑭** 5x3.2x6MM spacer 2PCS
			



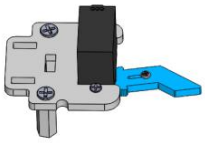
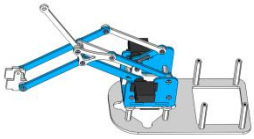
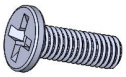


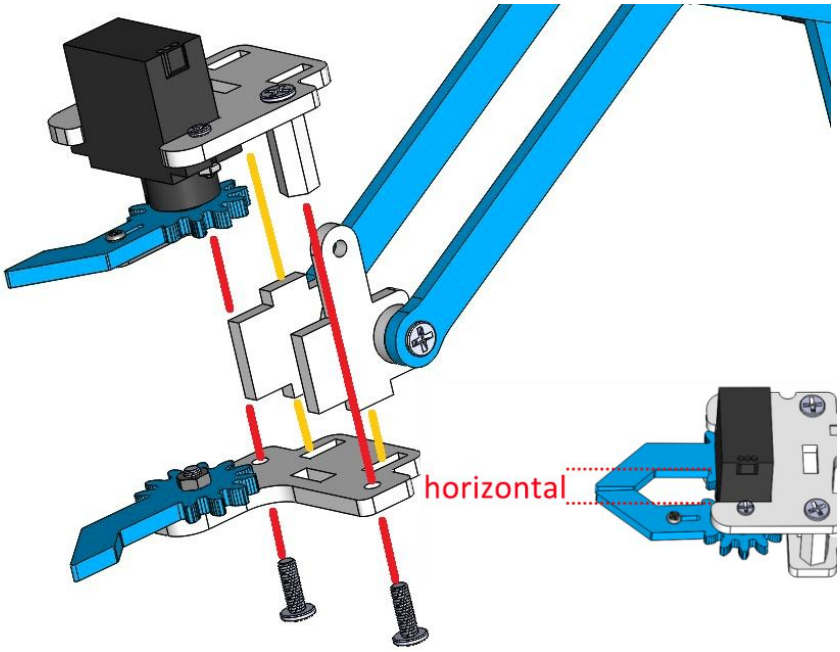


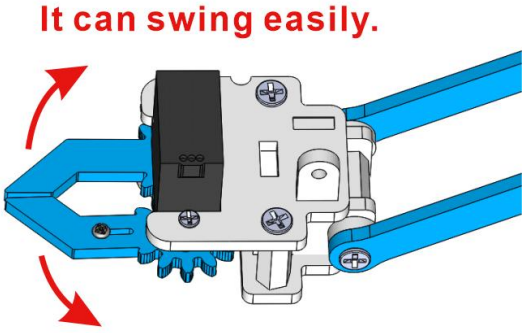


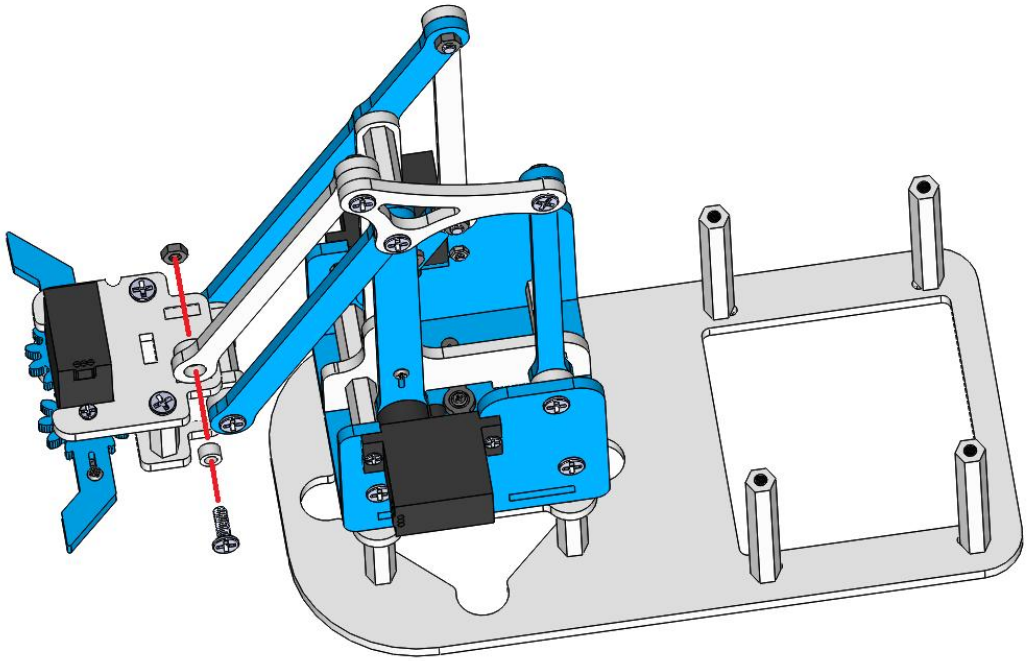
It can swing easily.

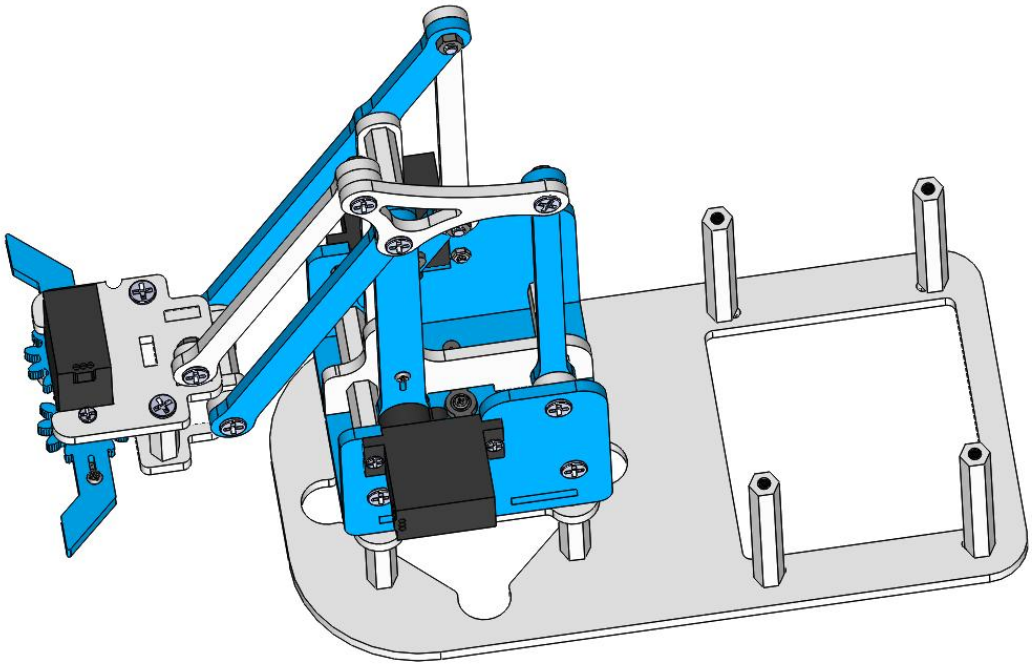
Step 24 - Attach Gripper to Wrist

The structure in Step 21	The structure in Step 22
	
The structure in Step 23	**Bag No.②** M3X9MMscrew3PCS
	
Bag No.⑤ M3 nut 1PCS	**Bag No.⑬** 5x3.2x3MM spacer 1PCS
	

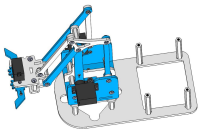


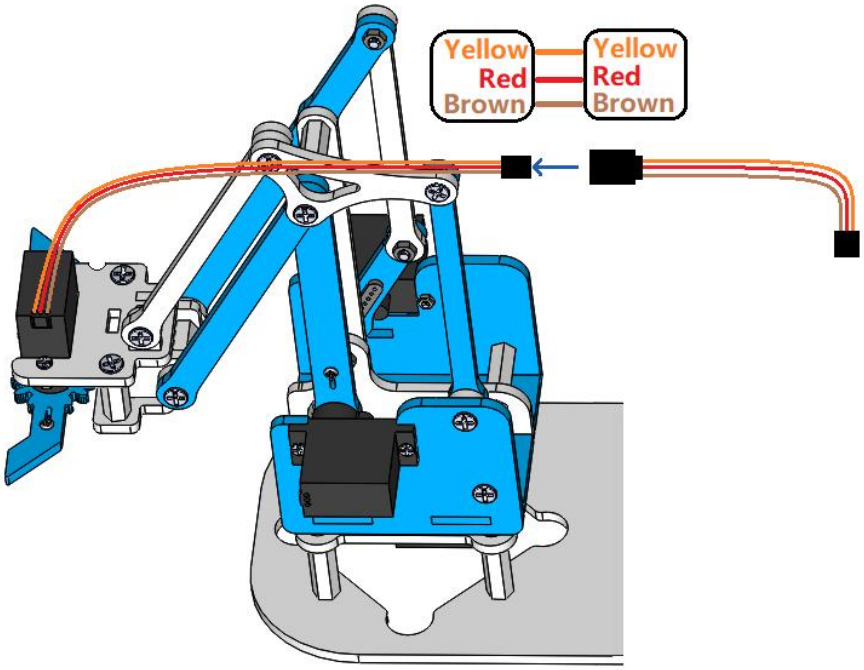


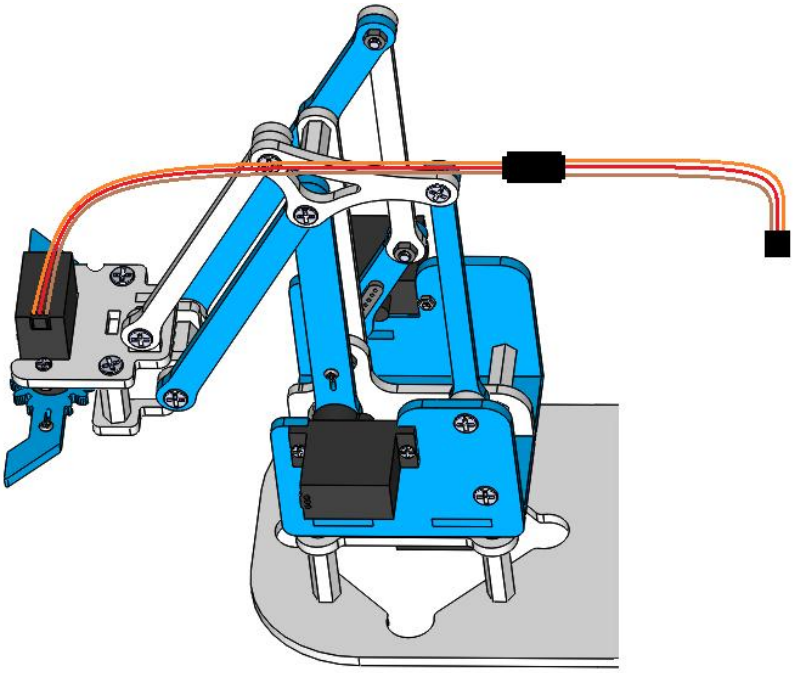




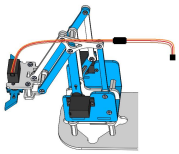
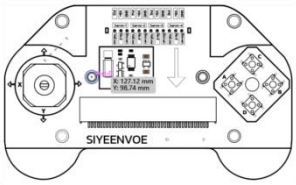
Step 25 - Connect Servo-4 with Extension Cable

The structure in Step 24	3P servo extension cable 1PCS
	



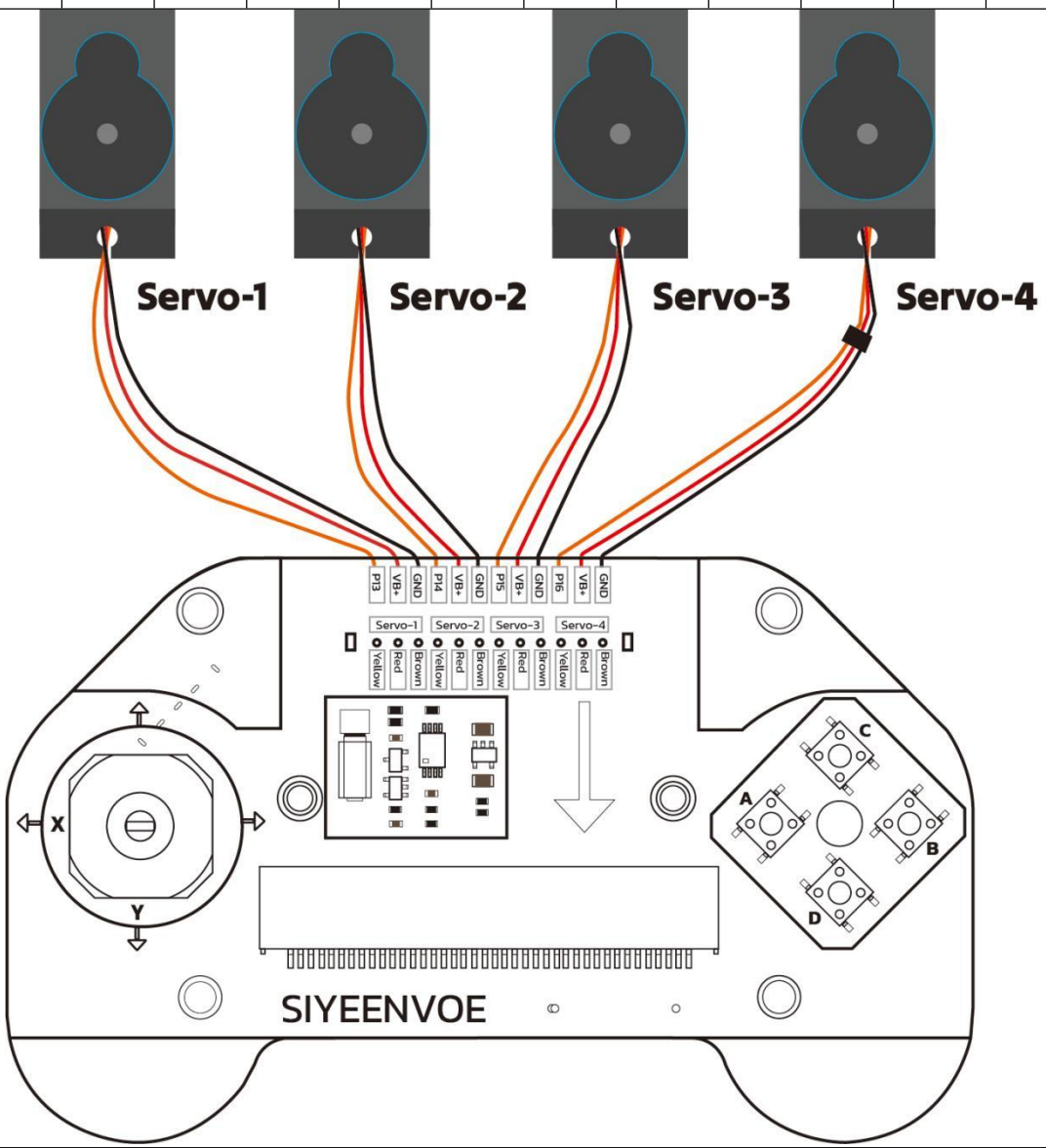


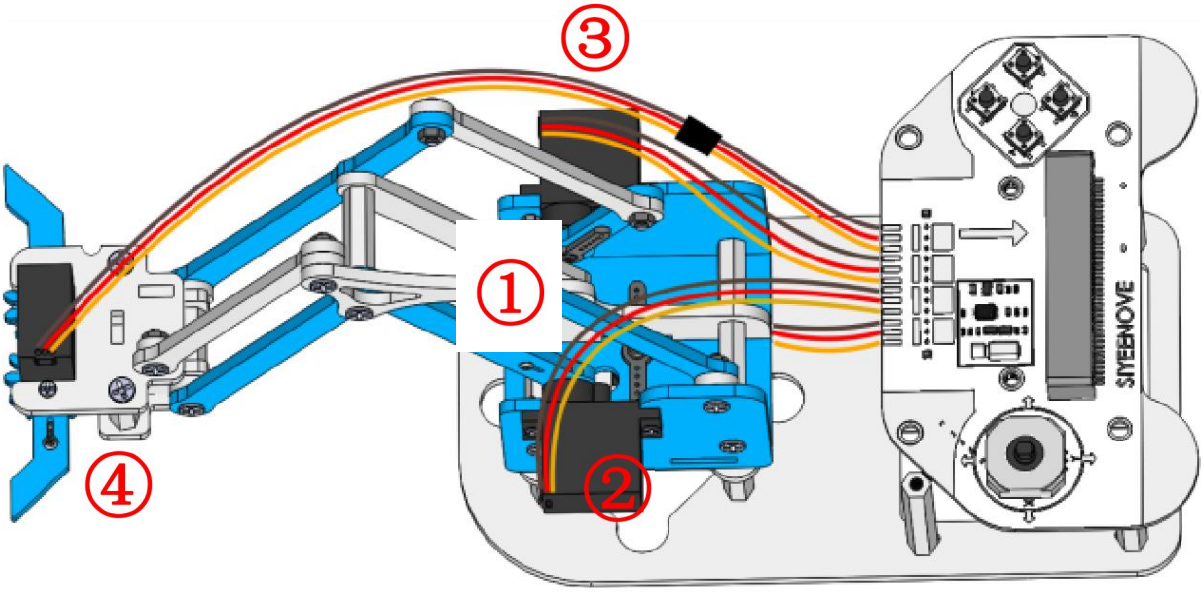
Step 26 - Wire Servos to Joystick Controller

The structure in Step 25	mJoystick
	

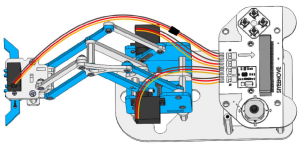
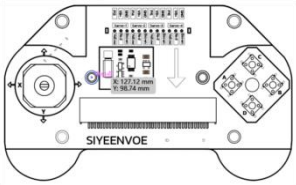
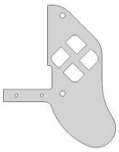
Wiring

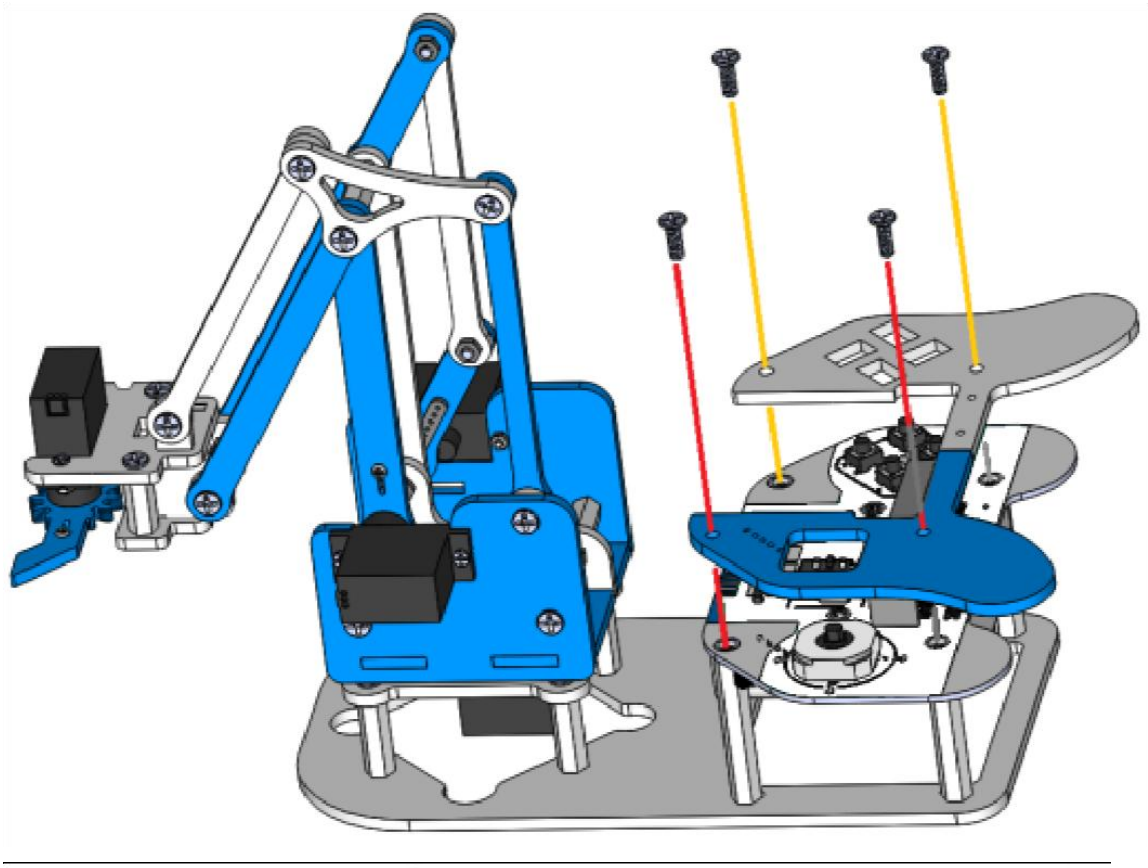
Servo-1 on the turntable			Servo-2 on the left			Servo-3 on the right			Servo-4 on the gripper		
P13	VB+	GND	P14	VB+	GND	P15	VB+	GND	P16	VB+	GND

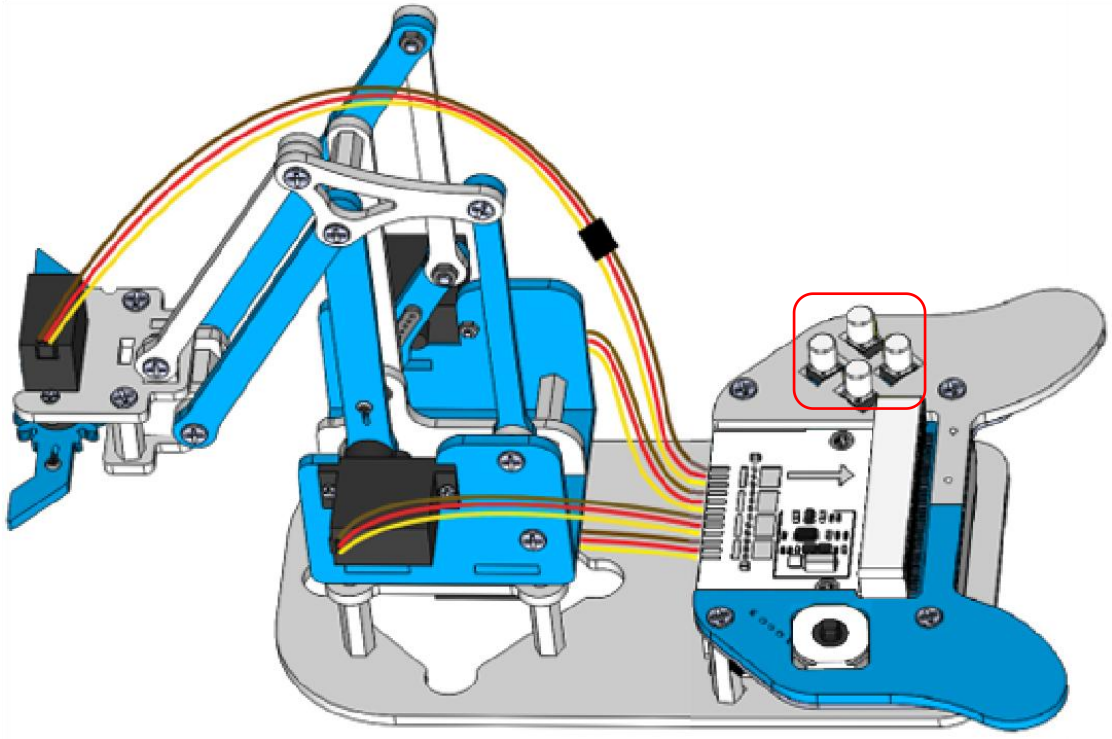




Step 27 - fix mJoystick to Base Frame

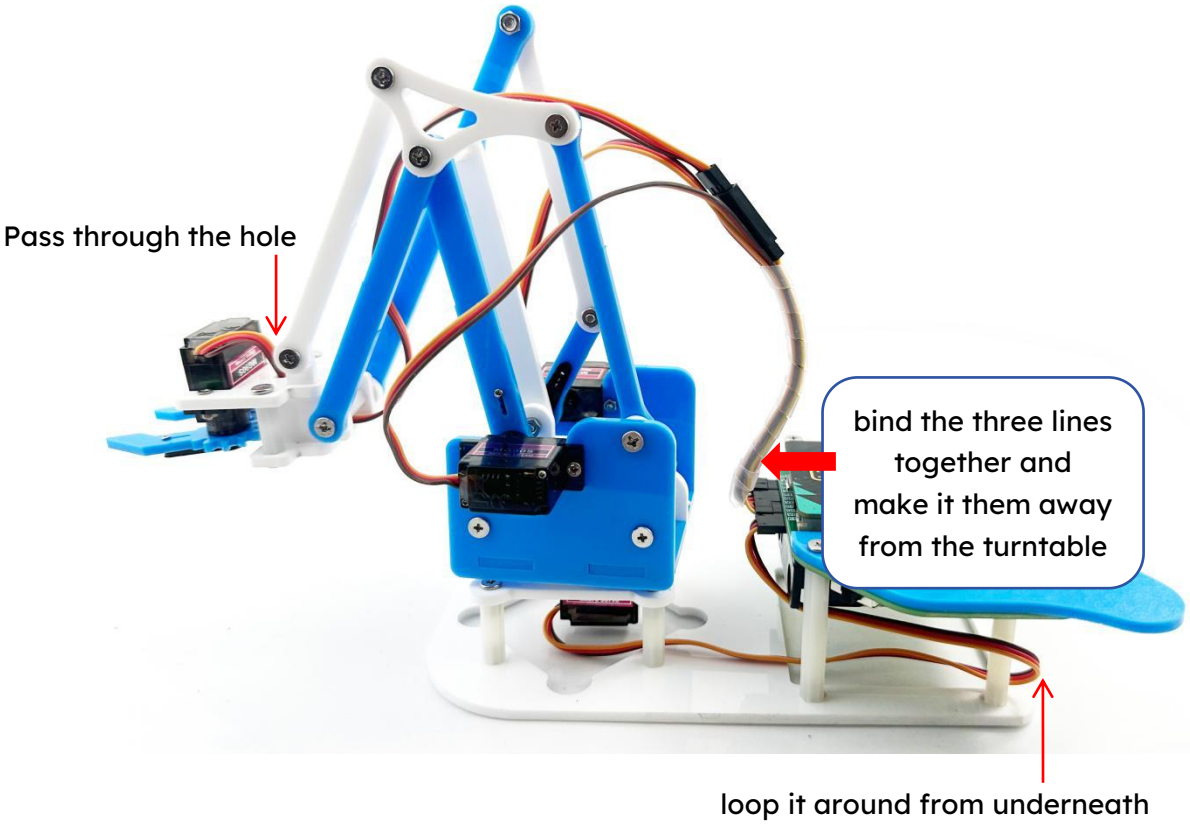
The structure in Step 26	mJoystick	**Bag No.②** M3X9MM screw 4PCS
		
Acrylic board 1PCS	Acrylic board 1PCS	**Bag No.⑰** Button cup 4PCS
		





Step 28 - Cable Management:

- **Spiral Wrap Tube Application:**
  Bundles multiple servo wires for cleaner aesthetics
  Prevents cable interference with moving parts
  Customizable length (trim with scissors as needed)



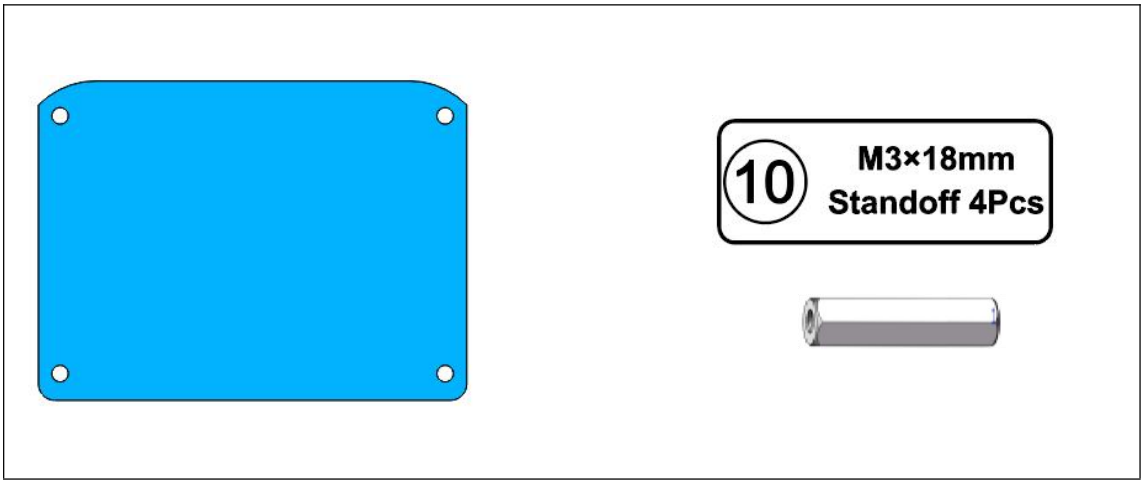
Assembly complete!

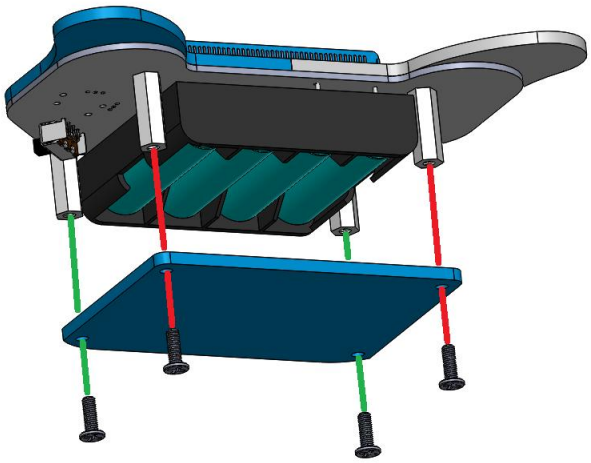
Function of the Remaining Acrylic Board

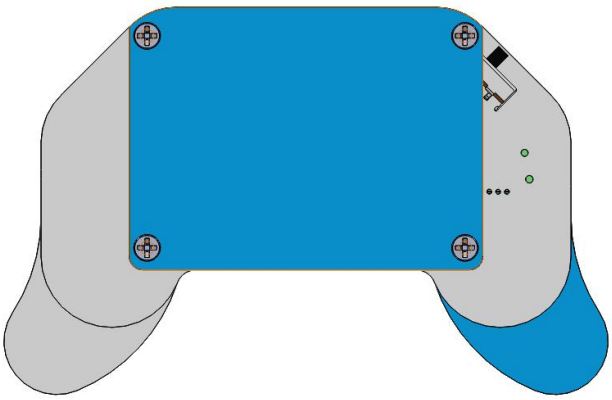
After assembly completion, you will have **1 spare acrylic board and M3x18mm Standoff 4pcs** for the joystick. This component serves critical purposes when using the joystick independently:

Primary Functions:

- Battery Compartment Cover
  - Securely retains 4×AA batteries in position
  - Prevents accidental dislodgement during operation
- Safety Barrier
  - Blocks direct finger contact with battery terminals
  - Eliminates short-circuit risks from metallic objects
- Structural Reinforcement
  - Adds rigidity to the handle's battery housing



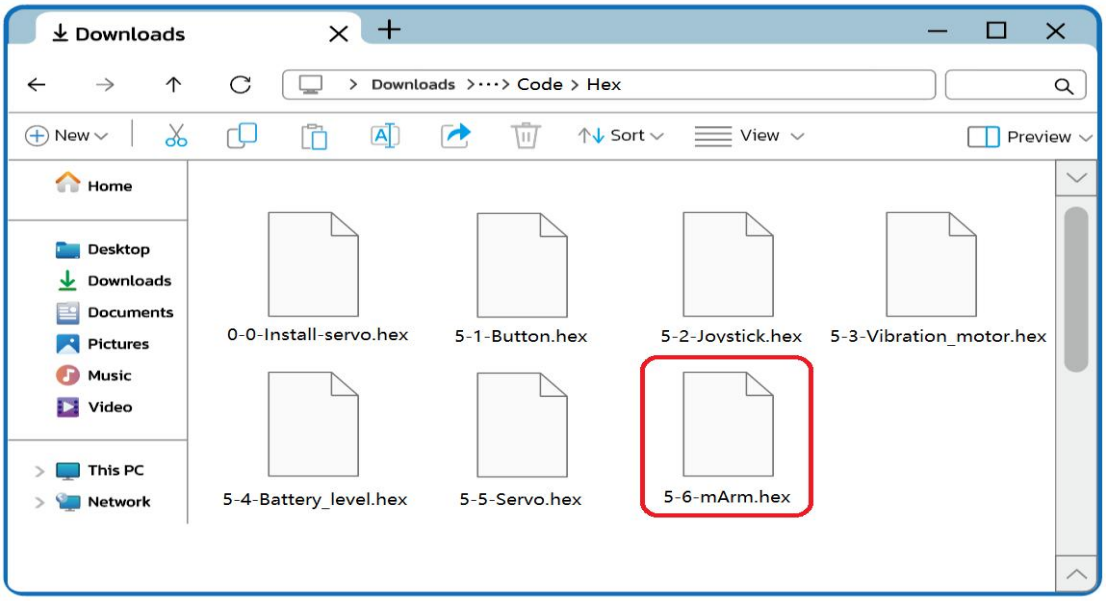




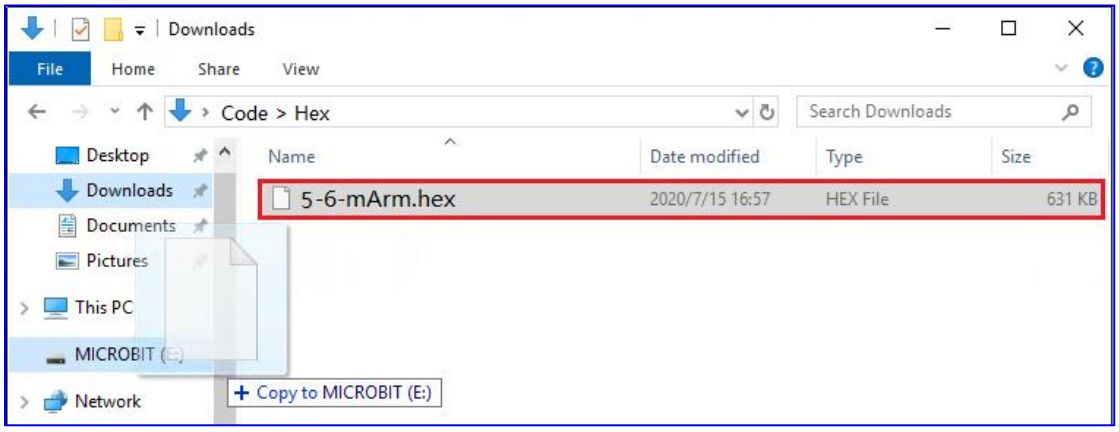
3.Control the Robot Arm

3.1 Upload the control code

The control code for the robotic arm, **5-6-mArm.hex**, is stored in the "**Code**" folder of the downloaded tutorial package.

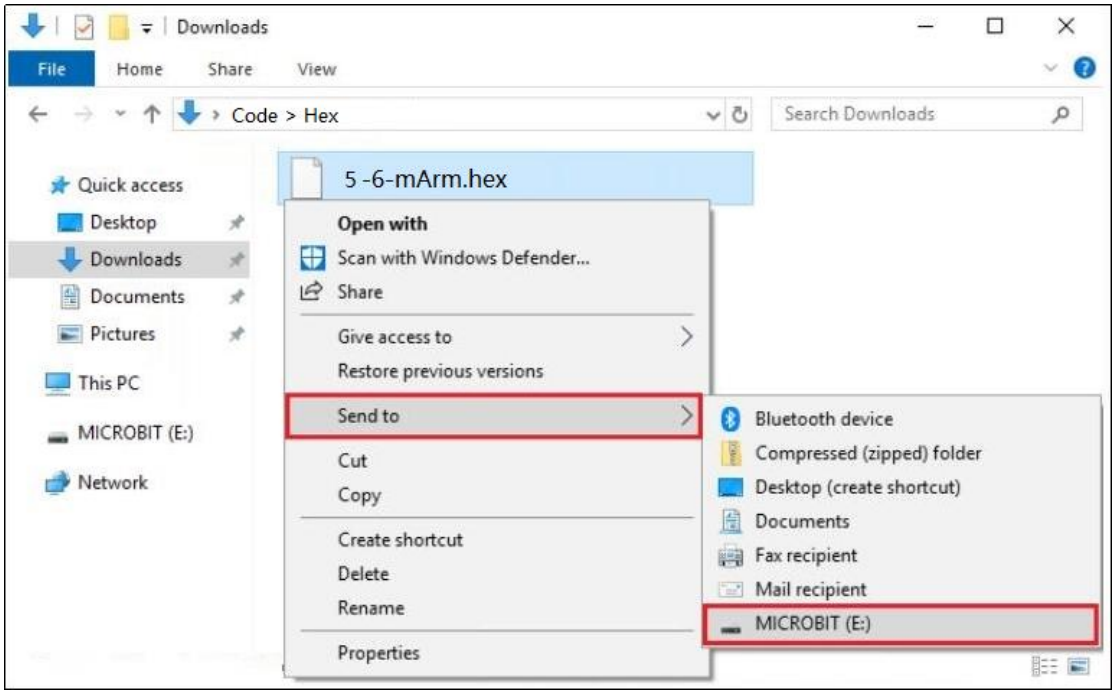


Simply **drag and drop** the HEX file into the **MICROBIT drive.**

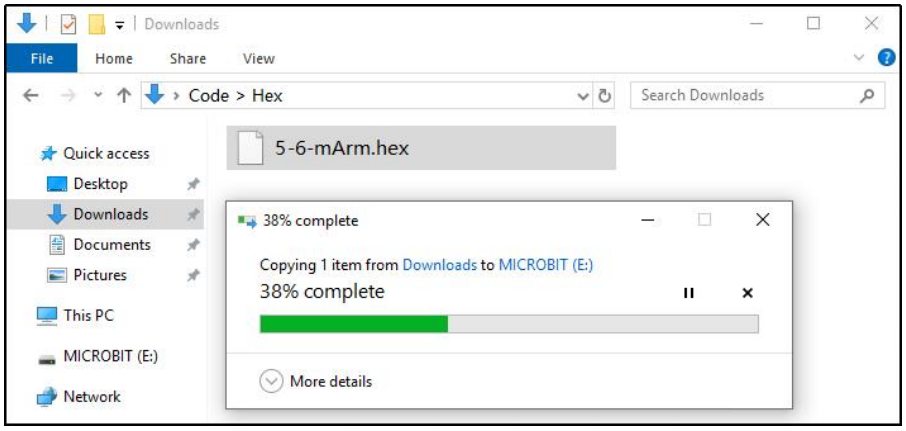


Note: This is a micro:bit v2-specific hex file. Dragging or sending it into a v1 will usually result in silent failure (the file copies successfully but the program doesn't run, or it only shows an error pattern). If you're using a micro:bit v1, we recommend the following from section 4 "micro:bit Python Editor Quick Start":
Use the online editor (via WebUSB) to flash directly — it generates a universal hex by default (compatible with both v1 and v2).
Or use the v1 board regenerate the hex file with the python editor, then drag it in.

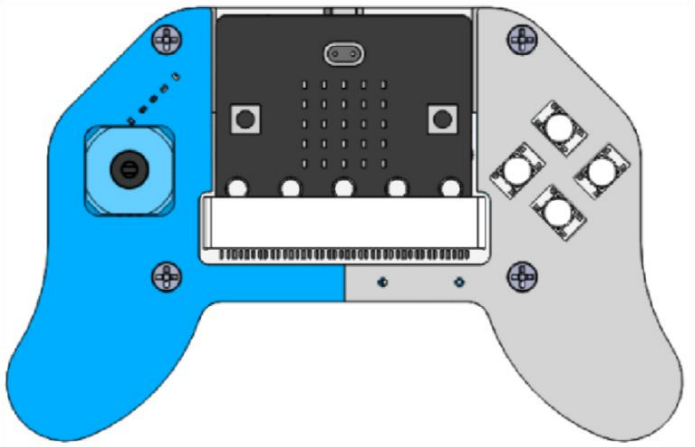
Or right-click the HEX file and select **"Send to" → MICROBIT**



Wait for the upload to complete (the micro:bit LED will blink during the process).

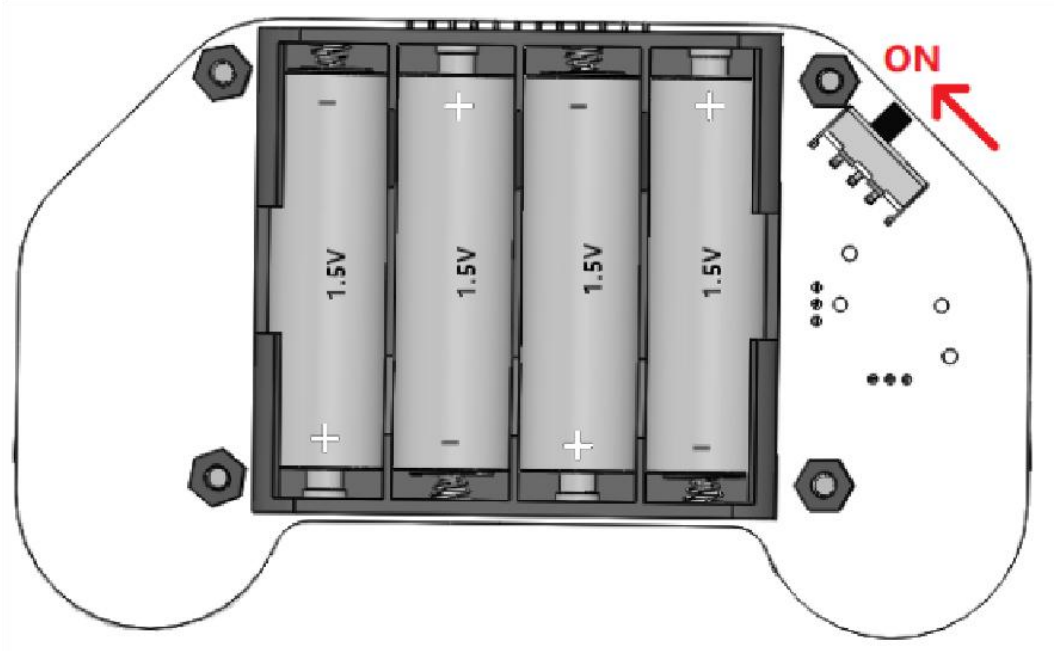


After the code is successfully uploaded, insert the micro:bit board into the robot arm slot.



3.2 Quick Start to Robot Arm Control

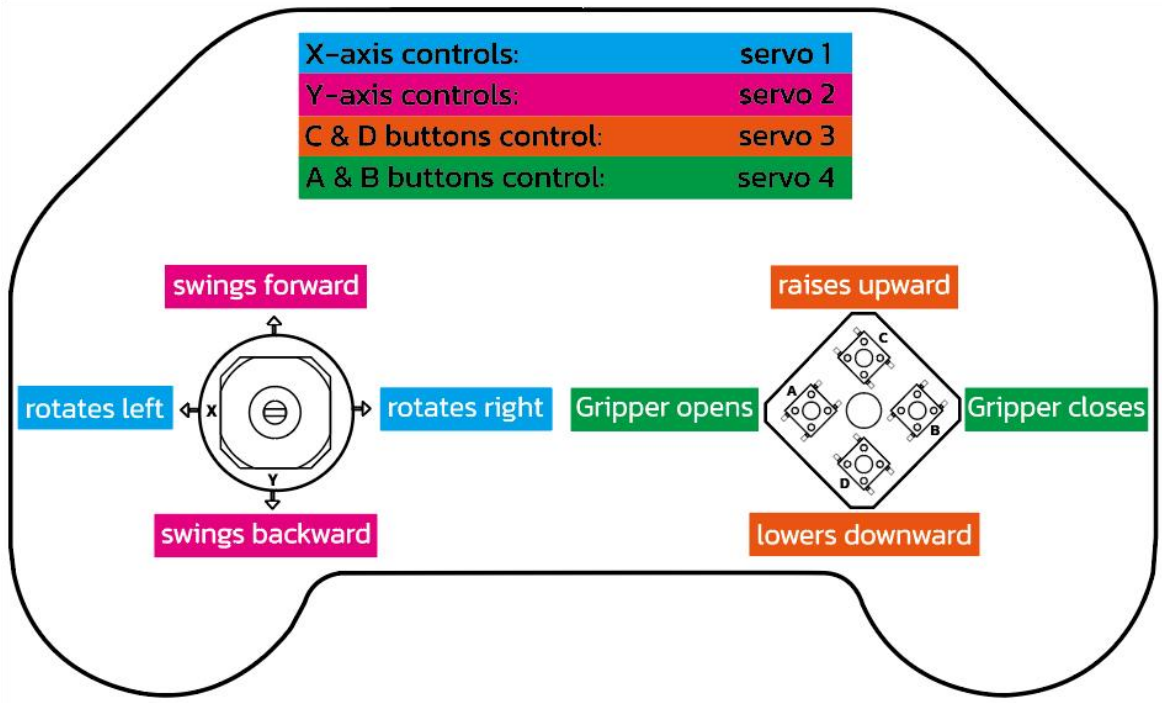
Turn on the power switch of the Robot Arm.



-After powering on, the Micro:bit's LED matrix will continuously cycle through displaying the battery level (0%–100%). When the battery level drops below 30%, replace all batteries immediately.

-You can control the robotic arm's movement using the joystick and four buttons.

-Pressing the touch logo on the micro:bit will activate the vibration motor on the controller.



4.micro:bit Python Editor Quick Start

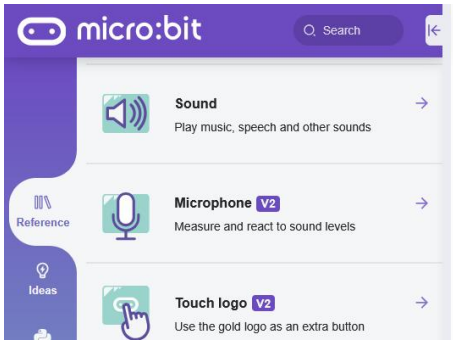
The **micro:bit Python Editor** is the perfect way to get started with MicroPython programming on the BBC micro:bit. It is free and works in browsers across all platforms.

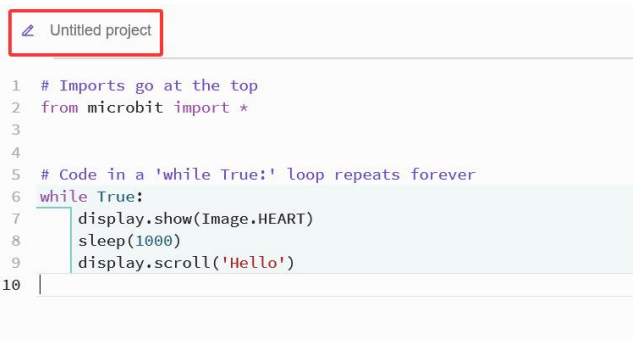
We recommend using Chrome or Edge browsers. WebUSB is a recent and developing web feature that allows you to access a micro:bit directly from a web page. It also lets you directly receive data into the micro:bit Python editor from the micro:bit. It works in Google Chrome and Microsoft Edge browsers.

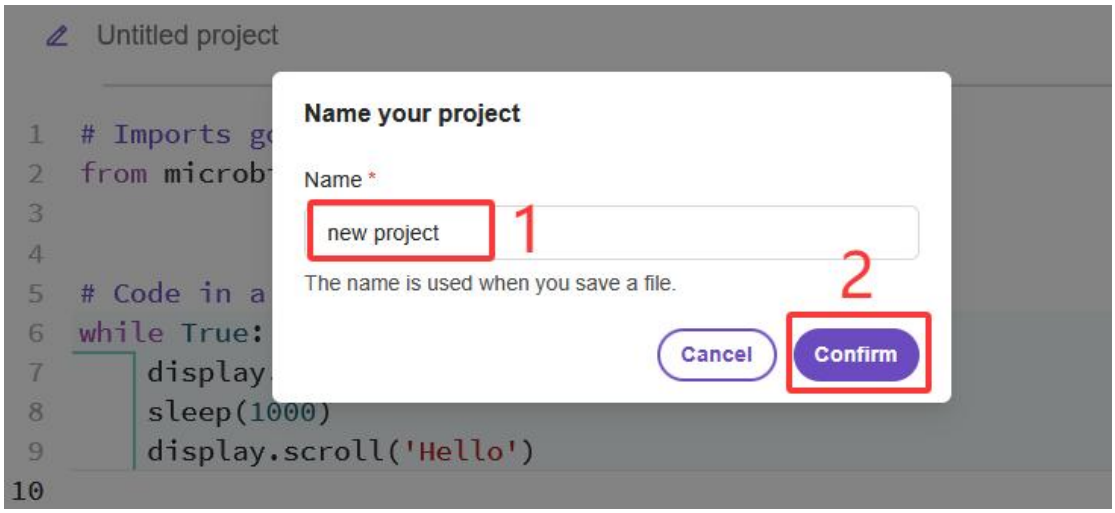
WebUSB support for your micro:bit
If you're not using a current version of the Chrome or Microsoft Edge browsers, make sure they are this version or newer:
Chrome (version 79 and newer) browser for Android, Chrome OS, Linux, macOS and Windows 10.
Microsoft Edge (version 79 and newer) browser for Android, Chrome OS, Linux, macOS and Windows 10.
Link to download the latest Google Chrome:
https://www.google.com/chrome/
Link to download the latest Microsoft Edge:
https://www.microsoft.com/en-us/edge/download

4.1 Create a new project

Open the micro:bit Python Editor on your browser:
https://python.microbit.org/v/3.
Once the editor loads, you will see a code editing area containing a simple starter template. Click on "**Untitled project**" at the top of the editor, then enter a meaningful name for your project (e.g., Joystick_Test).
You can edit the existing template or replace it entirely with your own Python code.





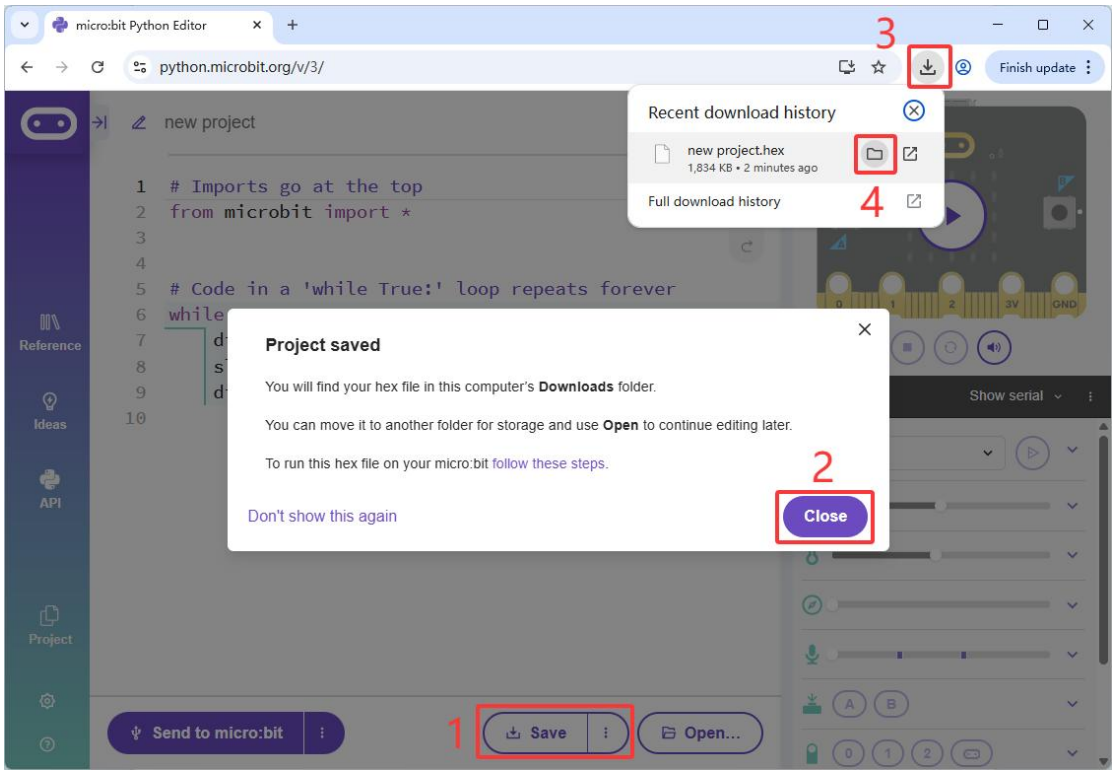


Note: The micro:bit Python Editor operates on a single-project model, meaning only one project can be edit at a time. It does not support multi-tab browsing, side-by-side editing of multiple files, or built-in project switching.

4.2 Save a project

1. Save Project as hex file

Click the "**Save**" button, the editor automatically saves your work in your browser's local storage. A downloaded hex file will be found in your browser's default download folder.



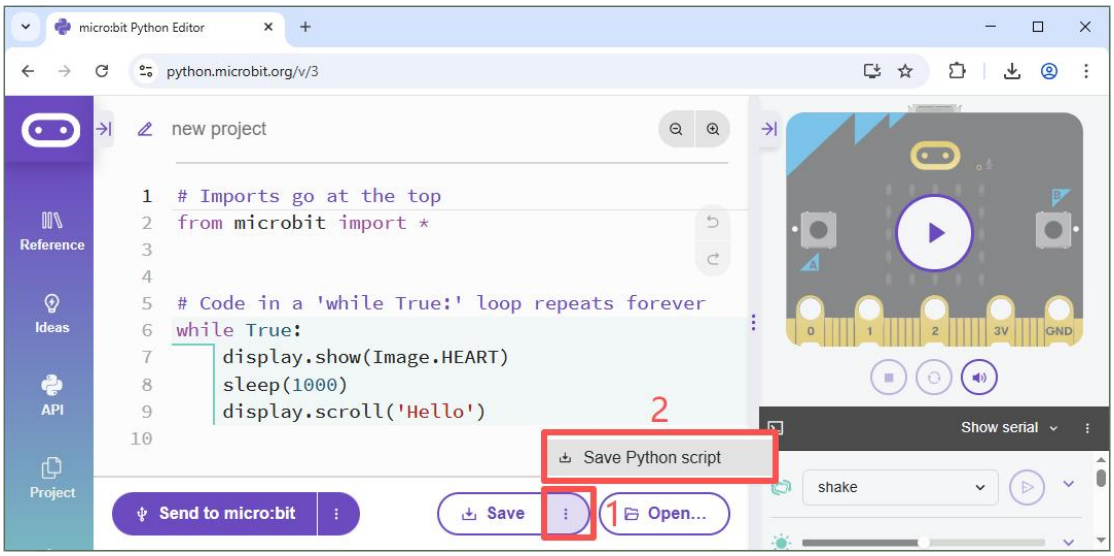
This is the final file format that the micro:bit can understand and execute.

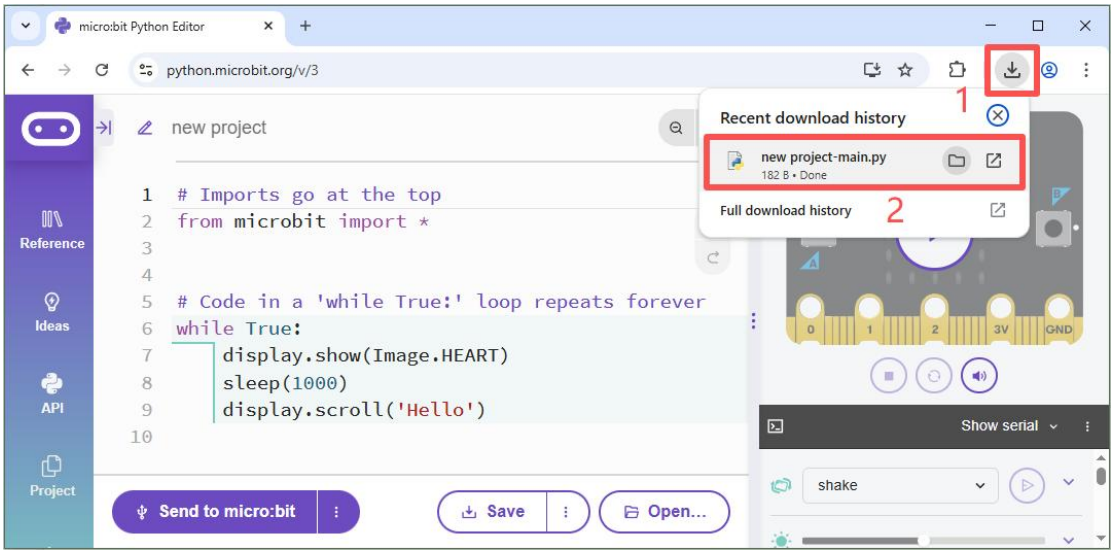
Functions:

Upload to micro:bit: Drag and drop the .hex file into the micro:bit's USB drive (or send it), and the micro:bit will load it into memory and start execution.
Project Backup and Sharing: You can save the .hex file to back up an immediately runnable version of the project or share it with others, who can use it directly without needing to access the source code.

2. Save Project as .py File

Click the three dots next to the "**Save**" button, then click the "**Save Python script**" button, the .py file will be saved to your computer's downloads folder.





This is the written Python code text.

Functions:

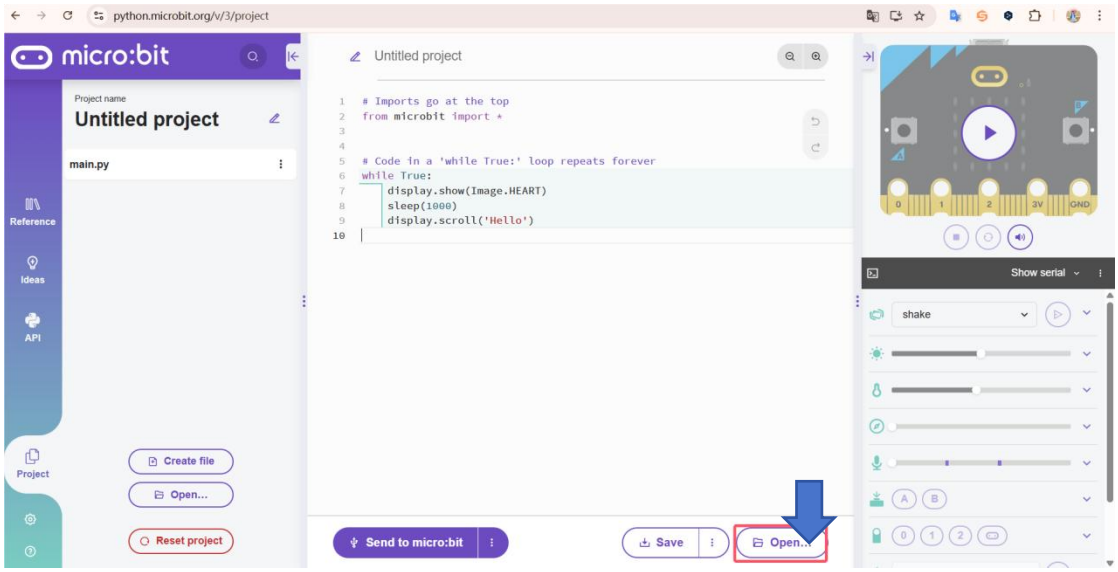
Editing and Creation: Write, modify, and debug your program in the micro:bit Python Editor or any text editor.

Remember: The micro:bit runs .hex files, but you develop in .py files.

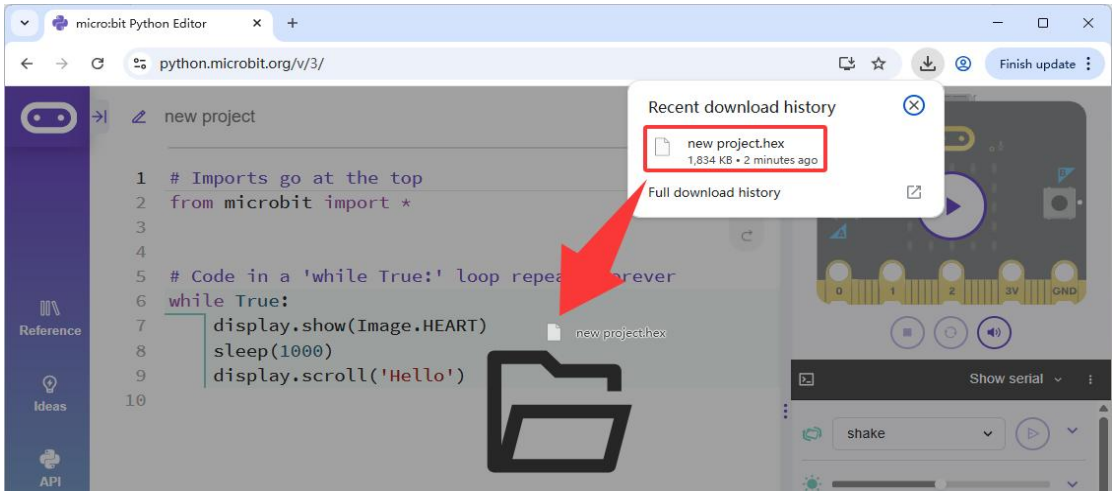
4.3 Import Files

To open a program (e.g. one you saved earlier or one provided by a teacher or third party), choose the 'Open' button.

From here, select the .hex or .py file that you wish to open and then click open.



Alternatively, you can drag and drop a .hex file or .py file into the editor.

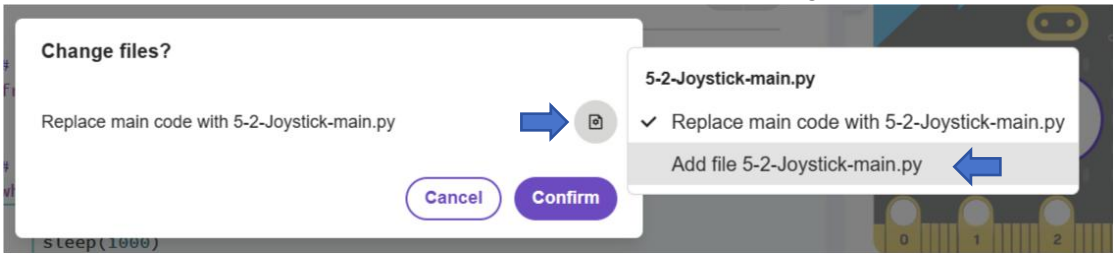


When importing files:

Opening a new .hex file will replace the current project immediately without any warning, and this action is irreversible. Be sure to save your work beforehand to avoid losing code.

Opening a new .py file will trigger a prompt: "Change files?" You can then choose to:

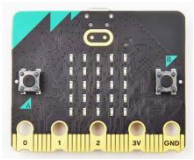
Confirm to replace the current code,
Cancel to keep the current project, or
Add to insert the new file as an extension to the existing code.



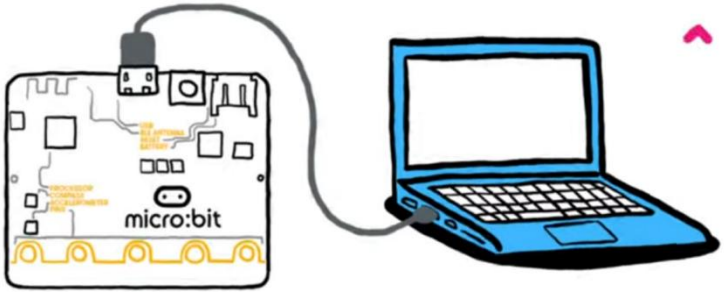
Note: The micro:bit Python Editor operates on a single-project model, meaning only one project can be open and active at a time. It does not support multi-tab browsing, side-by-side editing of multiple files, or built-in project switching. Loading a new project will replace the current project. This streamlined, focused approach simplifies the interface for beginners. It reduces cognitive load by letting students focus solely on the current task, avoiding confusion from switching between multiple projects and aligning with the "one thing at a time" teaching philosophy.

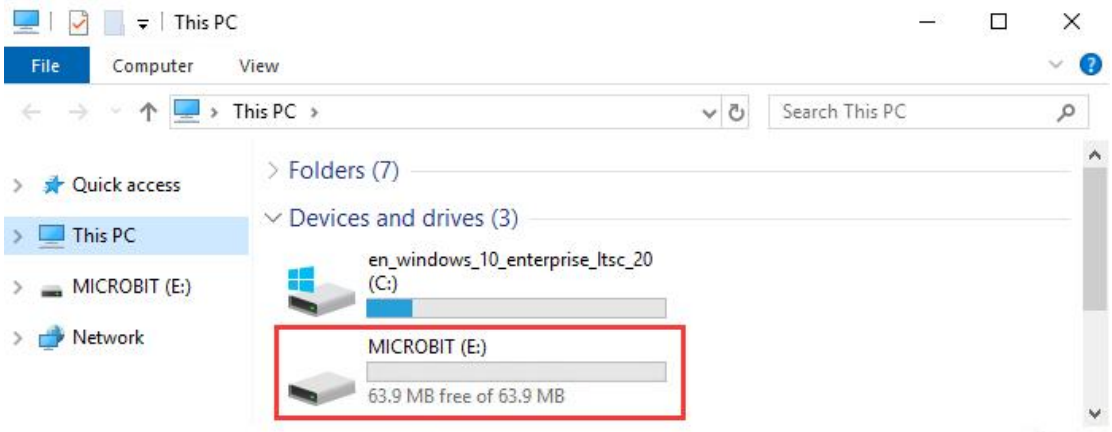
4.4 Upload code

Things you need:

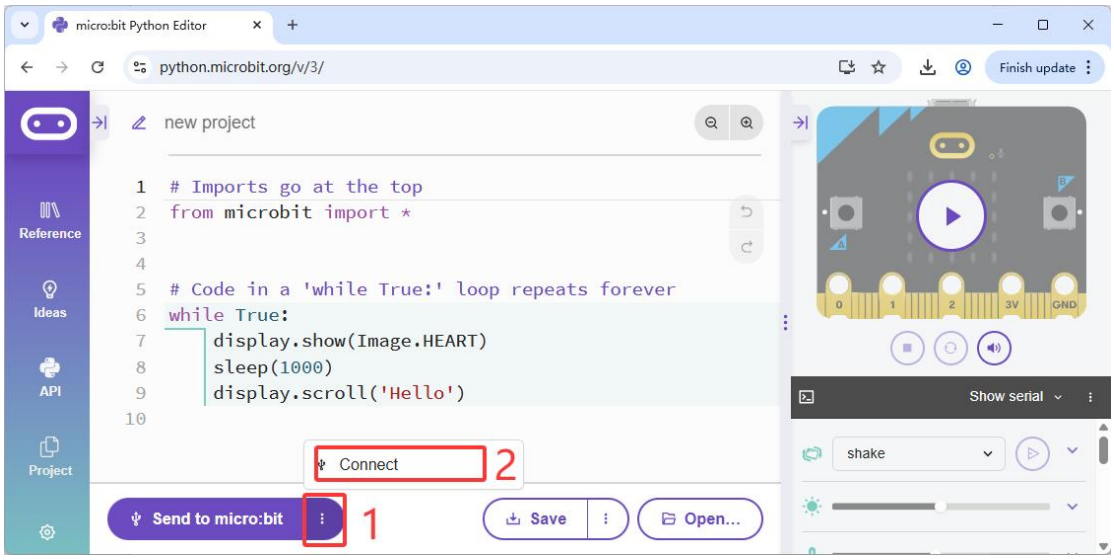
PC	Micro:bit v2.x.x	Micro USB Cable
		

Use a Micro USB cable to connect the micro:bit board to the PC. You will find a new USB disk called MICROBIT on the PC:

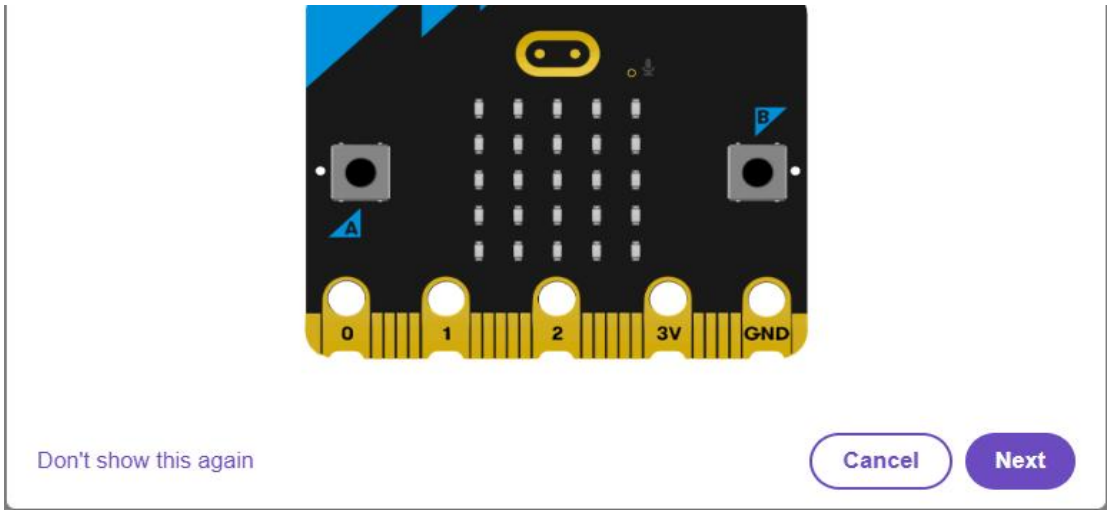


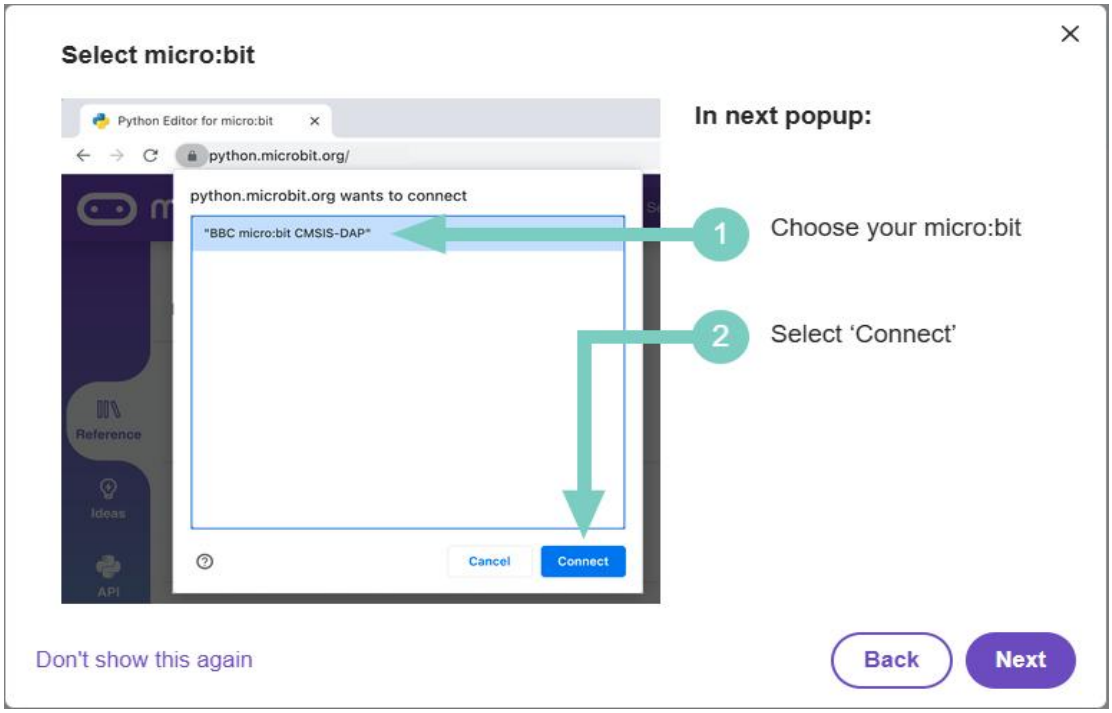


Direct Flashing (WebUSB): If you are using a compatible browser like Google Chrome or Microsoft Edge, you can use WebUSB to connect your micro:bit directly to the editor and flash the program without dragging and dropping files. Click the three dots next to the "**Send to micro:bit**" button, then click the "**Connect**" button：

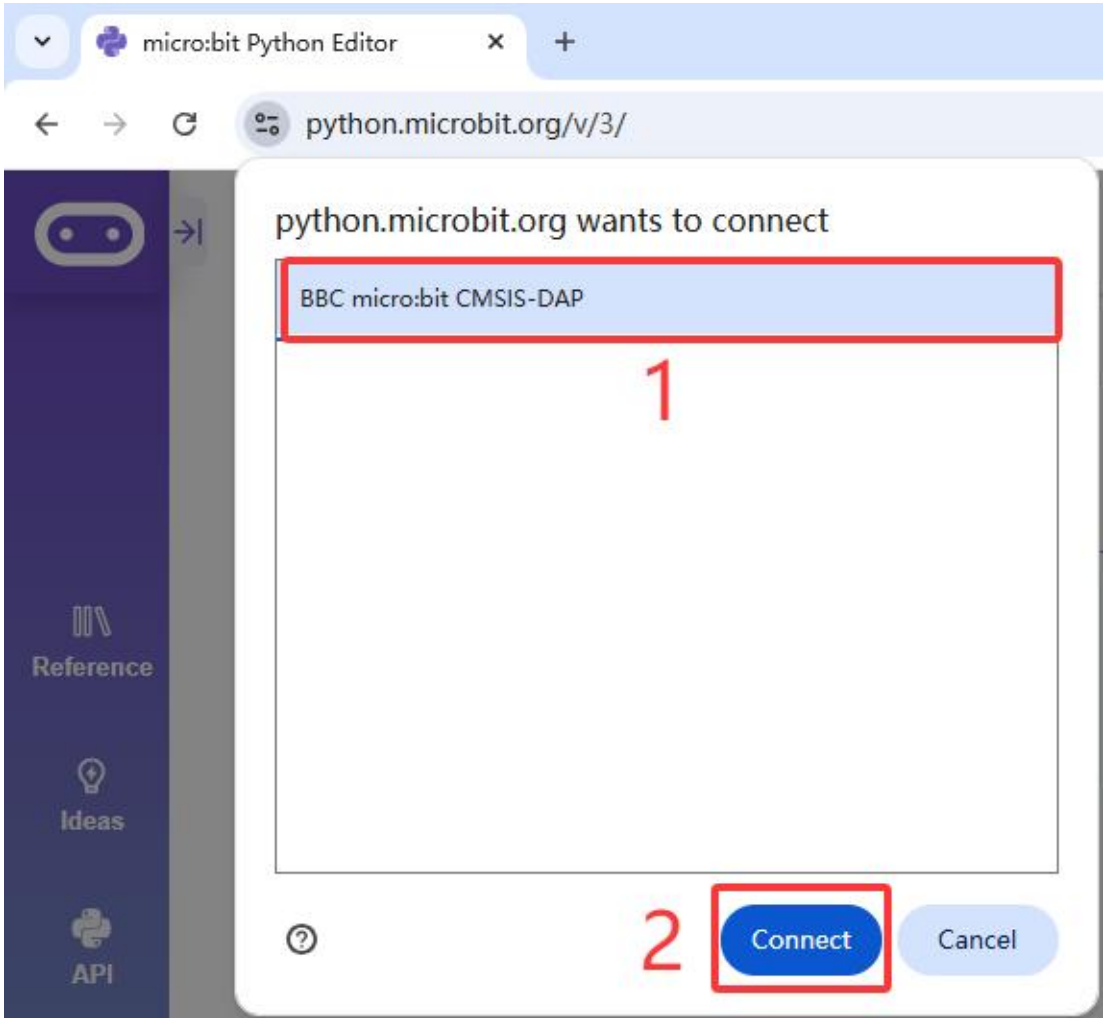


Click "**Next**"：

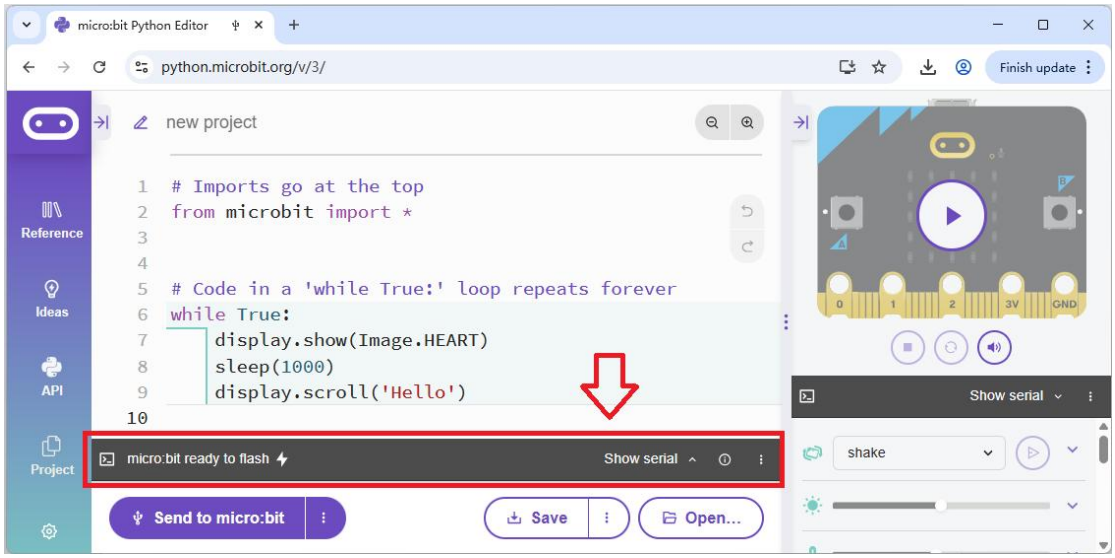




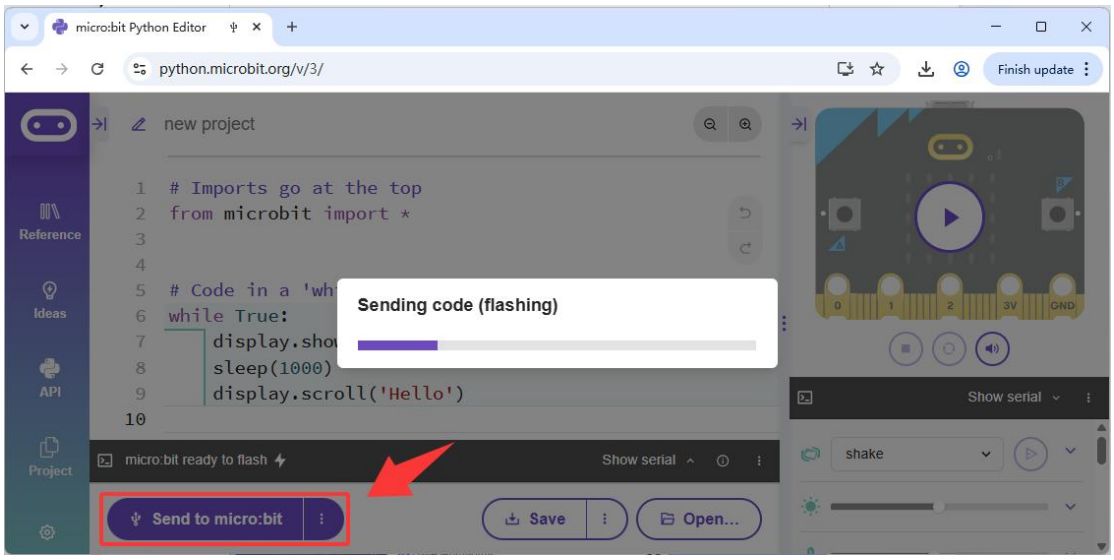
Choose your micro:bit,
then click "**Connect**".



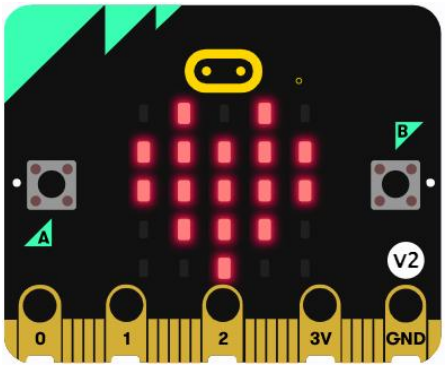
When the micro:bit ready to flash interface appears, it indicates that the micro:bit is successfully connected to the Python Editor.



Now you can click "Send to micro:bit" to upload the code from the Python Editor to the micro:bit:



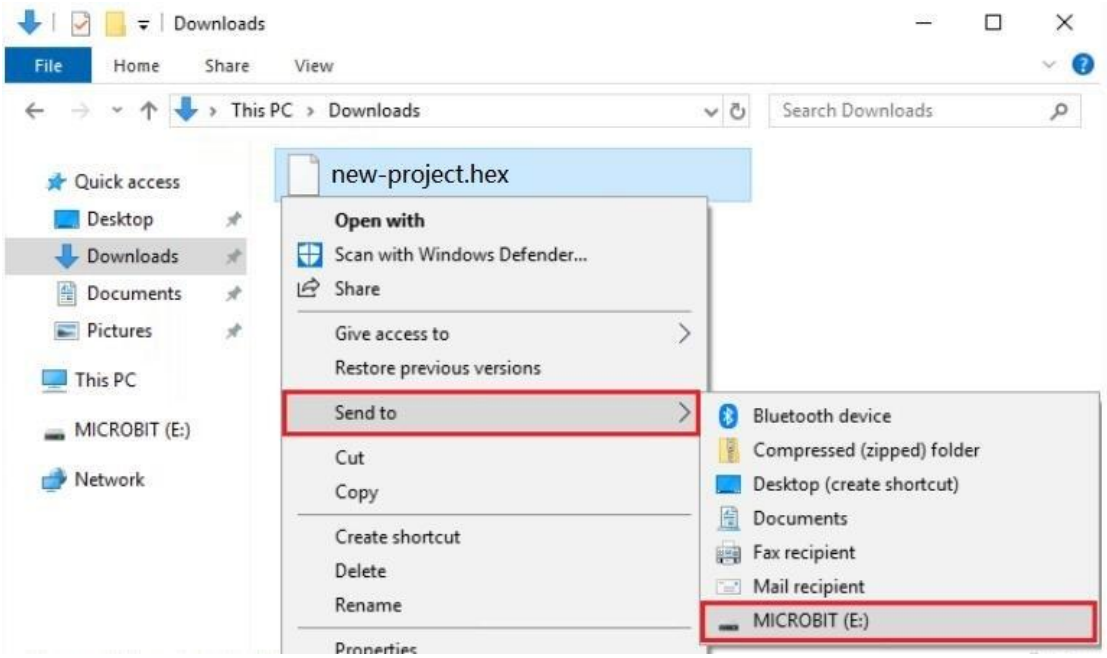
After the code is uploaded, the micro:bit's LED matrix will display a heart icon, followed by scrolling the text "Hello":



Two Alternative Methods for Uploading Code

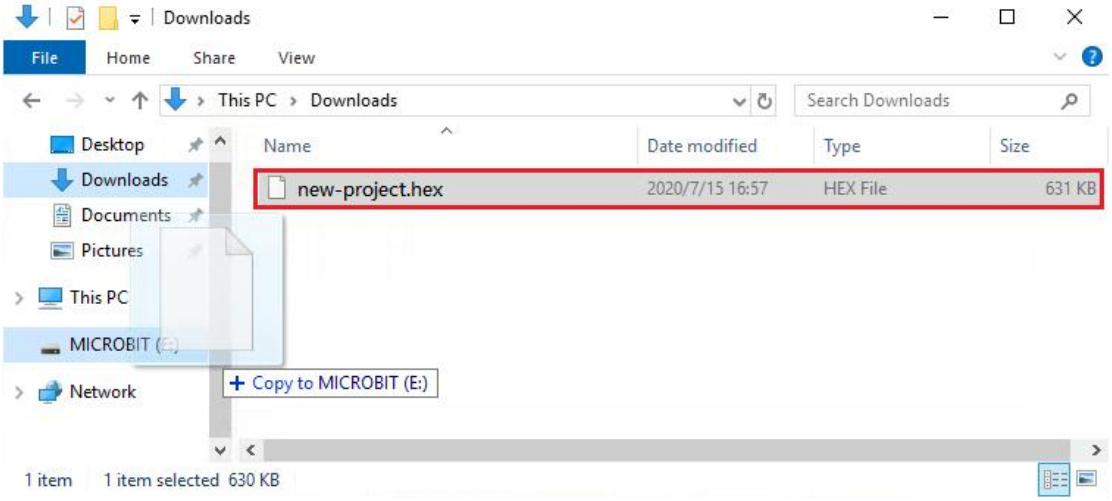
1. Send via Right-Click

Select the ".hex" file, right-click, and choose "Send to" the micro:bit drive:

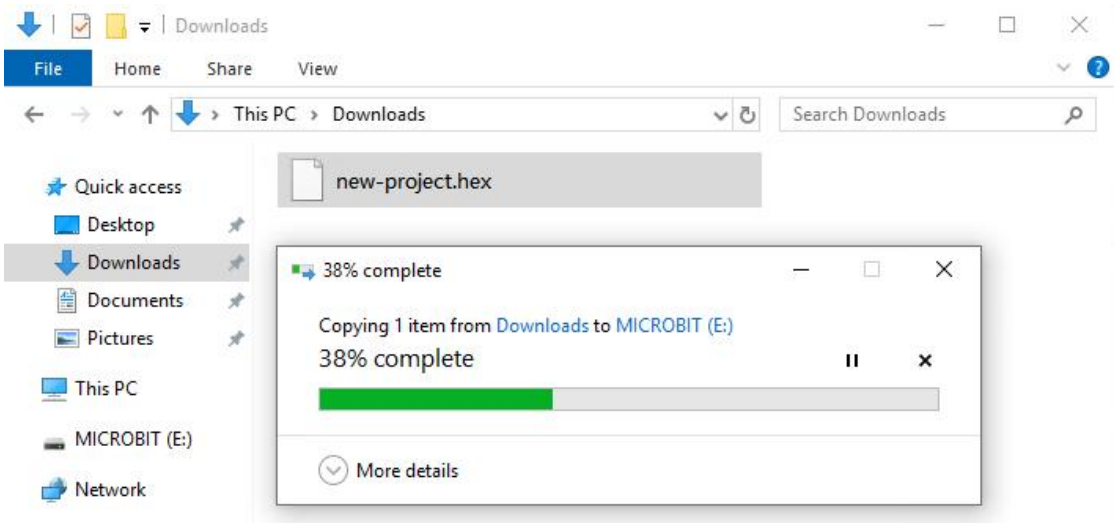


2. Drag and Drop

Alternatively, directly drag and drop the ".hex" file onto the micro:bit drive:



The following interface will appear, indicating that the code is being uploaded:



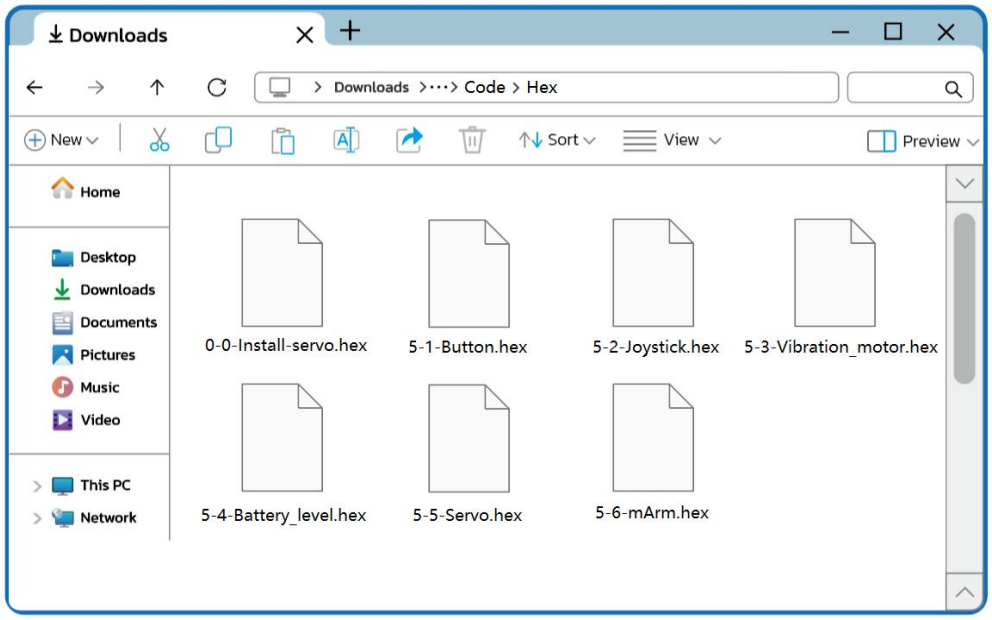
4.5 Learning Basic Syntax of the micro:bit Python Editor

The Micro:bit platform provides official reference documentation for the Python Editor.

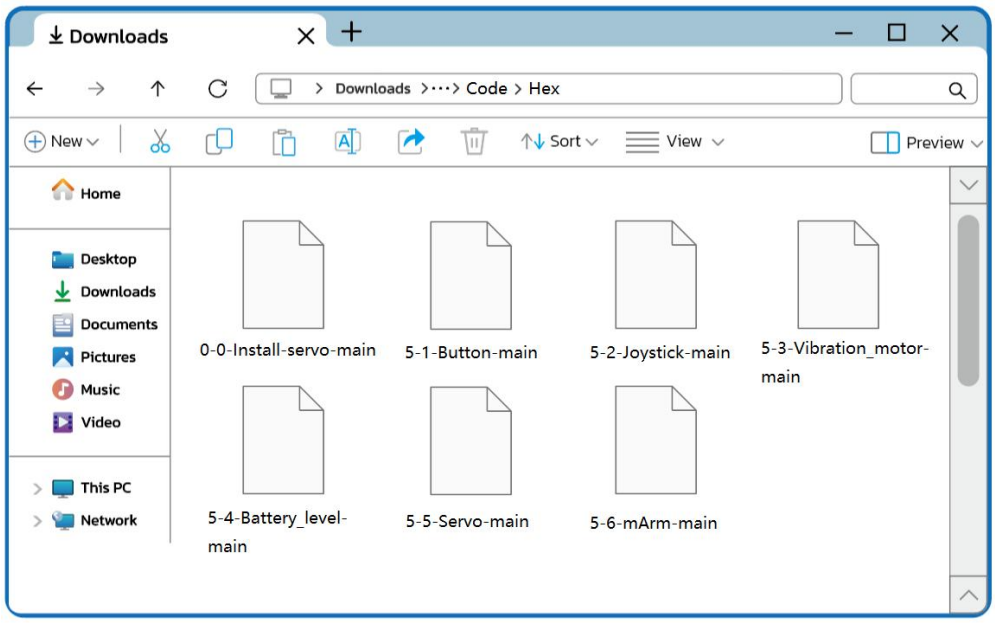
User Guide:
https://microbit.org/get-started/user-guide/python-editor/
MicroPython Documentation:
https://microbit-micropython.readthedocs.io/en/v2-docs/

5. Basic Example Projects

All basic example hex codes are stored in the "**Micropython-> Code-> Hex**" folder:



The basic example Python codes for the tutorial are saved in the "**Micropython > Code > Python**" folder:



Please read **Section 4.3: Import Files**. You can import the codes from the tutorial package into the editor.

Warning: Flashing a new program onto the micro:bit will overwrite any existing code. The previous program will be lost. To regain control of the robotic arm after flashing the new code, make sure to flash the final program provided in this chapter.

5.1 Button

Goal

Read the values of the joystick's A, B, C, and D buttons.

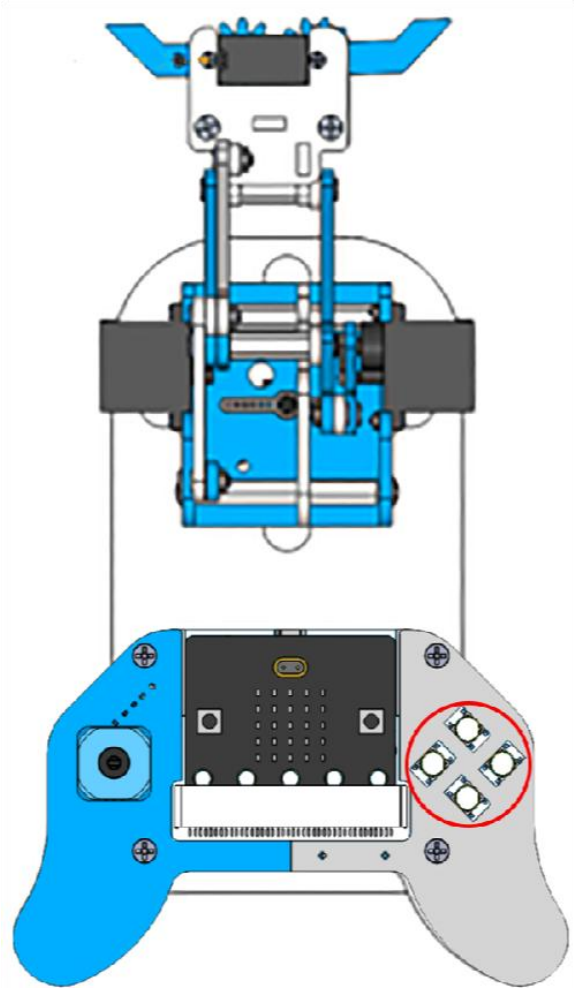
Code

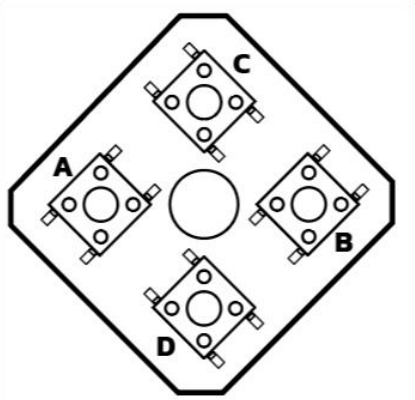
```
# Imports go at the top
from microbit import *

# Define the pins of the A, B, C, and D button
button_a = pin5
button_b = pin11
button_c = pin12
button_d = pin8

# I/O port with pull resistor
button_a.set_pull(button_a.PULL_UP)
button_b.set_pull(button_b.PULL_UP)
button_c.set_pull(button_c.PULL_UP)
button_d.set_pull(button_d.PULL_UP)

# Read button value function
def readButtonValue(button):
    # Return button value
    return button.read_digital()

# Code in a 'while True:' loop repeats forever
while True:
    # Determine whether button_a is pressed
    if readButtonValue(button_a) == 0:
        # If button_a is pressed, dot matrix displays character 'A'
        display.show('A')

    # Determine whether button_b is pressed
    if readButtonValue(button_b) == 0:
        # If button_b is pressed, dot matrix displays character 'B'
        display.show('B')

    # Determine whether button_c is pressed
    if readButtonValue(button_c) == 0:
        # If button_c is pressed, dot matrix displays character 'C'
        display.show('C')
```

```
    # Determine whether button_d is pressed
    if readButtonValue(button_d) == 0:
        # If button_d is pressed, dot matrix displays character 'D'
        display.show('D')

    # delay 100 millisecond
    sleep(100)
```

Result

After powering on, the micro:bit matrix display shows a heart shape. When button A, B, C, or D is pressed respectively, the Micro:bit's LED matrix displays 'A', 'B', 'C', or 'D'.





5.2 Joystick

Goal

Read the analog values of the joystick in real time and displays the corresponding directional letters on the micro:bit LED matrix based on the tilt direction of the joystick.

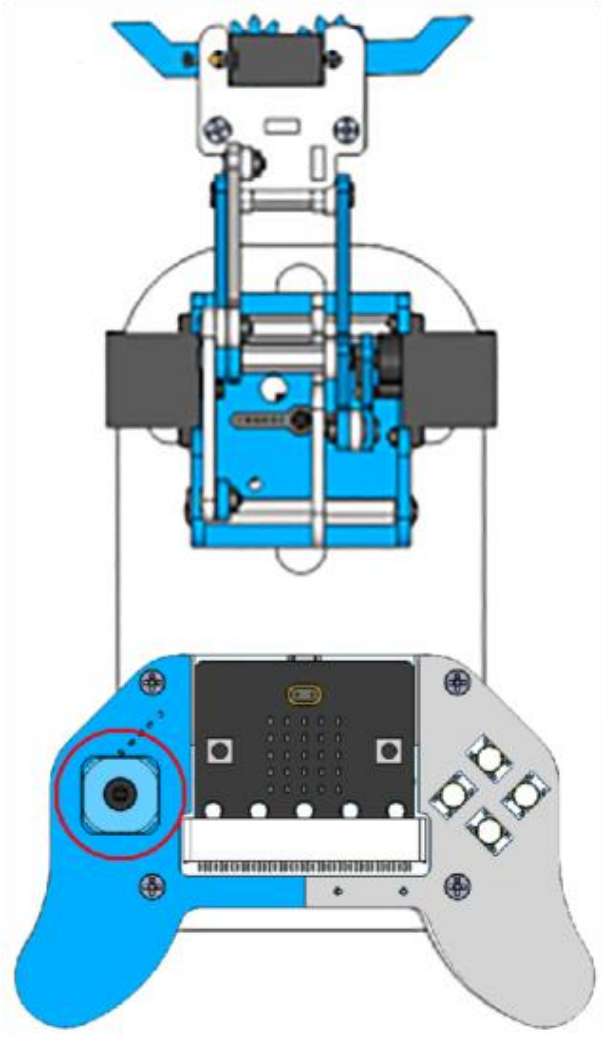
Code

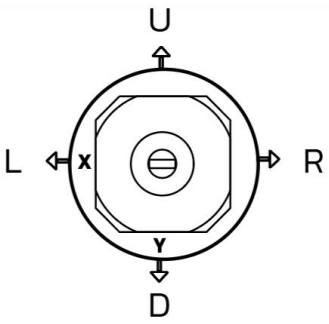
```
# Imports go at the top
from microbit import *

# Define the pins of the joystick
x_axis = pin1
y_axis = pin0

# Read the joystick value function
def readJoystickValue(axis):
    # Map the values from 0 to 1023 to -100 to +100,
    # and then return the mapped value
    return scale(axis.read_analog(), from_=(0, 1023), to=(-100, 100))

# Code in a 'while True:' loop repeats forever
while True:
    # Determine whether the value of the X-axis is greater than 80
    if readJoystickValue(x_axis) > 80:
        # Dot matrix displays character 'L'
        display.show('L')

    # Determine whether the value of the X-axis is  less than -80
    if readJoystickValue(x_axis) < -80:
        # Dot matrix displays character 'R'
        display.show('R')

    # Determine whether the value of the Y-axis is  greater than 80
    if readJoystickValue(y_axis) > 80:
        # Dot matrix displays character 'U'
        display.show('U')

    # Determine whether the value of the Y-axis is  less than -80
    if readJoystickValue(y_axis) < -80:
        # Dot matrix displays character 'D'
        display.show('D')
```

```
# delay 100 millisecond
sleep(100)
```

Result

When the micro:bit is powered on, its dot matrix displays a heart shape. When the joystick is pushed left, right, up, or down, the Micro:bit's LED matrix displays 'L', 'R', 'U', or 'D' respectively.





5.3 Vibration motor

Goal

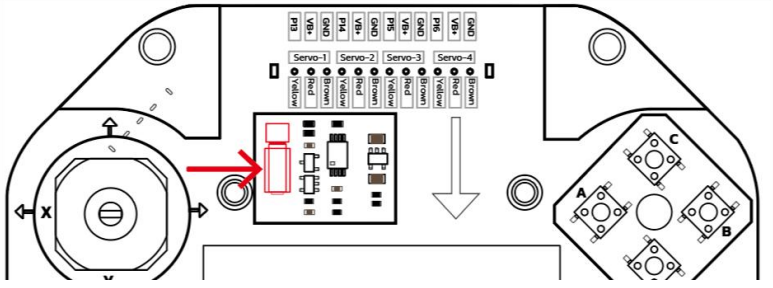
Activate the vibration motor of the joystick.

Code

```
# Imports go at the top
from microbit import *

# Define the vibration motor state variables
ON = 1
OFF = 0

# Control the vibration motor function
def setVibrationMotor(OnOrOff):
    # Determine whether the value of the variable OnOrOff is equal to 1
    if OnOrOff == ON:
        # Turn on the vibration motor
        pin2.write_digital(1)
    # Determine whether the value of the variable OnOrOff is equal to 0
    if OnOrOff == OFF:
        # Turn off the vibration motor
        pin2.write_digital(0)

# Code in a 'while True:' loop repeats forever
while True:
    # Turn on the vibration motor
    setVibrationMotor(ON)
    sleep(1000)  # delay 1000 millisecond

    # Turn off the vibration motor
    setVibrationMotor(OFF)
    sleep(1000)  # delay 1000 millisecond
```

Result

The vibration motor on the joystick vibrates for 1 second every second.



5.4 Battery voltage

Goal

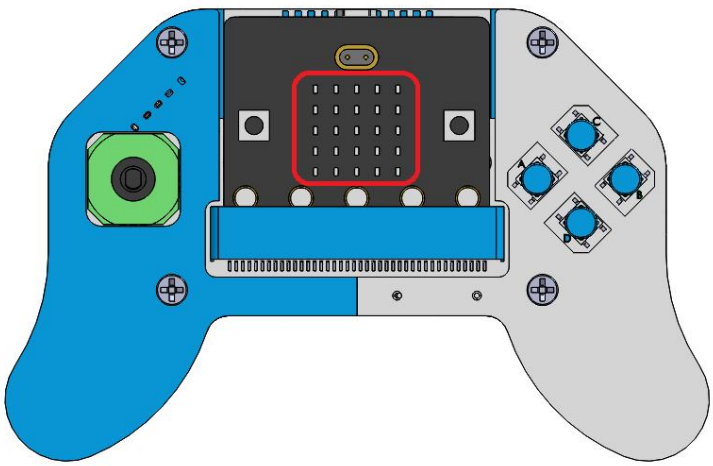
Read the battery level of the joystick.

Code

```
# Imports go at the top
from microbit import *

# Read the battery level
# 4 AA batteries
def readBatteryLevel():
    # Read the battery voltage value
    batLevel = pin2.read_analog()
    if batLevel > 310:   # 310=6V/6/0.0032226, 100%
            batLevel = 310
    if batLevel < 232:   # 232=4.5V/6/0.0032226, 0%
            batLevel = 232
    # Map 232-310 values to 0-100
    batLevel = scale(batLevel, from_=(232, 310), to=(0, 100))
    return batLevel  # return battery level

# Code in a 'while True:' loop repeats forever
while True:
    # Dot matrix display of battery level
    display.show(readBatteryLevel())
    sleep(1000)  # delay 1000 milliseconds
```

Result

The Micro:bit LED matrix displays the battery level value respectively, within a range of 0–100



Note

Reading the battery voltage and controlling the vibration motor share the same P2 pin on the Micro:bit. Do not switch too frequently between reading the battery voltage and controlling the vibration motor, as this may affect the accuracy of the battery voltage reading. It is recommended to read the voltage continuously multiple times after controlling the vibration motor and take the last value.

5.5 Servo

Goal

Control the rotation of the servo motor.

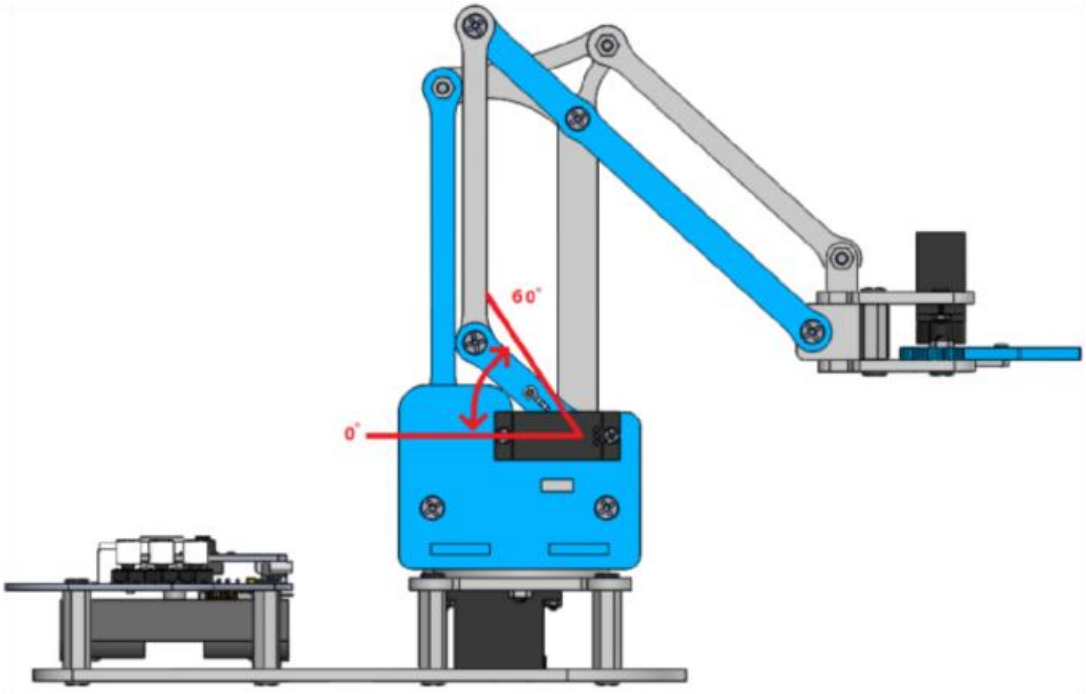
Code

```
# Imports go at the top
from microbit import *

# Servo class
class Servo:
    # The constructor of a class
    def __init__(self, pin):
        self.pin = pin
        # Servo parameters
        self.min_pulse = 500    # Pulse width at 0 degrees (microseconds)
        self.max_pulse = 2400   # Pulse width at 180 degrees (microseconds)
        self.period = 20000     # 20ms period (50Hz)
        self.pin.set_analog_period(20)  # Set period to 20ms

    # Set the servo Angle function
    def set_angle(self, angle):
        # Limit angle range
        angle = max(0, min(180, angle))

        # Calculate pulse width
        pulse_width = self.min_pulse + (angle / 180) * (self.max_pulse - self.min_pulse)

        # Convert to analog value (0-1023)
        analog_value = int((pulse_width / self.period) * 1023)

        # Set PWM
```

```
        self.pin.write_analog(analog_value)

    # Set the servo Angle function using pulse width
    def set_pulse(self, pulse_width):
        # Limit the maximum pulse value
        pulse_width = max(self.min_pulse, min(self.max_pulse, pulse_width))

        # Calculate pulse
        analog_value = int((pulse_width / self.period) * 1023)

         # Set PWM
        self.pin.write_analog(analog_value)

# Show happy face
display.show(Image.HAPPY)
# delay 100 milliseconds
sleep(100)

# Create 4 servo objects and define the pins to use
servo1 = Servo(pin13)  # servo1 uses pin13
servo2 = Servo(pin14)  # servo2 uses pin14
servo3 = Servo(pin15)  # servo3 uses pin15
servo4 = Servo(pin16)  # servo4 uses pin16

# Initialize the angles of the 4 servos
servo1.set_angle(90)  # Set the Angle of servo1
sleep(500)            # delay 500 milliseconds
servo2.set_angle(90)  # Set the Angle of servo2
sleep(500)            # delay 500 milliseconds
servo3.set_angle(60)  # Set the Angle of servo3
sleep(500)            # delay 500 milliseconds
servo4.set_angle(90)  # Set the Angle of servo4
sleep(500)            # delay 500 milliseconds

# Let servo 3 swing between 0 and 60 degrees.
while True:
    # The servo swings from 60 degrees to 0 degrees
    for i in range(60):
        # Set the Angle of servo3
        servo3.set_angle(60-i)
        # delay 20 milliseconds
        sleep(20)

    # The servo swings from 0 degrees to 60 degrees
```

```
    for i in range(60):
        # Set the Angle of servo3
        servo3.set_angle(i)
        # delay 20 milliseconds
        sleep(20)
```

Result



The swing arm of Servo-3 swings between 120 and 180 degrees, and the robot repeatedly performs the lifting and lowering motion of the robotic arm.

Note:

The joystick must be equipped with 4 AA batteries and the power switch must be turned on!

5.6 Control the Robot Arm

Goal

This section will provide a detailed explanation of the code used in "**3.2 Quick Start for Robotic Arm Control**" If you wish to use this robotic arm again after uploading other programs, please upload this code.

Code

```
# Imports go at the top
from microbit import *

# Define the pins of the A, B, C, and D button
button_a = pin5
button_b = pin11
button_c = pin12
button_d = pin8

# I/O port with pull resistor
button_a.set_pull(button_a.PULL_UP)
button_b.set_pull(button_b.PULL_UP)
button_c.set_pull(button_c.PULL_UP)
button_d.set_pull(button_d.PULL_UP)

# Read key value function
def readButtonValue(button):
    return button.read_digital()

# Define the pins of the joystick
x_axis = pin1
y_axis = pin0

# Read the joystick value function
def readJoystickValue(axis):
    # Map 0-1023 values to -100 - +100
    return scale(axis.read_analog(), from_=(0, 1023), to=(-100, 100))

# Define the vibration motor state variables
ON = 1
OFF = 0
# Control the vibration motor function
def setVibrationMotor(OnOrOff):
    if OnOrOff == ON:    # Turn on the vibration motor
```

```
        pin2.write_digital(1)
    if OnOrOff == OFF:   # Turn off the vibration motor
        pin2.write_digital(0)

# Read the battery level.
# 4 AA batteries
def readBatteryLevel():
    batLevel = pin2.read_analog()
    if batLevel > 310:   # 310=6V/6/0.0032226, 100%
            batLevel = 310
    if batLevel < 232:   # 232=4.5V/6/0.0032226, 0%
            batLevel = 232
    # Map 232-310 values to 0-100
    batLevel = scale(batLevel, from_=(232, 310), to=(0, 100))
    return batLevel

# Servo class
class Servo:
    def __init__(self, pin):
        self.pin = pin
        # Servo parameters
        self.min_pulse = 500    # Pulse width at 0 degrees (microseconds)
        self.max_pulse = 2400   # Pulse width at 180 degrees (microseconds)
        self.period = 20000     # 20ms period (50Hz)
        self.pin.set_analog_period(20)  # Set period to 20ms

    def set_angle(self, angle):
        # Limit angle range
        angle = max(0, min(180, angle))

        # Calculate pulse width
        pulse_width = self.min_pulse + (angle / 180) * (self.max_pulse - self.min_pulse)

        # Convert to analog value (0-1023)
        analog_value = int((pulse_width / self.period) * 1023)

        # Set PWM
        self.pin.write_analog(analog_value)

    def set_pulse(self, pulse_width):
        # Set pulse width directly (microseconds)
        pulse_width = max(self.min_pulse, min(self.max_pulse, pulse_width))
        analog_value = int((pulse_width / self.period) * 1023)
        self.pin.write_analog(analog_value)
```

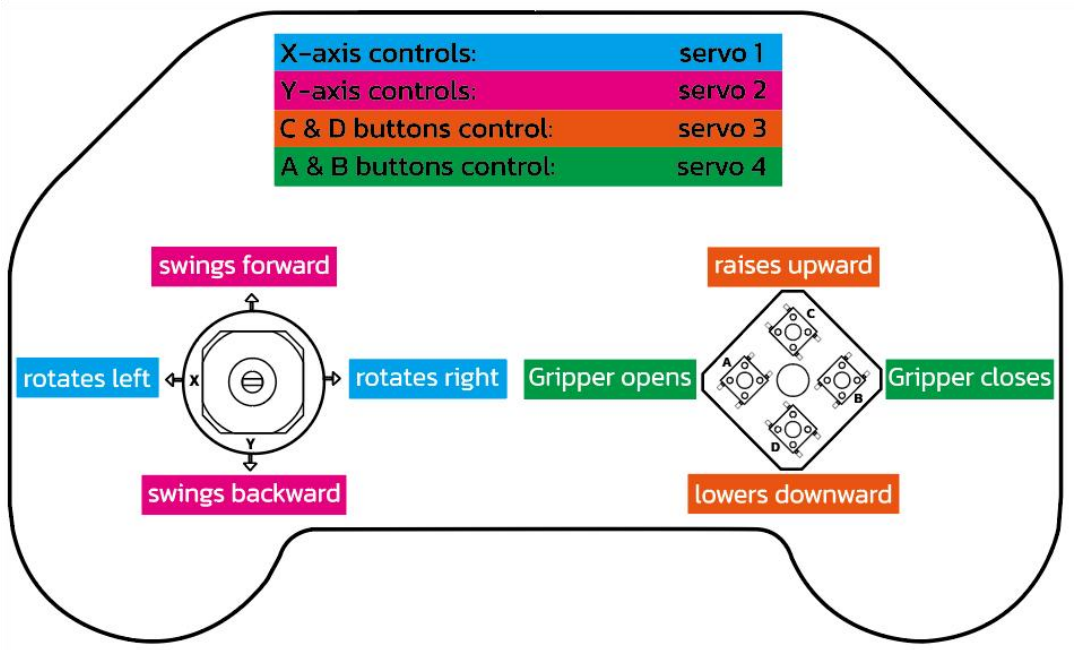
```
# Show happy face
display.show(Image.HAPPY)
sleep(100)  # delay 100 milliseconds

# Define the Angle variables of the 4 servos
servo1_degree = 90
servo2_degree = 90
servo3_degree = 60
servo4_degree = 90

# Create 4 servo objects and define the pins to use
servo1 = Servo(pin13)
servo2 = Servo(pin14)
servo3 = Servo(pin15)
servo4 = Servo(pin16)

# Initialize the angles of the 4 servos
servo1.set_angle(servo1_degree) # Set the Angle of servo1
sleep(500)   # delay 500 milliseconds
servo2.set_angle(servo2_degree)
sleep(500)
servo3.set_angle(servo3_degree)
sleep(500)
servo4.set_angle(servo4_degree)
sleep(500)

# Reads the current time when the program is running
last_time = running_time()
interval = 5000  # 5000ms
last_ticker = running_time()
ticker = 40000   # 40000ms

# An infinite loop statement.
while True:
    # Reads the current time when the program is running
    current_time = running_time()
    if current_time - last_time >= interval:
        # Display battery power value
        display.show(readBatteryLevel())
        last_time = current_time

    # Prevent the servo of the claws from overheating
    current_ticker = running_time()
```

```
    if current_ticker - last_ticker >= ticker:
        # Turn off the pulse of the servo
        servo4.set_pulse(0)

    # Read the value of the joystick X-axis and drive the Angle
    # of the servo1 according to the value.
    if readJoystickValue(x_axis) > 60 and servo1_degree < 180:
        servo1_degree = servo1_degree + 1   # Variable increments by 1
        servo1.set_angle(servo1_degree)     # Set the Angle of servo1
        sleep(15)
    if readJoystickValue(x_axis) < -60 and servo1_degree > 0:
        servo1_degree = servo1_degree - 1   # The variable decrements by 1
        servo1.set_angle(servo1_degree)
        sleep(15)

    # Read the value of the joystick Y-axis and drive the Angle
    # of the servo2 according to the value.
    if readJoystickValue(y_axis) > 60 and servo2_degree < 180:
        servo2_degree = servo2_degree + 1
        servo2.set_angle(servo2_degree)
        sleep(15)
    if readJoystickValue(y_axis) < -60 and servo2_degree > 0:
        servo2_degree = servo2_degree - 1
        servo2.set_angle(servo2_degree)
        sleep(15)

    # Read the value of the button_a and drive the Angle
    # of the servo4 according to the value.
    if readButtonValue(button_a) == 0 and servo4_degree > 5:
        servo4_degree = servo4_degree - 1
        servo4.set_angle(servo4_degree)
        sleep(15)

    # Read the value of the button_b and drive the Angle
    # of the servo4 according to the value.
    if readButtonValue(button_b) == 0 and servo4_degree < 180:
        servo4_degree = servo4_degree + 1
        servo4.set_angle(servo4_degree)
        sleep(15)

    # Read the value of the button_c and drive the Angle
    # of the servo3 according to the value.
    if readButtonValue(button_c) == 0 and servo3_degree > 0:
        servo3_degree = servo3_degree - 1
```

```
        servo3.set_angle(servo3_degree)
        sleep(15)

    # Read the value of the button_d and drive the Angle
    # of the servo3 according to the value.
    if readButtonValue(button_d) == 0 and servo3_degree < 120:
        servo3_degree = servo3_degree + 1
        servo3.set_angle(servo3_degree)
        sleep(15)

    # Executed when the micro:bit v2 logo is touched.
    # Attention! Not compatible with microbit v1.
    if pin_logo.is_touched():
        setVibrationMotor(ON)
        sleep(200)
        setVibrationMotor(OFF)
```

Result

- After powering on, the micro:bit's LED matrix will continuously cycle through displaying the battery level (0%–100%). If the battery level drops below 30%, please replace all batteries immediately.
- You can use the joystick and four buttons to control the robotic arm's movements.
- Pressing the touch icon on the Micro:bit will activate the vibration motor on the controller.



Attention! micro:bit v1 does not support logo touch function, if you want to use this program in micro:bit v1, you need to delete the following code:

```
# Executed when the micro:bit v2 logo is touched.
# Attention! Not compatible with microbit v1.
if pin_logo.is_touched():
    setVibrationMotor(ON)
    sleep(200)
    setVibrationMotor(OFF)
```

6.QA

6.1 micro:bit code upload failed

☛Verify that the USB cable supports data transfer (charging-only cables won't work).

☛Check the USB cable and port for physical damage or connection issues.

6.2 The robotic arm is not working.

☛Confirm that the 4×AA batteries are installed correctly and the power switch is in the ON position.

☛Check the battery voltage level (recommended minimum: 5.5V).

☛Verify that the servo initialization sequence has been completed according to the tutorial guide.

6.3 micro:bit driver is exhibiting abnormal behavior.

☛ Drives labeled “MAINTENANCE” (rather than “MICROBIT”) indicate an accidental entry into firmware recovery mode. Solution:

- Use the official tool to reflash the firmware:

  https://microbit.org/get-started/user-guide/firmware/

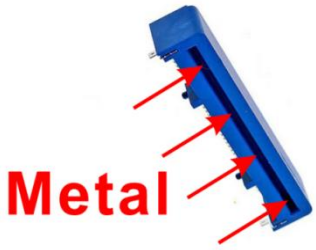
- Prevent recurrence by avoiding pressing the reset button during power cycling.

6.4 The button or joystick is unresponsive

☛Make sure the micro:bit is fully inserted into the mJoystick slot.

☛Make sure the micro:bit’ s edge connector is clean.

☛Make sure the mJoystick slot is clean.



6.5 Errors when using micro:bit v1

If you encounter an error when using the sample code “`5-6-mArm.hex`” from Chapter 5.6 on the micro:bit v1, this is because the code uses the statement “`pin_logo.is_touched()`”, which is available only on the micro:bit v2.

☛The micro:bit v1 lacks a logo touch function, which causes an error. Simply delete the “`pin_logo.is_touched()`” statement, regenerate the .hex file, and then upload it to the micro:bit v1 board.

6.6 Other Issues

☛Please check whether the assembly is correct.

☛Please check whether the batteries used meet the specifications.

7.Contact Us

If you can't find the solutions mentioned above, please contact our support team.

To help us assist you quickly, please have the following information ready:

Your order number.
Product model (e.g., robotic arm kit M1R0000) and software version (e.g., MicroPython v3.X.X).
The question or a detailed description of your issue.
The steps you have already tried.
Any screenshots, photos, or code snippets of related error messages.

Other queries?

We value your feedback and are always looking for ways to improve. Please feel free to contact us:

Tutorial errors and feedback: Help us improve the documentation.
Product Ideas and Suggestions: We'd love to hear your great ideas.
Partnerships and Collaboration: Interested in collaborating? Let's talk.
Discounts and promotions: Inquire about educational or bulk pricing.
Any other content: For all other non-technical questions.

Support channels

support@siyeenove.com

https://siyeenove.com